DEPARTMENT OF COMPUTER ENGINEERING

DevOps Laboratory • Mini-Project Report

Academic Year 2025–2026

A MINI-PROJECT REPORT ON

DEVOPS MINI-PROJECT REPORT

GetDocs — Citizen Certificate Request Portal

SUBMITTED BY:

Name: Tanishq Kulkarni

Roll No: 24102A2007

Branch / Div: CMPN / A

PROJECT GUIDE:

Prof. Snehal Andhare

Department of Computer Engineering

Submitted in partial fulfillment of the requirements for the Degree of Bachelor of Engineering in Computer Engineering

Table of Tasks & Executive Index

Task No	Task Title & Scope	Reference Link
Task 01	**Problem Definition, Real-World Need & MVP Scope** Phase 1 • Document ID: CCRP-DOC-01	Section 01 →
Task 02	**Agile Methodology, Product Backlog & DevOps Workflow** Phase 2 • Document ID: CCRP-DOC-02	Section 02 →
Task 03	**Software Requirements Specification & System Architecture** Phase 3 • Document ID: CCRP-DOC-03	Section 03 →
Task 04	**Version Control, Feature Branching, Merge Conflict Resolution & Release Management** Phases 4 - 6 • Document ID: CCRP-DOC-04	Section 04 →
Task 05	**Continuous Integration with Jenkins & Tomcat Deployment** Phases 7 - 8 • Document ID: CCRP-DOC-05	Section 05 →
Task 06	**Automated Browser Testing with Selenium WebDriver & Selenium Grid** Phase 9 • Document ID: CCRP-DOC-06	Section 06 →
Task 07	**Continuous Testing & Automated Staging Quality Gate** Phase 10 • Document ID: CCRP-DOC-07	Section 07 →
Task 08	**Containerization with Docker, Private Registry & Continuous Deployment** Phases 11 - 12 • Document ID: CCRP-DOC-08	Section 08 →
Task 09	**Configuration Management with Ansible, Idempotency & Reliability** Phases 13 - 14 • Document ID: CCRP-DOC-09	Section 09 →
Task 10	**Final Delivery Report, Multi-Environment Release & Roadmap** Phase 15 • Document ID: CCRP-DOC-10	Section 10 →

TASK 01 • PHASE 1

Problem Definition, Real-World Need & MVP Scope

Reference Document: CCRP-DOC-01

Field	Value
Document ID	CCRP-DOC-01
Title	Problem Definition and MVP Scope
Version	1.0
Status	Approved (scope frozen)
Owner	Tanishq
Related phase	Phase 1 — Problem definition
Last updated	2026-10-11

Table of contents

1. Context and real-world need
2. Problem statement
3. Target users and personas
4. Pain points
5. Stakeholders
6. Objectives (SMART)
7. Constraints
8. Assumptions
9. Risks
10. Success criteria and KPIs
11. MVP scope freeze
12. Out of scope
13. Glossary
14. Sign-off

---

1. Context and real-world need

In many Indian local government offices (tehsil, municipal ward, gram panchayat and similar offices), certificates such as **birth, income, domicile and marriage** certificates are still requested and issued through a largely paper- and counter-based process:

- The citizen collects a printed form, fills it by hand and attaches photocopies of supporting documents.
- The form is submitted at a counter during office hours, often after queuing.
- Applications are recorded in manual registers; the physical file moves between desks.
- The citizen has no reliable way to know the status of the application other than returning to the office or phoning.
- Supporting documents can be misplaced, and incomplete applications are often discovered only on a later visit.
- The opacity of the process creates room for informal intermediaries ("agents") who charge for navigating it.
- Issued paper certificates are difficult for third parties (banks, schools, employers) to verify.

Several states operate e-district and service-delivery portals that address parts of this problem. This project does not claim to replace them; it builds a **focused, self-contained reference implementation** of the core request–review–issue workflow, engineered with a complete DevOps toolchain, as the vehicle for the course.

> *Note: All observations in this section are qualitative. Where numbers appear elsewhere in this document they are **targets or assumptions**, not measured statistics.*

2. Problem statement

> *Citizens who need a government certificate must make repeated in-person visits, cannot see where their application stands, and risk lost or incomplete paperwork; officers process applications from manual registers with no consolidated queue or audit trail. **GetDocs** provides a web portal where a citizen applies online with one validated supporting document, receives a tracking ID and follows the status, while an officer reviews a*

> *filtered queue, approves or rejects with a recorded reason, and the system issues a digitally verifiable certificate — with every status change audited and the whole application delivered through an automated, tested, containerised and reproducibly provisioned pipeline.*

3. Target users and personas

Persona 1 — Citizen (Applicant)

Attribute	Detail
Name (illustrative)	Priya Sharma, 29, private-sector employee, tier-2 city
Digital literacy	Comfortable with smartphone and web forms; not technical
Goals	Apply for an income certificate without taking leave; know exactly what to upload; see progress; obtain a certificate she can share
Frustrations	Multiple visits for one certificate; told about missing documents only after waiting; no status visibility; being approached by middlemen
Success looks like	Submit once in a few minutes, get a tracking ID, check status online, print a verifiable certificate

Persona 2 — Reviewer / Officer

Attribute	Detail
Name (illustrative)	R. Venkatesh, 46, revenue department officer
Digital literacy	Uses office software daily; prefers simple, predictable screens
Goals	See all pending applications in one queue; filter by type/status; approve or reject quickly with complete information; be protected by a clear record of decisions
Frustrations	Incomplete applications; physical files moving between desks; no single view of workload; disputes about who decided what and when
Success looks like	A queue with KPIs, one-click review actions, mandatory rejection reasons, automatic certificate numbering and an audit trail

Persona 3 — Administrator / DevOps engineer

Attribute	Detail
Name (illustrative)	Tanishq, platform owner (the project developer)
Goals	Every commit built and tested automatically; a broken user journey blocks release; versioned, reproducible deployments; servers provisioned from code; fast rollback
Frustrations	"Works on my machine"; manual, undocumented server setup; deployments that cannot be reversed; regressions discovered by users
Success looks like	Jenkins pipeline green from commit to running container, Selenium quality gate, Ansible-provisioned node, health check and rollback within minutes

4. Pain points

#	Pain point	Who	Current impact (qualitative)	How GetDocs addresses it	Task
P1	Repeated in-person visits	Citizen	Lost time and wages, travel cost	Online submission from any device	CCR-3
P2	Errors found late (invalid ID, missing fields)	Citizen, Officer	Rework, rejected applications	Strict server-side validation at submission	CCR-4
P3	Lost or unreadable supporting documents	Citizen, Officer	Re-submission, delays	Digital upload with type/size/magic-byte checks and SHA-256	CCR-5
P4	No proof of submission	Citizen	Disputes, no reference	Tracking ID and acknowledgement	CCR-6
P5	No status visibility	Citizen	Phone calls, visits, reliance on middlemen	Status page with timeline	CCR-7
P6	No consolidated workload view	Officer	Missed or aged applications	Review queue with filters and KPIs	CCR-9
P7	Inconsistent decisions, unexplained rejections	Citizen, Officer	Grievances, re-applications	State machine and mandatory rejection reason	CCR-10

#	Pain point	Who	Current impact (qualitative)	How GetDocs addresses it	Task
P8	Certificates hard to verify	Third parties	Fraud risk, manual verification	Verification page with SHA-256 hash	CCR-11
P9	No accountability record	Department, Auditor	Disputes about decisions	Immutable audit trail of status changes	CCR-12
P10	Manual, error-prone releases	DevOps	Downtime, regressions	Jenkins CI/CD, Selenium gate, Docker, Ansible, rollback	CCR-14, CCR-15

5. Stakeholders

Stakeholder	Role	Interest	Influence	Involvement
Citizens / applicants	Primary end users	Fast, transparent, low-effort certificate issuance	Medium	Usability feedback, acceptance testing of citizen journey
Reviewers / officers	Primary end users	Efficient queue, clear decisions, protection via audit trail	High	Workflow validation, acceptance of officer journey
Department head (issuing authority)	Business owner (hypothetical)	Throughput, accountability, compliance	High	Approves workflow rules and certificate format
Third-party verifiers (banks, schools)	Secondary users	Quick authenticity check	Low	Use the verification page
Project owner — Tanishq	Product Owner, Scrum Master, Developer, DevOps	Deliver MVP and complete DevOps lifecycle	High	Full-time, all ceremonies
Faculty guide	Academic supervisor	Learning outcomes, phase deliverables, evaluation	High	Phase reviews, sign-off, final evaluation
Data protection / IT security function	Compliance (hypothetical)	Lawful processing of personal data (DPDP Act 2023)	Medium	Review of data handling and access control

6. Objectives (SMART)

ID	Objective	Measure	Target	Deadline
O1	Deliver the citizen journey (apply, upload, tracking ID, status)	Stories CCR-3 to CCR-7 Done	5/5	End of Sprint 1 (2026-09-18)
O2	Deliver the officer journey and release v1.0.0	Stories CCR-8 to CCR-13 Done; Git tag `v1.0.0`	6/6 + tag	End of Sprint 2 (2026-09-25)
O3	Enforce quality with an automated E2E gate	Selenium suite covering all critical journeys runs in Jenkins and blocks on failure	100% critical journeys covered	End of Sprint 3 (2026-10-02)
O4	Automate build, containerisation and deployment	Commit triggers pipeline producing a versioned image in `localhost:5000` and a deployed container	Fully automated, no manual steps	End of Sprint 4 (2026-10-09)
O5	Provision infrastructure as code	Ansible provisions a clean Ubuntu 24.04 node; second run reports `changed=0`; rollback restores previous release	Idempotent, rollback < 2 min	End of Sprint 4 (2026-10-09)

7. Constraints

Type	Constraint
Technical	Java 21, Spring Boot 4.1, Thymeleaf server-rendered UI, Spring Data JPA, Spring Security, Flyway, H2 file DB (PostgreSQL mode), Maven wrapper; packaged as executable WAR deployable standalone and on Apache Tomcat 11.
Technical	Toolchain fixed by the course: Git/GitHub, Jenkins, Selenium WebDriver 4 + JUnit 5, Docker, Ansible. All tooling must run on a single developer workstation (local registry at `localhost:5000`).
Time	Five 1-week sprints, 2026-09-07 to 2026-10-09, mapped to 15 course phases.
Budget	Zero spend: open-source software only, no paid cloud services or SaaS.
Regulatory / data privacy	Awareness of the Digital Personal Data Protection Act, 2023 (DPDP Act): collect only data needed for the certificate (purpose limitation, data minimisation), restrict access to authenticated officers, keep an audit trail. National ID values are synthetic in all test and demo data; no real Aadhaar numbers are used. The MVP is not certified for production use with real personal data.

Type	Constraint
Security	Officer pages require authentication; CSRF protection on forms; uploaded files validated by content (magic bytes), not extension; no secrets committed to Git.
Team	Solo developer performing Product Owner, Scrum Master, Developer and DevOps roles; limited peer review capacity.

8. Assumptions

ID	Assumption
A1	A single issuing office with a single approval level is representative enough for the MVP.
A2	One supporting document (PDF/PNG/JPEG, up to 2 MB) per application is sufficient.
A3	Citizens do not need accounts; the tracking ID (plus registered details) is enough to look up status.
A4	Officer accounts are pre-seeded; self-registration of officers is not required.
A5	An H2 file database is acceptable for the MVP; the PostgreSQL-compatible mode keeps a future migration straightforward.
A6	Verhoeff checksum validation is an adequate proxy for national ID format validity; no external identity verification is performed.
A7	The developer workstation has enough resources (approx. 16 GB RAM) to run Jenkins, Selenium Grid, Docker registry and target containers concurrently.
A8	Target timings in Section 10 are measured on that workstation and are indicative, not production SLAs.

9. Risks

ID	Risk	Likelihood	Impact	Mitigation
R1	Spring Boot 4.1 / Tomcat 11 (Jakarta EE 11) incompatibilities with libraries or tutorials	Medium	High	Pin versions in the Maven BOM; spike in Sprint 0; prefer first-party Spring starters
R2	Flaky Selenium tests block the pipeline	High	Medium	Page Object Model, explicit waits, stable `data-testid` selectors, screenshots on failure, Grid with pinned browser versions
R3	Workstation resource exhaustion running the full toolchain	Medium	Medium	Resource limits on containers, stop idle services, headless browsers
R4	Scope creep (payments, notifications, e-KYC)	Medium	High	Scope frozen in Section 11; changes require a new backlog item and explicit re-planning
R5	Solo developer — single point of failure and limited review	High	Medium	Small PRs, PR template checklist, automated gates (tests, Selenium, build), documentation as part of DoD
R6	Mishandling of personal data	Low	High	Synthetic data only, minimal fields, access control, audit trail, no real IDs in repo
R7	Malicious file uploads	Medium	High	Allow-list by magic bytes, size limit, files stored outside web root, never executed or served inline as HTML
R8	Failed deployment leaves environment broken	Medium	High	Health check after deploy, versioned images and releases, Ansible rollback playbook
R9	Jenkins configuration drift or loss	Medium	Medium	Dockerised Jenkins with Configuration-as-Code and Jenkinsfile in the repository

10. Success criteria and KPIs

All values below are **targets** for the MVP, measured on the reference workstation.

Area	KPI	Target	How measured
Citizen experience	Time to complete a submission	< 3 minutes for a prepared applicant	Timed walkthrough; Selenium journey duration as proxy
Data quality	Server-side validation coverage of documented rules	100% of rules in CCR-4 / CCR-5	Unit and MockMvc tests mapped to each rule
Correctness	Invalid state transitions accepted	0	State-machine unit tests (CCR-10)
Accountability	Status changes with an audit entry	100%	Integration test asserting audit rows per transition

Area	KPI	Target	How measured
Quality gate	Selenium E2E suite as release gate	Green on every release; red blocks image push	Jenkins stage result
Build health	Unit + integration tests passing on `main`	100%	Jenkins test reports
Delivery speed	Commit to running container	< 15 minutes	Jenkins pipeline duration
Recovery	Rollback to previous release	< 2 minutes	Timed Ansible rollback run
Reproducibility	Ansible re-run on provisioned node	`changed=0`	Playbook recap
Availability check	Health endpoint after deploy	`UP` within 60 seconds	Actuator `/actuator/health` probe
Security	Critical vulnerabilities at build	0 known critical	Dependency/image scan in pipeline

11. MVP scope freeze

The following 15 tasks constitute the frozen MVP scope. Any addition requires a new backlog item and explicit re-planning by the Product Owner.

Key	Title	Description	Phase mapping
CCR-1	Repository & project skeleton	Maven wrapper, Spring Boot 4.1 / Java 21 project, WAR packaging, Flyway baseline, H2 file DB, README, `.gitignore`, PR template	3, 4
CCR-2	Design system & UI shell (grainy gradients, liquid glass, theming)	Layout, typography, grainy gradient backgrounds, liquid-glass components, light/dark theme toggle	3, 4
CCR-3	Certificate request submission form	Form for BIRTH, INCOME, DOMICILE, MARRIAGE with conditional fields (annual income, spouse name)	5
CCR-4	Applicant data validation	Bean Validation and custom validators: name, DOB/age, email, Indian mobile, 12-digit ID with Verhoeff, PIN, purpose, conditional fields	5
CCR-5	Supporting document upload & validation	One required file, PDF/PNG/JPEG by magic bytes, max 2 MB, SHA-256 recorded	5
CCR-6	Tracking ID generation & acknowledgement	Unique `CCR-2026-XXXXXX` ID, acknowledgement page	5
CCR-7	Status tracking with timeline	Lookup by tracking ID; timeline of SUBMITTED → UNDER_REVIEW → APPROVED/REJECTED	5
CCR-8	Officer authentication	Spring Security form login, seeded officer accounts, protected `/officer/**` routes	6
CCR-9	Officer review queue with filters & KPIs	Queue with filter by type/status/search, KPI cards per status	6
CCR-10	Review actions — start review / approve / reject (state machine)	Enforced transitions; rejection requires reason; approval issues certificate number	6
CCR-11	Certificate issuance & verification view	`SND-<TYPE-CODE>-2026-NNNNNN` number, printable certificate page with SHA-256 verification hash	6
CCR-12	Audit trail of status changes	Append-only record of every transition with actor, timestamp, from/to status, note	6
CCR-13	Health, info & JSON status API	Actuator health/info; JSON endpoint for application status	6, 14
CCR-14	Selenium E2E suite for critical journeys	Selenium 4 + JUnit 5, Page Object Model, screenshots on failure, failsafe, Grid in CI, quality gate	9, 10
CCR-15	Containerised CI/CD & provisioning (Jenkins → Docker → Ansible)	Jenkins CaC + Jenkinsfile, Tomcat deploy, multi-stage Docker image, registry push, Ansible provisioning, health check, rollback	7, 8, 11, 12, 13, 14, 15

Phases 1, 2 and 15 (problem definition, Agile planning, final release and project demonstration) are covered by documentation deliverables rather than individual stories.

12. Out of scope

The following are explicitly excluded from the MVP:

- Payments and fee collection
- SMS and email notifications
- DigiLocker or Aadhaar e-KYC integration

- Multi-language (vernacular) user interface
- Multi-level approval hierarchy
- PostgreSQL high-availability deployment
- Kubernetes or other container orchestration
- OCR or automated content analysis of uploaded documents

13. Glossary

Term	Meaning
CCRP	Citizen Certificate Request Portal (product name: GetDocs)
Tracking ID	Applicant-facing reference, format `CCR-2026-XXXXXX`
Certificate number	Issued on approval, format `SND-<TYPE-CODE>-2026-NNNNNN`
Verhoeff	Checksum algorithm used by 12-digit Aadhaar-style identifiers
DPDP Act 2023	India's Digital Personal Data Protection Act, 2023
Quality gate	Pipeline stage whose failure stops the release

14. Sign-off

Name	Role	Decision	Date
Tanishq	Project owner	Approved	2026-09-07
_____	Faculty guide	Approved	2026-09-07

TASK 02 • PHASE 2

Agile Methodology, Product Backlog & DevOps Workflow

Reference Document: CCRP-DOC-02

Field	Value
Document ID	CCRP-DOC-02
Title	Agile Plan, Product Backlog and DevOps Workflow
Version	1.0
Status	Approved
Owner	Tanishq
Related phase	Phase 2 — Agile planning and DevOps workflow
Last updated	2026-10-11

Table of contents

1. Approach
2. Team roles and ceremonies
3. Epics
4. User stories and acceptance criteria
5. Product backlog
6. Sprint plan
7. Velocity and burndown
8. Kanban board snapshot
9. Definition of Ready
10. Definition of Done
11. Branching and commit conventions
12. DevOps lifecycle
13. CI/CD pipeline
14. Ansible provisioning flow

---

1. Approach

- **Framework:** Scrum with a Kanban board for flow visibility (Scrumban).
- **Cadence:** 1-week sprints, Monday to Friday, five sprints from 2026-09-07 to 2026-10-09.
- **Estimation:** Story points on the Fibonacci scale (1, 2, 3, 5, 8, 13), relative to CCR-1 (3 points) as the reference story.
- **Prioritisation:** MoSCoW (Must / Should / Could / Won't).
- **Tracking:** GitHub Issues (one issue per story, key `CCR-<n>`) on a GitHub Projects board. The backlog is also exported as `agile/backlog.csv` for import into GitHub Projects or Jira.
- **Traceability:** Branch name, commit messages and PR title reference the issue key, e.g. `feature/CCR-5-document-upload`, `feat(upload): verify magic bytes (CCR-5)`.

2. Team roles and ceremonies

This is a **solo project**. One person (Tanishq) performs all Scrum roles. To keep this honest and still useful, each role is time-boxed and has an explicit output.

Role	Held by	Responsibilities in practice
Product Owner	Tanishq	Owns and orders the backlog, writes acceptance criteria, accepts stories against the DoD
Scrum Master	Tanishq	Keeps the cadence, maintains WIP limits, records impediments and retrospective actions
Developer	Tanishq	Implements, tests, reviews own PRs against the PR checklist after a cooling-off period
DevOps engineer	Tanishq	Owns Jenkins, Docker, registry, Ansible and environments
Faculty guide	Faculty guide	Acts as external stakeholder at sprint reviews; provides phase feedback

Ceremony	When	Time-box	Output
Sprint planning	Monday, start of sprint	30 min	Sprint goal, committed stories, tasks on board
Daily check-in	Each working day	10 min (written)	Short log: done / next / blockers in the sprint issue
Backlog refinement	Wednesday	20 min	Upcoming stories meet the Definition of Ready
Sprint review	Friday	20 min	Demo of increment; evidence captured in `docs/evidence`
Retrospective	Friday	15 min	One to three improvement actions carried into next sprint

Limitations acknowledged: with no second developer, peer review is replaced by a self-review checklist plus automated gates (tests, Selenium quality gate, pipeline). Estimates are calibrated against one person's velocity only.

3. Epics

Epic	Name	Goal	Stories
EP-1	Citizen services	A citizen can apply online with a validated document, receive a tracking ID and follow status	CCR-2, CCR-3, CCR-4, CCR-5, CCR-6, CCR-7
EP-2	Officer workflow	An officer can authenticate, triage a queue, decide applications, issue verifiable certificates with a full audit trail	CCR-8, CCR-9, CCR-10, CCR-11, CCR-12
EP-3	Platform & DevOps	The product is built, tested, packaged, deployed, provisioned and monitored automatically	CCR-1, CCR-13, CCR-14, CCR-15

4. User stories and acceptance criteria

CCR-1 — Repository & project skeleton

Epic: EP-3 | **Priority:** Must | **Points:** 3

As a **developer**, I want a version-controlled Spring Boot project skeleton with a reproducible build, so that every subsequent feature starts from a consistent, buildable base.

- **Given** a fresh clone, **when** I run `./mvnw verify`, **then** the build succeeds on Java 21 and produces an executable WAR.
- **Given** the application starts, **when** Flyway runs, **then** the baseline migration is applied to the H2 file database in PostgreSQL mode.
- **Given** the repository, **when** I open a pull request, **then** the PR template and `.gitignore` are present and no build artefacts or secrets are tracked.

CCR-2 — Design system & UI shell (grainy gradients, liquid glass, theming)

Epic: EP-1 | **Priority:** Should | **Points:** 5

As a **citizen**, I want a clean, consistent and readable interface, so that I can complete tasks without confusion on any device.

- **Given** any page, **when** it renders, **then** it uses the shared layout with header, navigation, grainy gradient background and liquid-glass surface components.
- **Given** I toggle the theme, **when** I reload the page, **then** my light/dark preference persists and system preference is used by default.
- **Given** a 360 px-wide viewport, **when** I view any page, **then** there is no horizontal scrolling and text contrast meets WCAG AA.

CCR-3 — Certificate request submission form

Epic: EP-1 | **Priority:** Must | **Points:** 5

As a **citizen**, I want to fill an online application for a BIRTH, INCOME, DOMICILE or MARRIAGE certificate, so that I do not need to visit the office to apply.

- **Given** I open the apply page, **when** I choose a certificate type, **then** the form shows the common fields and the type-specific fields (annual income for INCOME, spouse name for MARRIAGE).
- **Given** I submit a complete valid form, **when** the server accepts it, **then** an application is persisted with status SUBMITTED.
- **Given** I submit an invalid form, **when** the page re-renders, **then** my entered values are retained and errors are shown next to each field.

CCR-4 — Applicant data validation

Epic: EP-1 | **Priority:** Must | **Points:** 5

As an **officer**, I want every application validated on the server before it reaches me, so that I only review complete and well-formed data.

- **Given** a name outside 3–100 characters or containing characters other than letters, spaces, `.`, `'` or `-`, **when** I submit, **then** the name is rejected.

- **Given** a date of birth in the future or implying age above 120, or an invalid email, or a mobile not matching 10 digits starting 6–9, or a PIN not matching 6 digits without a leading 0, **when** I submit, **then** the corresponding field error is shown.
- **Given** a 12-digit national ID that fails the Verhoeff checksum, **when** I submit, **then** it is rejected with a checksum error.
- **Given** type INCOME without an annual income between 0 and 1,00,00,000, or type MARRIAGE without a spouse name, or a purpose outside 10–300 characters, **when** I submit, **then** the submission is rejected.

CCR-5 — Supporting document upload & validation

Epic: EP-1 | **Priority:** Must | **Points:** 5

As a **citizen**, I want to upload one supporting document with my application, so that the officer has the evidence needed without a physical visit.

- **Given** no file is attached, **when** I submit, **then** the submission is rejected with "document required".
- **Given** a file larger than 2 MB, **when** I submit, **then** it is rejected with a size error.
- **Given** a file whose content is not PDF, PNG or JPEG by magic bytes (for example a renamed `.exe` with `.pdf` extension), **when** I submit, **then** it is rejected.
- **Given** a valid file, **when** the application is saved, **then** the file is stored outside the web root and its SHA-256 hash, media type and size are recorded.

CCR-6 — Tracking ID generation & acknowledgement

Epic: EP-1 | **Priority:** Must | **Points:** 3

As a **citizen**, I want a unique tracking ID and an acknowledgement after submitting, so that I have proof of submission and a reference for follow-up.

- **Given** a successful submission, **when** the acknowledgement page loads, **then** it shows a tracking ID matching `CCR-2026-[A-Z0-9]{6}` (unambiguous base32-style alphabet), type, and submission time.
- **Given** many submissions, **when** IDs are generated, **then** each is unique (enforced by a database unique constraint with retry on collision).
- **Given** the acknowledgement page, **when** I refresh it, **then** the application is not submitted twice (Post/Redirect/Get).

CCR-7 — Status tracking with timeline

Epic: EP-1 | **Priority:** Must | **Points:** 3

As a **citizen**, I want to look up my application by tracking ID and see its status history, so that I know where it stands without calling or visiting.

- **Given** a valid tracking ID, **when** I search, **then** I see the current status and a timeline of each status change with timestamp.
- **Given** an unknown or malformed tracking ID, **when** I search, **then** I see a neutral "not found" message that does not reveal other applications.
- **Given** a REJECTED application, **when** I view it, **then** the rejection reason is shown; **given** an APPROVED one, **then** a link to the certificate is shown.

CCR-8 — Officer authentication

Epic: EP-2 | **Priority:** Must | **Points:** 3

As an **officer**, I want to log in securely, so that only authorised staff can view and decide applications.

- **Given** I am not logged in, **when** I request any `/officer/**` page, **then** I am redirected to the login page.
- **Given** valid seeded credentials, **when** I log in, **then** I land on the review queue; **given** invalid credentials, **then** a generic error is shown.
- **Given** I am logged in, **when** I log out, **then** my session is invalidated and protected pages are no longer accessible. Passwords are stored as BCrypt hashes and forms are CSRF-protected.

CCR-9 — Officer review queue with filters & KPIs

Epic: EP-2 | **Priority:** Must | **Points:** 5

As an **officer**, I want a queue of applications with filters and summary KPIs, so that I can prioritise my workload.

- **Given** I open the queue, **when** it loads, **then** KPI cards show counts for SUBMITTED, UNDER_REVIEW, APPROVED and REJECTED.
- **Given** filters for status, certificate type and a search on tracking ID or name, **when** I apply them, **then** only matching applications are listed, oldest first, with pagination.
- **Given** no applications match, **when** the list renders, **then** an empty state is shown.

CCR-10 — Review actions: start review / approve / reject (state machine)

Epic: EP-2 | **Priority:** Must | **Points:** 8

As an **officer**, I want to start review, approve or reject an application through enforced transitions, so that decisions are consistent and cannot skip steps.

- **Given** a SUBMITTED application, **when** I click "Start review", **then** its status becomes UNDER_REVIEW and I am recorded as the reviewer.

- **Given** an UNDER_REVIEW application, **when** I approve it, **then** its status becomes APPROVED and a certificate number `SND-<TYPE-CODE>-2026-NNNNNN` is issued.
- **Given** an UNDER_REVIEW application, **when** I reject without a reason, **then** the action is refused; **with** a reason, **then** status becomes REJECTED and the reason is stored.
- **Given** any illegal transition (for example SUBMITTED to APPROVED, or any change from APPROVED/REJECTED), **when** requested, **then** it is refused and the state is unchanged.

CCR-11 — Certificate issuance & verification view

Epic: EP-2 | **Priority:** Must | **Points:** 5

As a **citizen**, I want to view and print a digitally verifiable certificate, so that I can present it and third parties can confirm it is genuine.

- **Given** an APPROVED application, **when** I open its certificate page, **then** it shows certificate number, applicant details, type, issue date and a SHA-256 verification hash.
- **Given** the verification page, **when** a verifier enters the certificate number and hash, **then** the system confirms whether they match an issued certificate.
- **Given** an application that is not APPROVED, **when** its certificate URL is requested, **then** no certificate is shown; **given** the print action, **then** a print-optimised layout is used.

CCR-12 — Audit trail of status changes

Epic: EP-2 | **Priority:** Must | **Points:** 3

As a **department head**, I want every status change recorded with who, when and why, so that decisions are accountable and disputes can be resolved.

- **Given** any status transition (including initial SUBMITTED), **when** it occurs, **then** an audit entry is written in the same transaction with actor, timestamp, from-status, to-status and note.
- **Given** an application, **when** an officer opens its detail page, **then** the audit trail is shown in chronological order.
- **Given** the audit table, **when** the application code runs, **then** no update or delete operations exist for audit entries (append-only).

CCR-13 — Health, info & JSON status API

Epic: EP-3 | **Priority:** Should | **Points:** 2

As a **DevOps engineer**, I want health, info and a JSON status endpoint, so that deployments can be verified automatically and status can be queried by other systems.

- **Given** the running application, **when** I call `/actuator/health`, **then** it returns `UP` with HTTP 200; `/actuator/info` returns version and Git commit.
- **Given** a valid tracking ID, **when** I call the JSON status API, **then** it returns tracking ID, type, status and last-updated time without personal data; unknown IDs return 404.
- **Given** non-public Actuator endpoints, **when** requested anonymously, **then** they are not exposed.

CCR-14 — Selenium E2E suite for critical journeys

Epic: EP-3 | **Priority:** Must | **Points:** 8

As a **DevOps engineer**, I want an automated browser test suite for the critical journeys, so that a broken user journey blocks the release.

- **Given** the suite, **when** it runs, **then** it covers: citizen submit and acknowledgement, validation errors, invalid document rejection, status lookup, officer login, start review, approve with certificate, reject with reason.
- **Given** a failing test, **when** it fails, **then** a screenshot and page source are saved and archived by Jenkins.
- **Given** `./mvnw verify -Pe2e`, **when** run locally or in Jenkins against Selenium Grid, **then** tests run via Maven Failsafe using the Page Object Model, and any failure fails the build.
- **Given** a deliberately introduced defect, **when** the pipeline runs, **then** the quality gate fails and passes again after the fix.

CCR-15 — Containerised CI/CD & provisioning (Jenkins to Docker to Ansible)

Epic: EP-3 | **Priority:** Must | **Points:** 13

As a **DevOps engineer**, I want an automated pipeline from commit to provisioned, health-checked deployment, so that releases are repeatable, fast and reversible.

- **Given** a push to the repository, **when** Jenkins polls SCM, **then** the Jenkinsfile pipeline builds, tests, packages the WAR, builds a multi-stage Docker image and runs the Selenium gate against an ephemeral staging container.
- **Given** the gate passes, **when** the pipeline continues, **then** a versioned image is pushed to `localhost:5000` and deployed to Tomcat 11 (environment chosen by parameter) and as a production container, followed by a health check.

- **Given** a clean Ubuntu 24.04 node, **when** the Ansible playbook runs, **then** the node is provisioned and the application is healthy; a second run reports `changed=0`.
- **Given** a faulty release, **when** the rollback playbook runs, **then** the previous release is restored and healthy within 2 minutes.

5. Product backlog

Ordered by delivery sequence. All items are in status **Done** as of 2026-10-11.

Key	Title	Epic	Type	Priority	Points	Sprint	Status
CCR-1	Repository & project skeleton	EP-3	Story	Must	3	Sprint 0	Done
CCR-2	Design system & UI shell (grainy gradients, liquid glass, theming)	EP-1	Story	Should	5	Sprint 0	Done
CCR-3	Certificate request submission form	EP-1	Story	Must	5	Sprint 1	Done
CCR-4	Applicant data validation	EP-1	Story	Must	5	Sprint 1	Done
CCR-5	Supporting document upload & validation	EP-1	Story	Must	5	Sprint 1	Done
CCR-6	Tracking ID generation & acknowledgement	EP-1	Story	Must	3	Sprint 1	Done
CCR-7	Status tracking with timeline	EP-1	Story	Must	3	Sprint 1	Done
CCR-8	Officer authentication	EP-2	Story	Must	3	Sprint 2	Done
CCR-9	Officer review queue with filters & KPIs	EP-2	Story	Must	5	Sprint 2	Done
CCR-10	Review actions — start review / approve / reject (state machine)	EP-2	Story	Must	8	Sprint 2	Done
CCR-11	Certificate issuance & verification view	EP-2	Story	Must	5	Sprint 2	Done
CCR-12	Audit trail of status changes	EP-2	Story	Must	3	Sprint 2	Done
CCR-13	Health, info & JSON status API	EP-3	Story	Should	2	Sprint 2	Done
CCR-14	Selenium E2E suite for critical journeys	EP-3	Story	Must	8	Sprint 3	Done
CCR-15	Containerised CI/CD & provisioning (Jenkins → Docker → Ansible)	EP-3	Story	Must	13	Sprint 4	Done
	Total				**76**		

Won't (this release): payments/fees, SMS/email notifications, DigiLocker/Aadhaar e-KYC, multi-language UI, multi-level approval, PostgreSQL HA, Kubernetes, OCR of documents.

6. Sprint plan

Sprint	Name	Dates	Course phases	Sprint goal	Committed stories	Points	Capacity (focus hours)
0	Foundation	2026-09-07 – 2026-09-11	1–4	Problem defined, plan agreed, buildable skeleton and UI shell on GitHub	CCR-1, CCR-2	8	20 h (includes docs for phases 1–3)
1	Citizen journey	2026-09-14 – 2026-09-18	5	A citizen can apply with a validated document, get a tracking ID and check status	CCR-3, CCR-4, CCR-5, CCR-6, CCR-7	21	25 h
2	Officer journey & release v1.0.0	2026-09-21 – 2026-09-25	6	An officer can review and decide applications; MVP merged to `main` and tagged `v1.0.0`	CCR-8, CCR-9, CCR-10, CCR-11, CCR-12, CCR-13	26	25 h
3	CI & quality gate	2026-09-28 – 2026-10-02	7–10	Every commit is built and tested by Jenkins, deployed to Tomcat, and gated by Selenium	CCR-14 + enabler tasks (below)	8	25 h
4	Containers & config mgmt	2026-10-05 – 2026-10-09	11–15	Versioned images in the registry, Ansible-provisioned node with rollback; final release	CCR-15	13	25 h

Sprint 3 enabler tasks (tracked as checklist items on CCR-14, not separately pointed because they are infrastructure set-up rather than user-visible value):

Task	Phase	Output
Install Jenkins in Docker with Configuration-as-Code; create first CI job	7	Jenkins container, `jenkins.yaml`, build history
Jenkinsfile pipeline-as-code with SCM polling; Tomcat 11 deploy with `TARGET_ENV` parameter	8	`Jenkinsfile`, Tomcat deployment evidence
Selenium suite design and local execution	9	Page objects, local run report
Selenium in Jenkins via Grid; deliberate defect, red build, fix, green build	10	Failed and recovered build evidence

Release milestones

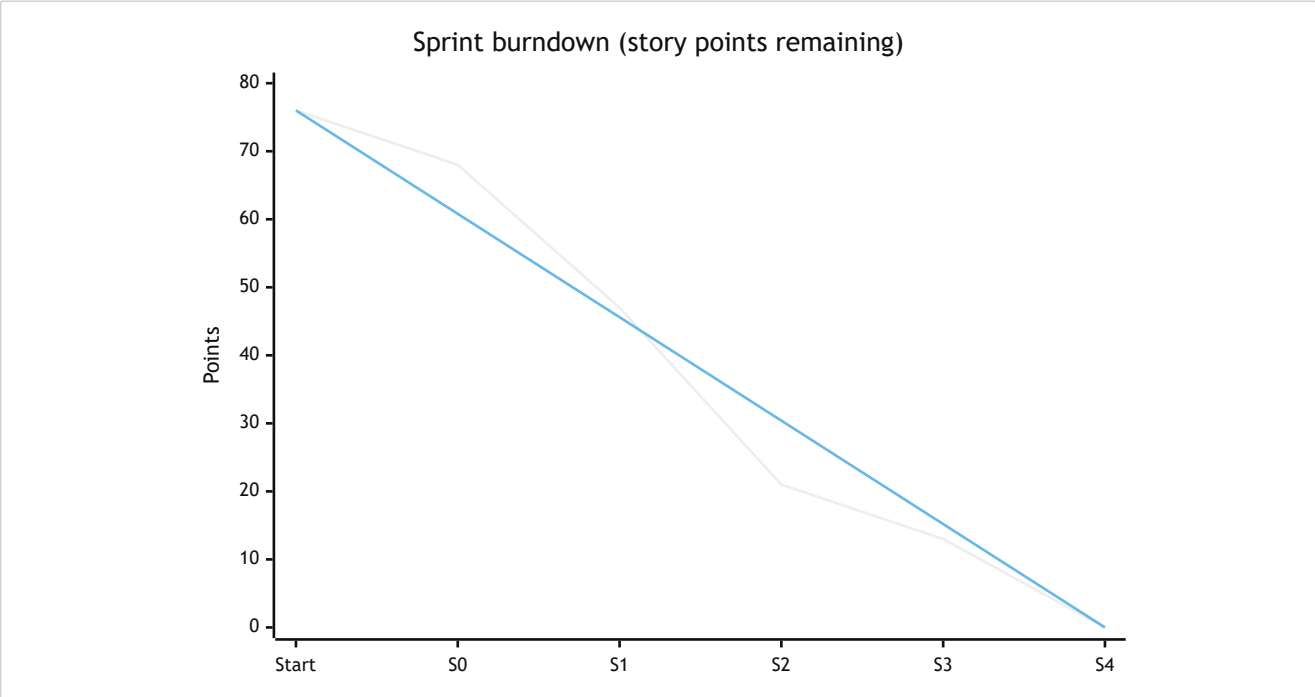
Version	Date	Content
v0.1.0	2026-09-11	Skeleton and UI shell
v1.0.0	2026-09-25	MVP: citizen and officer journeys (phase 6 tag)
v1.1.0	2026-10-02	CI pipeline and Selenium quality gate
v1.2.0	2026-10-09	Containerised CD, Ansible provisioning, final release

7. Velocity and burndown

Total scope: 76 story points.

Sprint	Committed (pts)	Completed (pts)	Remaining at sprint end (pts)	Ideal remaining (pts)	Notes
Start	—	—	76	76	Backlog frozen 2026-09-07
0	8	8	68	60.8	Low points by design; phases 1–3 documentation consumed capacity
1	21	21	47	45.6	Validation rules (Verhoeff, magic bytes) took longer than estimated; absorbed within sprint
2	26	26	21	30.4	Highest velocity; merge conflict exercise and `v1.0.0` tag completed
3	8	8	13	15.2	Enabler tasks (Jenkins, Tomcat) consumed most capacity
4	13	13	0	0	Final release

Average velocity: 15.2 points per sprint. Sprint 3 and 4 points understate effort because infrastructure enabler work is not pointed.



8. Kanban board snapshot

Final state as of 2026-10-11 (end of Sprint 4).

Backlog	Ready	In progress	In review	Done
—	—	—	—	CCR-1 Repository & project skeleton
				CCR-2 Design system & UI shell
				CCR-3 Certificate request submission form
				CCR-4 Applicant data validation
				CCR-5 Supporting document upload & validation
				CCR-6 Tracking ID generation & acknowledgement
				CCR-7 Status tracking with timeline
				CCR-8 Officer authentication
				CCR-9 Officer review queue with filters & KPIs
				CCR-10 Review actions (state machine)
				CCR-11 Certificate issuance & verification view
				CCR-12 Audit trail of status changes
				CCR-13 Health, info & JSON status API
				CCR-14 Selenium E2E suite for critical journeys
				CCR-15 Containerised CI/CD & provisioning

Column policies

Column	Entry rule	WIP limit
Backlog	Any idea or request with a CCR key	None
Ready	Meets the Definition of Ready	None
In progress	Branch created from `develop`; issue assigned	**3**
In review	PR open, CI green, awaiting self-review checklist	**2**
Done	Meets the Definition of Done; PR merged	None

WIP limits are deliberately low for a solo developer: if In review is full, finishing reviews takes priority over starting new work.

9. Definition of Ready

A story may move to **Ready** only when:

- [ ] It has a CCR key, title and "As a / I want / so that" statement.
- [ ] It has 2–4 testable Given/When/Then acceptance criteria.
- [ ] It is estimated in story points and assigned a MoSCoW priority.
- [ ] It belongs to an epic and is within the frozen MVP scope.
- [ ] Dependencies are identified and either done or scheduled earlier in the sprint.
- [ ] It is small enough to finish within one sprint (13 points or fewer).
- [ ] UI stories identify the pages affected; data stories identify validation rules and migrations.

10. Definition of Done

A story is **Done** only when all of the following hold:

Area	Criterion
Code	Implemented on a `feature/CCR-<n>-<slug>` branch following project conventions; no compiler warnings introduced; no secrets committed
Unit tests	New logic covered by JUnit 5 unit tests (validators, state machine, ID generation)

Area	Criterion
Integration tests	Web and persistence behaviour covered by Spring Boot / MockMvc integration tests
E2E tests	If user-facing, the journey is covered or extended in the Selenium suite
Acceptance	All acceptance criteria demonstrably met
CI	Jenkins pipeline green on the PR branch and after merge
Review	PR opened with template completed, self-review checklist ticked, linked to issue, merged into `develop`
Documentation	README, relevant docs and changelog updated
Deployment	Deployed to the staging environment (Tomcat or staging container) and health check `UP`
Quality	No open critical or high defects; no known critical vulnerabilities in dependencies or image

11. Branching and commit conventions

Branching model (Git Flow, simplified):

Branch	Purpose	Created from	Merges into
`main`	Released, tagged code only	—	—
`develop`	Integration branch	`main`	`release/*`
`feature/CCR-<n>-<slug>`	One story	`develop`	`develop` via PR
`fix/CCR-<n>-<slug>`	Defect fix during development	`develop`	`develop` via PR
`ci/<slug>`	Pipeline and infrastructure changes	`develop`	`develop` via PR
`docs/<slug>`	Documentation only	`develop`	`develop` via PR
`release/x.y.z`	Release stabilisation	`develop`	`main` (tagged `vx.y.z`) and back to `develop`
`hotfix/<slug>`	Urgent fix to production	`main`	`main` (patch tag) and `develop`

Commit messages follow Conventional Commits:

```
<type>(<scope>): <summary in imperative mood> (CCR-<n>)

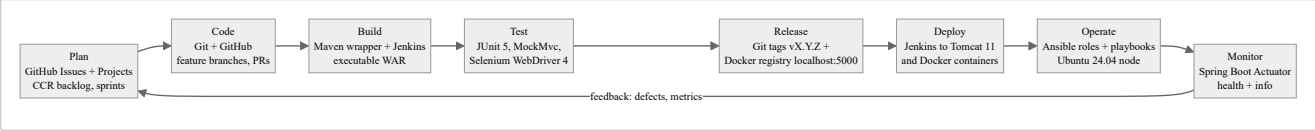
feat(upload): reject files whose magic bytes are not PDF/PNG/JPEG (CCR-5)
fix(validation): accept apostrophe in applicant name (CCR-4)
ci(jenkins): add Selenium quality gate stage (CCR-14)
docs: add agile plan
```

Allowed types: `feat`, `fix`, `test`, `refactor`, `docs`, `ci`, `build`, `chore`, `perf`, `style`.

Pull requests: one story per PR; title in Conventional Commit form with the CCR key; PR template sections for summary, linked issue, test evidence, screenshots for UI changes, and DoD checklist. `main` and `develop` are protected: merges only via PR with a green build.

Versioning: Semantic Versioning; Git tags `vX.Y.Z` on `main` are the source of Docker image tags.

12. DevOps lifecycle

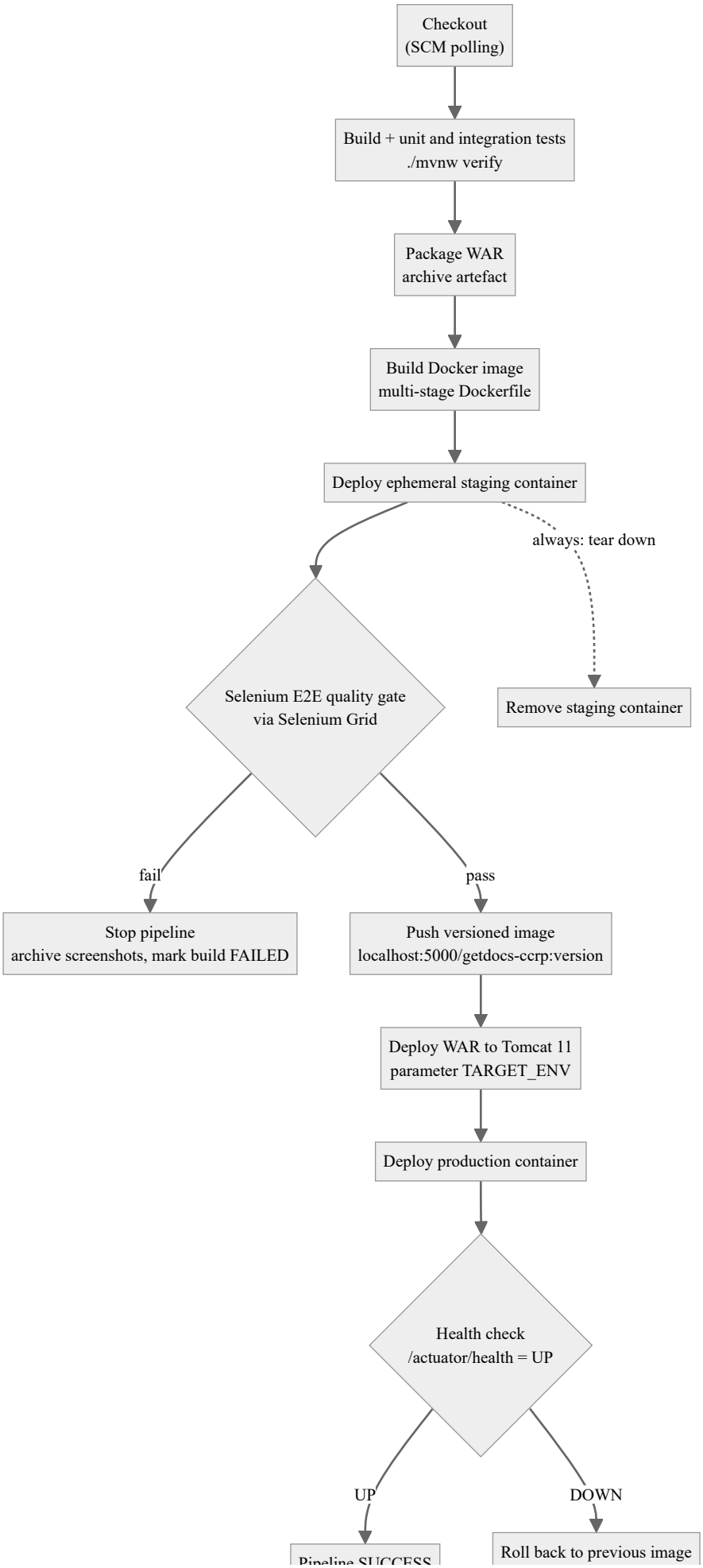


Stage	Tooling	Key artefact
Plan	GitHub Issues, GitHub Projects	Backlog, sprint board, `backlog.csv`
Code	Git, GitHub, PR template	Feature branches, reviewed PRs
Build	Maven wrapper, Jenkins	`ccrp-<version>.war`
Test	JUnit 5, MockMvc, Selenium 4 + Grid, Maven Failsafe	Surefire/Failsafe reports, failure screenshots

Stage	Tooling	Key artefact
Release	Git tags, Docker multi-stage build, local registry	`localhost:5000/getdocs-ccrp:<version>`
Deploy	Jenkins pipeline, Tomcat 11, Docker	Running WAR on Tomcat, running container
Operate	Ansible inventory, roles, playbooks	Provisioned node, rollback playbook
Monitor	Spring Boot Actuator	`/actuator/health`, `/actuator/info`

13. CI/CD pipeline

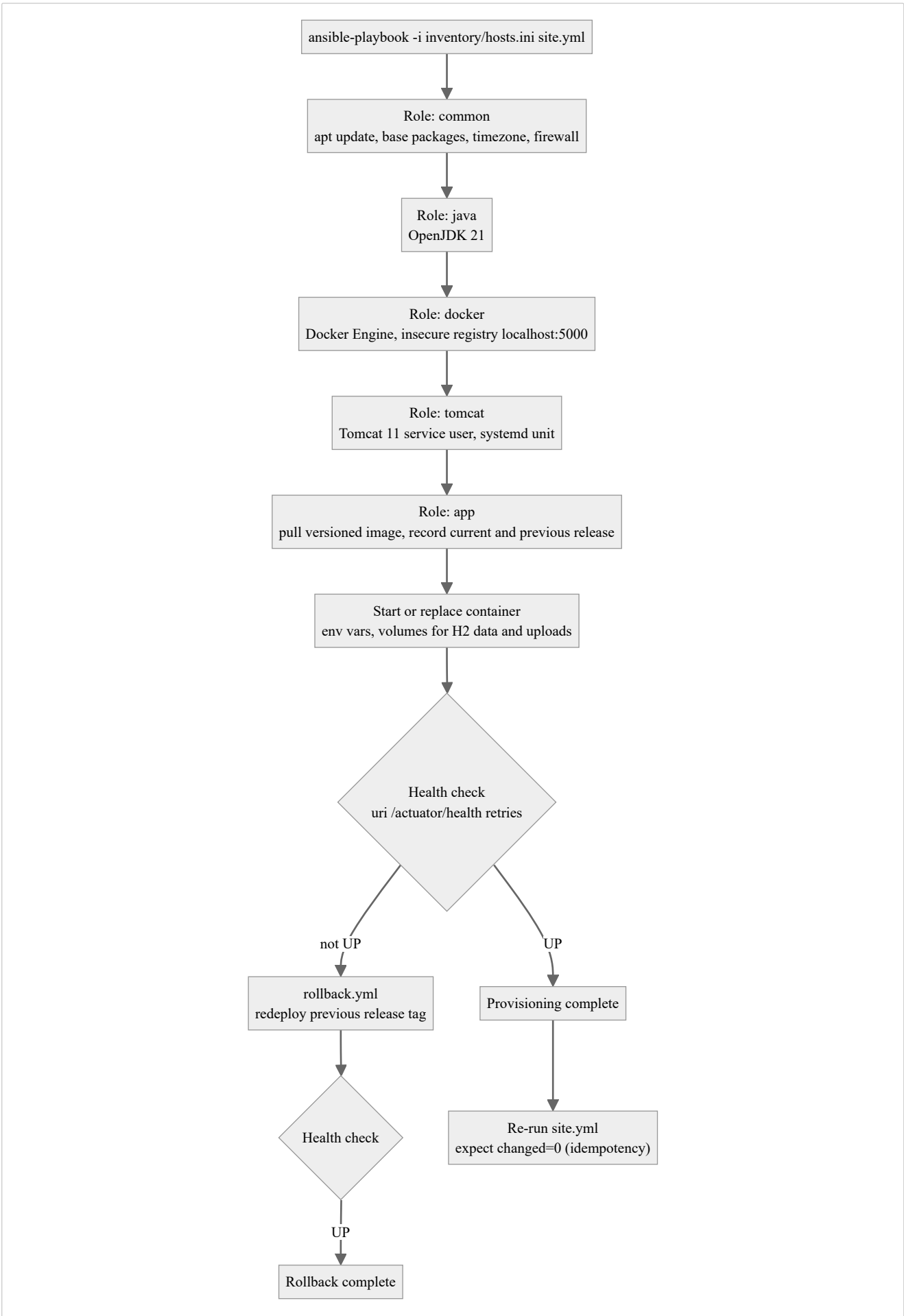
The Jenkinsfile implements the following stages. The Selenium stage is a hard quality gate: on failure the pipeline stops and nothing is pushed or deployed.


Checkout
(SCM polling)
Build + unit and integration tests
./mvnw verify
Package WAR
archive artefact
Build Docker image
multi-stage Dockerfile
Deploy ephemeral staging container
always: tear down
Selenium E2E quality gate
via Selenium Grid
Remove staging container
fail
pass
Stop pipeline
archive screenshots, mark build FAILED
Push versioned image
localhost:5000/getdocs-ccrp:version
Deploy WAR to Tomcat 11
parameter TARGET_ENV
Deploy production container
Health check
/actuator/health = UP
UP
DOWN
Roll back to previous image



Stage	Fails the build when	Evidence kept
Build & tests	Compilation error or any test failure	JUnit reports
Package WAR	Packaging error	WAR artefact
Build image	Docker build error	Image ID, tags
Selenium gate	Any E2E test fails	Failsafe report, screenshots, page source
Push image	Registry unreachable	Registry tag list
Tomcat deploy	Deployment not reachable	Tomcat manager response
Health check	Health not `UP` within 60 s	Health response, rollback log

14. Ansible provisioning flow


ansible-playbook -i inventory/hosts.ini site.yml
Role: common
apt update, base packages, timezone, firewall
Role: java
OpenJDK 21
Role: docker
Docker Engine, insecure registry localhost:5000
Role: tomcat
Tomcat 11 service user, systemd unit
Role: app
pull versioned image, record current and previous release
Start or replace container
env vars, volumes for H2 data and uploads
Health check
uri /actuator/health retries
not UP
UP
rollback.yml
redeploy previous release tag
Provisioning complete
Health check
Re-run site.yml
expect changed=0 (idempotency)
UP
Rollback complete

Step	Command	Expected result
Provision clean node	`ansible-playbook -i inventory/hosts.ini site.yml`	All roles applied, health `UP`
Idempotency check	Re-run the same command	Play recap `changed=0`, `failed=0`
Deploy a version	`ansible-playbook site.yml -e app_version=1.2.0`	New container running, previous version recorded
Roll back	`ansible-playbook rollback.yml`	Previous version running and healthy in under 2 minutes

TASK 03 • PHASE 3

Software Requirements Specification & System Architecture

Reference Document: CCRP-DOC-03

Field	Value
Document ID	CCRP-DOC-03
Version	1.0
Status	Approved
Owner	Tanishq
Last updated	2026-10-11

Contents

1. SRS summary
2. Use cases
3. Architecture
4. Data model
5. Status lifecycle
6. HTTP interface (routes and API)
7. Technology choices
8. Configuration reference
9. Local development setup

---

1. SRS summary

1.1 Purpose and scope

GetDocs lets a citizen request one of four certificates (Birth, Income, Domicile, Marriage), attach one supporting document and follow the request until an officer approves or rejects it. Scope is frozen to the 15 MVP items in 01-problem-definition.md.

1.2 Actors

Actor	Description	Authentication
Citizen	Member of the public applying for a certificate	None. Proves ownership of a request with *tracking ID* + *email*.
Officer	Government reviewer who decides requests	Username + password (form login, session cookie)
Operator	DevOps engineer running the platform	Infrastructure access (Jenkins, Docker, Ansible), `/actuator/health`
System	Validation and certificate issuance logic	—

1.3 Functional requirements

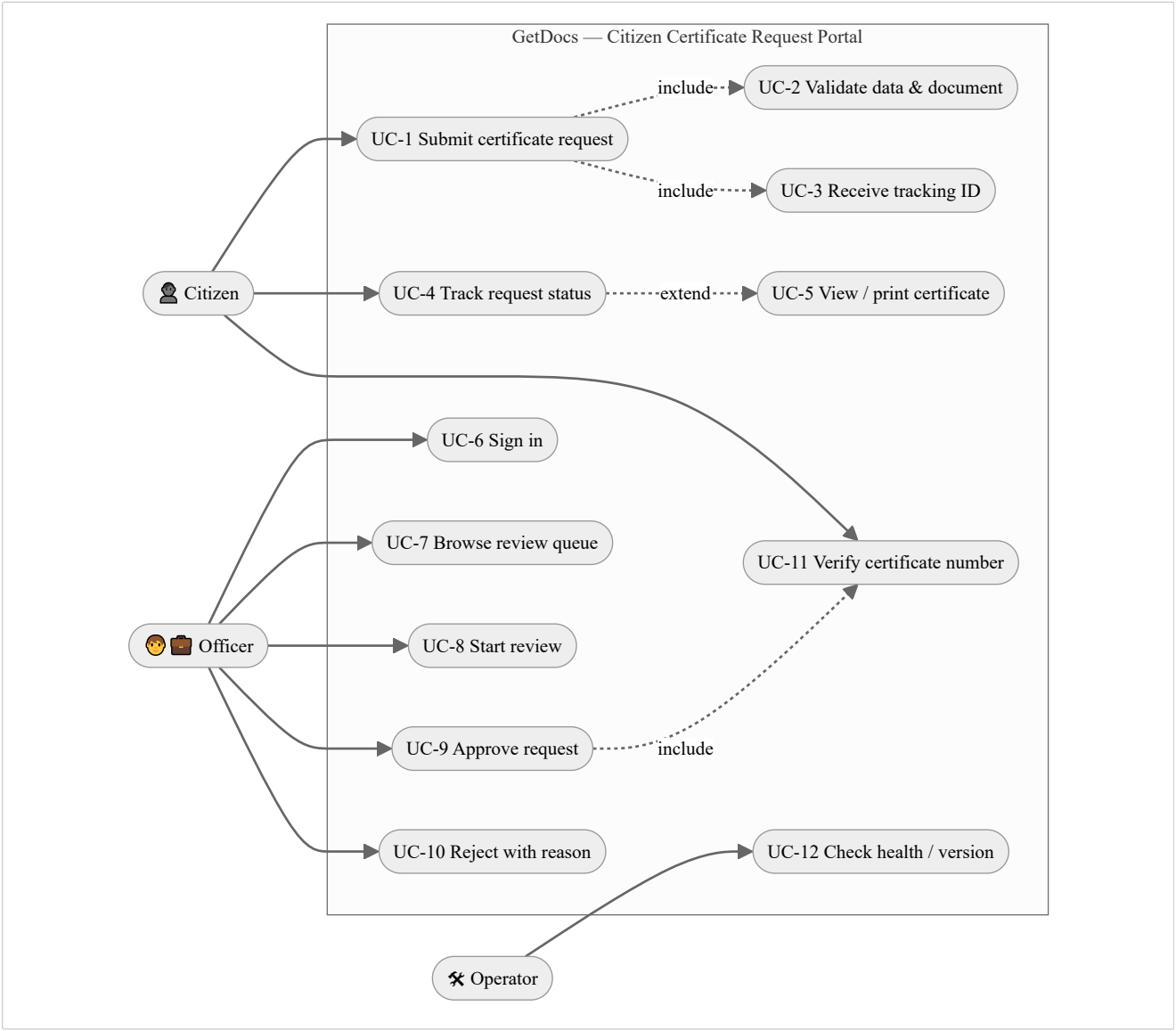
ID	Requirement	MVP item
FR-01	The citizen can choose a certificate type and submit a request with personal details, purpose and one document.	CCR-3
FR-02	All fields are validated on the server. Invalid input is returned to the form with field-level messages and the input preserved.	CCR-4
FR-03	The national ID (12 digits) is validated with the Verhoeff checksum. Only a masked form (`XXXX XXXX 1234`) and a SHA-256 fingerprint are stored.	CCR-4
FR-04	Type-specific rules: *Income* requires annual income; *Marriage* requires the spouse's name.	CCR-4
FR-05	The document must be PDF, PNG or JPEG, at most 2 MB, and its content must match its type (magic bytes). Its SHA-256 is stored.	CCR-5

ID	Requirement	MVP item
FR-06	On success the system generates a unique, non-sequential tracking ID `CCR-<year>-<6 chars>` and shows an acknowledgement page.	CCR-6
FR-07	The citizen can look up a request with tracking ID + email and see the current status, a progress stepper and the full history.	CCR-7
FR-08	Officers sign in. Every `/officer/**` page requires the `OFFICER` role.	CCR-8
FR-09	Officers see KPI counts per status and a queue filterable by status and searchable by tracking ID or name.	CCR-9
FR-10	Officers can *start review* (SUBMITTED → UNDER_REVIEW), *approve* or *reject* (UNDER_REVIEW → APPROVED/REJECTED). Rejection requires a reason of at least 10 characters. Illegal transitions are refused.	CCR-10
FR-11	Approval issues a certificate number `SND-<code>-<year>-<6 digits>` and a SHA-256 verification hash. The citizen can open a printable certificate after a successful lookup. Anyone can verify a certificate number through the API.	CCR-11
FR-12	Every status change records actor, from/to status, remarks and time.	CCR-12
FR-13	Liveness/readiness health, build/git info and a JSON status endpoint are exposed.	CCR-13

1.4 Non-functional requirements

ID	Category	Requirement
NFR-01	Performance	Page render < 300 ms (p95) on the reference container (1 vCPU, 512 MB heap) for ≤ 10k requests.
NFR-02	Security	CSRF protection on every form; BCrypt password hashing; session cookie `HttpOnly` + `SameSite=Lax`; officer pages unreachable anonymously; uploaded files never served from a public path.
NFR-03	Privacy	Data minimisation: the full national ID is never persisted; tracking lookups require the email on file; documents are visible only to officers.
NFR-04	Reliability	Graceful shutdown (20 s); Flyway-versioned schema; health probes used by Docker, Jenkins and Ansible.
NFR-05	Portability	One artefact (`.war`) runs with `java -jar`, on Tomcat 11, or in a container. All settings come from environment variables.
NFR-06	Testability	Unit + MockMvc integration tests on every build; Selenium journeys as a release gate.
NFR-07	Usability / a11y	Responsive down to 360 px; keyboard reachable; visible focus; respects `prefers-reduced-motion` and `prefers-color-scheme`.
NFR-08	Observability	Version, commit and environment are visible in the footer and at `/actuator/info`.

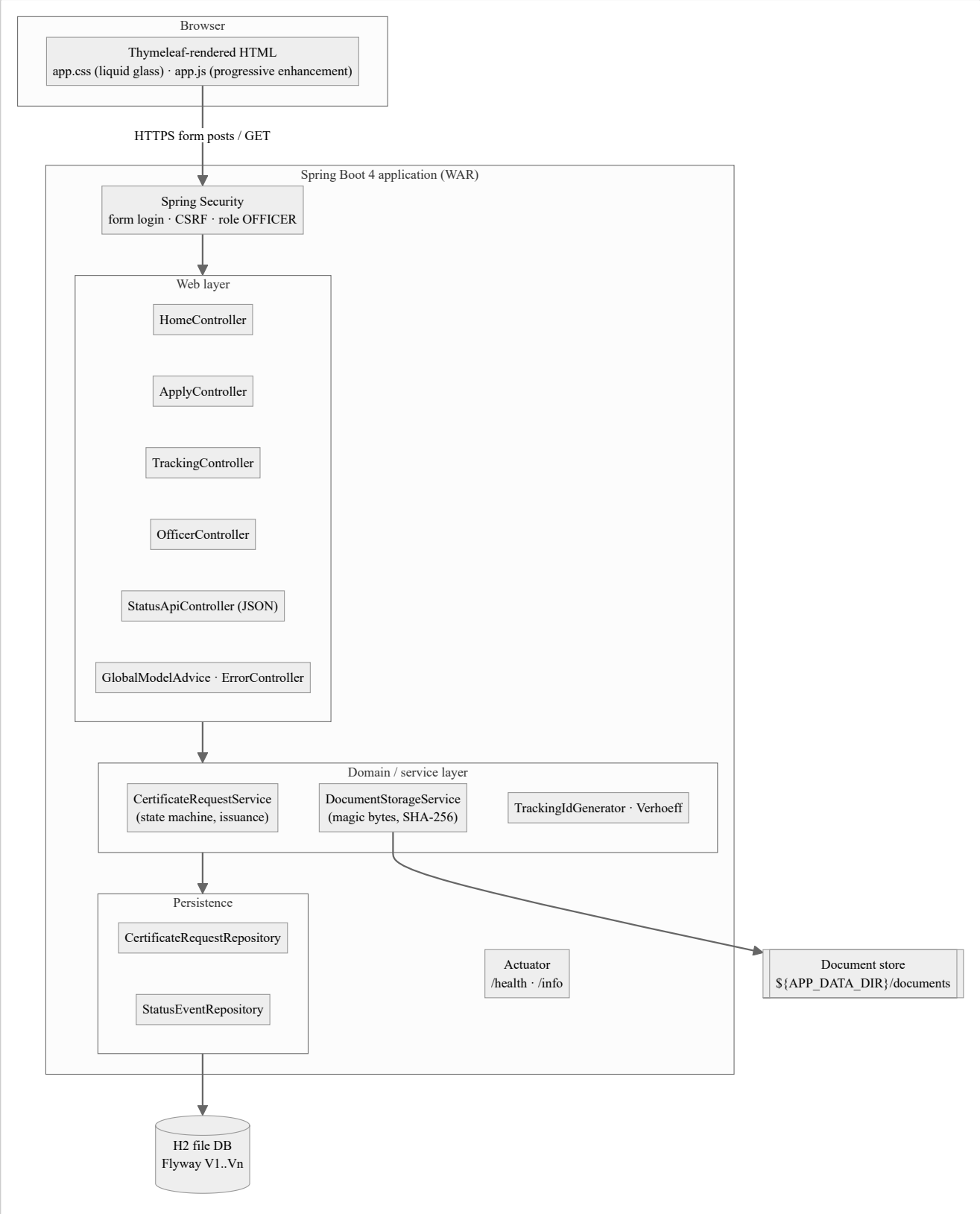
2. Use cases



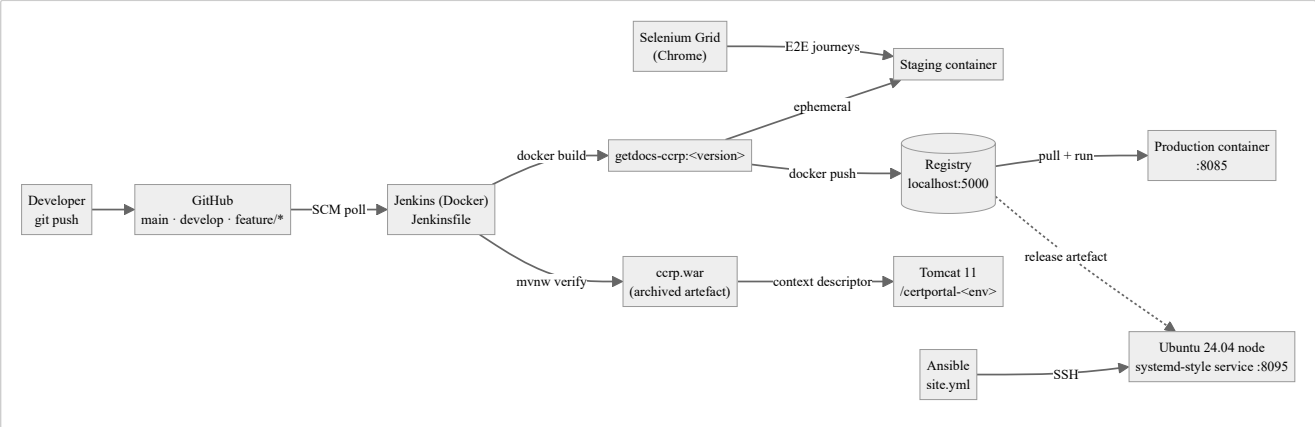
Use case	Primary flow	Alternate / error flows
UC-1/2/3 Submit	Choose type → fill details → attach document → submit → acknowledgement with tracking ID	Field errors re-render the form with messages and keep the input; a bad document shows a document error; a file over the transport limit gives a friendly 2 MB message
UC-4 Track	Enter tracking ID + email → status, stepper, timeline	Unknown ID or email mismatch → the same generic "not found" message (no enumeration)
UC-5 Certificate	From an *Approved* tracking result → open the certificate → print	Not approved, or not looked up in this session → 404
UC-8/9/10 Decide	Open the request → Start review → Approve / Reject (reason)	Illegal transition → error banner; missing reason → field error

3. Architecture

3.1 Logical view (layers)



3.2 Deployment view



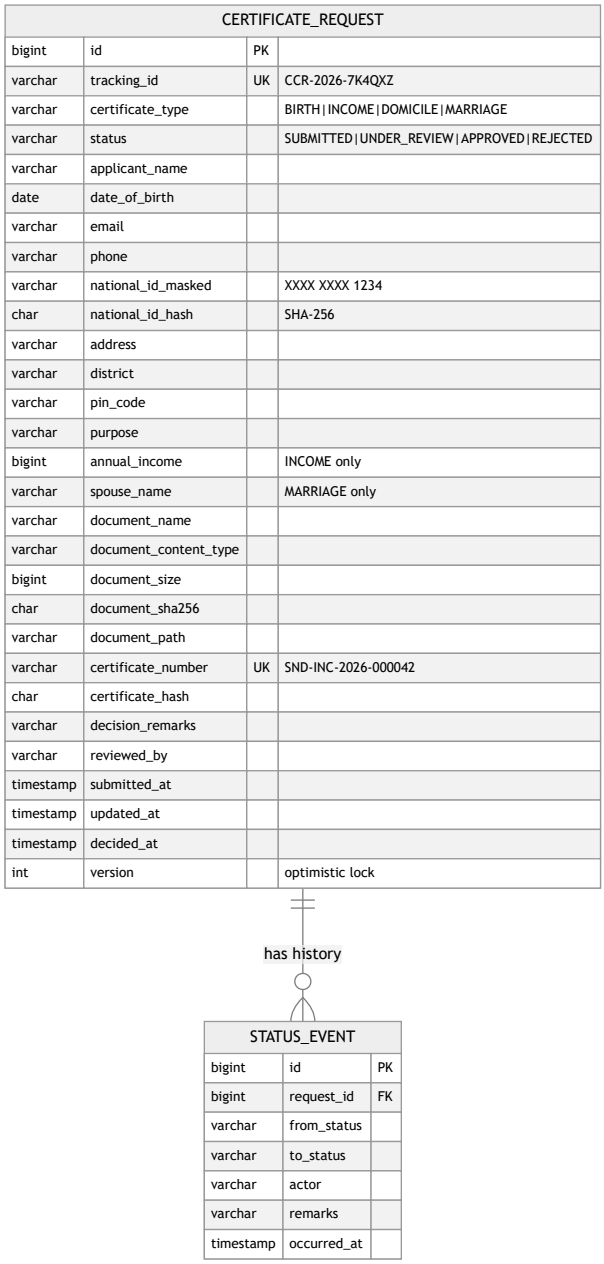
Environment	Runtime	URL	Provisioned by
local	`./mvnw spring-boot:run` or `java -jar`	http://localhost:8080	Developer
e2e	`spring-boot:start` during `-Pe2e`	http://localhost:18080	Maven
staging (ephemeral)	Docker container on the CI network	http://ccrp-staging-<build>:8080	Jenkins
tomcat-<env>	Tomcat 11 container	http://localhost:8888/certportal-<env>	Jenkins (`TARGET_ENV`)
production	Docker container	http://localhost:8085	Jenkins
ansible-node	Ubuntu 24.04 + JRE 21	http://localhost:8095	Ansible

3.3 Package structure

```
com.sanad.portal
├── PortalApplication            # entry point + SpringBootServletInitializer (Tomcat)
├── config/                      # AppProperties, SecurityConfig
├── domain/                      # entities, enums, repositories
├── service/                     # CertificateRequestService, DocumentStorageService, validation helpers
└── web/                         # MVC controllers, form objects, JSON API, advice, error handling
```

---

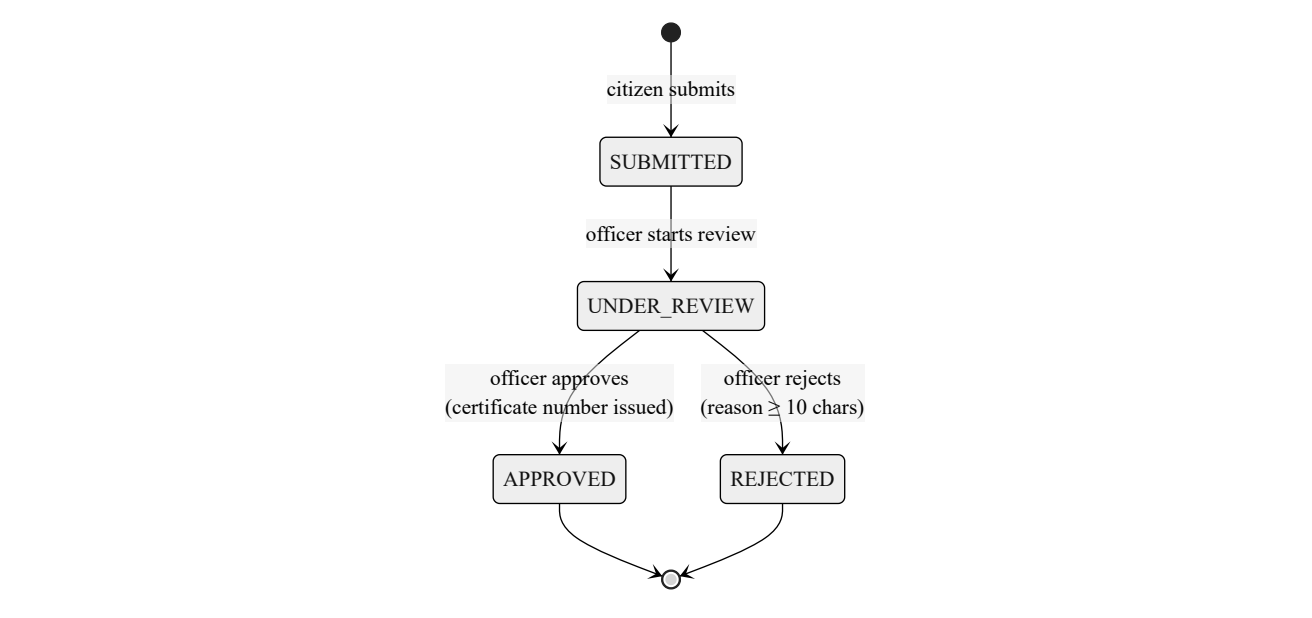
4. Data model



The schema is owned by Flyway (`src/main/resources/db/migration`). Hibernate runs with `ddl-auto=validate`, so a mismatch between entities and migrations fails at start-up rather than in production.

---

5. Status lifecycle



Any other transition raises `IllegalStatusTransitionException` and is shown as an error, never applied.

---

6. HTTP interface

6.1 Web routes (HTML)

Method	Path	Access	Purpose
GET	`/`	public	Landing page
GET	`/apply?type=`	public	Application form (optionally preselected type)
POST	`/apply`	public + CSRF	Submit request (multipart)
GET	`/apply/submitted/{trackingId}`	public (flash-guarded)	Acknowledgement
GET	`/track`	public	Lookup form
POST	`/track`	public + CSRF	Lookup by tracking ID + email (POST keeps the email out of URLs and logs)
GET	`/certificate/{trackingId}`	session-verified citizen, or officer	Printable certificate
GET	`/officer/login`	public	Officer sign-in
POST	`/officer/login` / `/officer/logout`	CSRF	Spring Security endpoints
GET	`/officer?status=&q=`	OFFICER	KPIs + queue
GET	`/officer/requests/{trackingId}`	OFFICER	Request detail
GET	`/officer/requests/{trackingId}/document`	OFFICER	Stream the supporting document (`Content-Disposition: inline`)
POST	`/officer/requests/{trackingId}/start-review`	OFFICER + CSRF	SUBMITTED → UNDER_REVIEW
POST	`/officer/requests/{trackingId}/approve`	OFFICER + CSRF	UNDER_REVIEW → APPROVED
POST	`/officer/requests/{trackingId}/reject`	OFFICER + CSRF	UNDER_REVIEW → REJECTED

6.2 JSON API (`/api/v1`)

Method	Path	Request	Response
GET	`/api/v1/certificate-types`	—	`[{"code":"INCOME","label":"Income cer`

Method	Path	Request	Response
POST	`/api/v1/status`	`{"trackingId":"CCR-2026-7K4QXZ","email":"a@b.in"}`	`200 {"trackingId","status","certifica`
GET	`/api/v1/certificates/{number}/verify`	—	`200 {"valid":true,"certificateNumber","ce` · `404 {"valid":false}`

The API is stateless and exempt from CSRF; it is read-only and needs no authentication.

6.3 Operations

Path	Purpose
`/actuator/health`	Aggregate health (`UP` / `DOWN`). Used by the Docker `HEALTHCHECK`, the Jenkins smoke test and the Ansible health check.
`/actuator/health/liveness`, `/actuator/health/readiness`	Probe groups
`/actuator/info`	Build version, git commit, environment, Java runtime

---

7. Technology choices

Concern	Choice	Why
Language / runtime	Java 21 (LTS)	Long-term support, records, pattern matching; the lab standard
Framework	Spring Boot 4.1 (Spring MVC, Security, Data JPA, Validation, Actuator)	Industry default; secure defaults; first-class Tomcat WAR support
Views	Thymeleaf, hand-written CSS/JS, no front-end build	One Maven build produces everything; pages work without JS and are easy to drive from Selenium
Build tool	**Maven** via wrapper (`./mvnw`, Maven 3.9.16)	Reproducible builds with no local Maven install; native Jenkins/Surefire/Failsafe reporting
Database	H2 2.x in file mode (PostgreSQL compatibility), Flyway	No external DB server for the MVP; Flyway migrations keep a later move to PostgreSQL to a config change
Packaging	Executable **WAR**	Same artefact for `java -jar`, Docker and **Tomcat 11**
Deployment targets	Tomcat 11 (Jakarta Servlet 6.1) and Docker (`eclipse-temurin:21-jre`)	Both deployment styles the course requires
Tests	JUnit 5, Spring MockMvc, Selenium WebDriver 4 (Chrome)	Unit → integration → E2E pyramid
CI/CD	Jenkins LTS (JDK 21) in Docker, Configuration-as-Code, Jenkinsfile	Pipeline-as-code, reproducible controller
Config management	Ansible	Agentless (SSH); YAML playbooks are easy to review

---

8. Configuration reference

Property	Env var	Default	Notes
`server.port`	`SERVER_PORT`	`8080`	
`app.environment`	`APP_ENVIRONMENT`	`local`	Shown as a badge in the footer: local / e2e / staging / production
`app.data-dir`	`APP_DATA_DIR`	`./data`	H2 files in `db/`, uploads in `documents/`
`app.documents.max-size`	`APP_DOCUMENTS_MAX_SIZE`	`2MB`	Business limit for uploads
`app.officer.username`	`APP_OFFICER_USERNAME`	`officer`	

Property	Env var	Default	Notes
`app.officer.password`	`APP_OFFICER_PASSWORD`	`Officer@123` (local only)	Injected from Jenkins credentials or Ansible Vault outside local
`spring.profiles.active`	`SPRING_PROFILES_ACTIVE`	—	`dev` disables template caching

---

9. Local development setup

9.1 Prerequisites

Tool	Version	Check
JDK	21	`java -version`
Git	≥ 2.40	`git --version`
Google Chrome	any current	`google-chrome --version` (needed only for Selenium)
Docker Engine + Compose	≥ 24 / v2	`docker compose version` (needed only for CI/CD and containers)

Maven does **not** need to be installed. `./mvnw` downloads the pinned version on first use.

9.2 Run

```
git clone git@github.com:tanishqbuilds/citizen-certificate-portal.git
cd citizen-certificate-portal

./mvnw spring-boot:run -Dspring-boot.run.profiles=dev     # http://localhost:8080
# or
./mvnw -DskipTests package && java -jar target/*.war
```

Officer sign-in for local runs: `officer` / `Officer@123`.

9.3 Test

```
./mvnw verify            # unit + MockMvc integration tests, JaCoCo report in target/site/jacoco
./mvnw verify -Pe2e      # also starts the app on :18080 and runs the Selenium journeys (headless Chrome)
```

9.4 Verified local setup (evidence)

```
$ ./mvnw -B verify
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS

$ java -jar target/citizen-certificate-portal-0.1.0-SNAPSHOT.war &
$ curl -s localhost:8080/actuator/health
{"groups":["liveness","readiness"],"status":"UP"}
$ curl -s localhost:8080/actuator/info
{"app":{"name":"GetDocs — Citizen Certificate Request Portal","environment":"local"},
 "git":{"branch":"main","commit":{"id":"2c7d1b2",…}},
 "build":{"artifact":"citizen-certificate-portal","version":"0.1.0-SNAPSHOT",…}}
```

TASK 04 • PHASES 4 - 6

Version Control, Feature Branching, Merge Conflict Resolution & Release Management

Reference Document: CCRP-DOC-04

Field	Value
Document ID	CCRP-DOC-04
Version	1.0
Status	Approved
Owner	Tanishq
Last updated	2026-10-11

Contents

1. Repository
2. Branch model and naming rules
3. Commit convention
4. Issues, labels and milestones
5. Phase 5: feature branch, pull request and review
6. Phase 6: second feature branch and merge conflict
7. Release v1.0.0 and tagging
8. Backlog status at v1.0.0
9. Everyday Git command reference

---

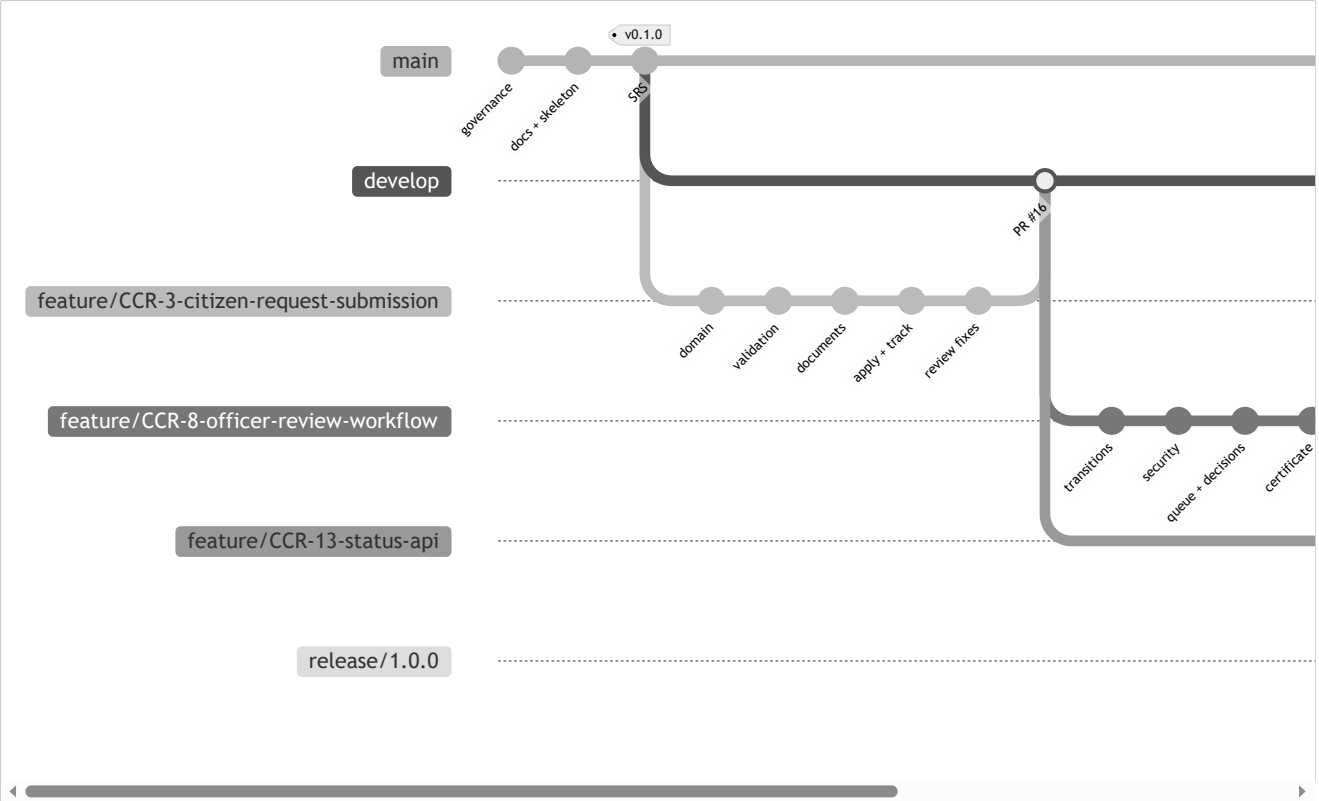
1. Repository

Item	Value
URL	https://github.com/tanishqbuilds/citizen-certificate-portal
Visibility	Public
Default branch	`main`
Merge strategy	**Merge commits only** (squash and rebase disabled) so feature boundaries remain visible in `git log --graph`
Templates	`.github/ISSUE_TEMPLATE/{user_story,bug_report,devops_task}.yml`, `.github/pull_request_template.md`
Ownership	`.github/CODEOWNERS` → `@tanishqbuilds`
Policy	`CONTRIBUTING.md`

Initial commits on `main`:

```
2c7d1b2 chore: initialize repository with governance files
c6b4010 docs: add problem definition, stakeholders and frozen MVP scope
4bcb806 build: add Spring Boot 4 skeleton with Maven wrapper and design system
f48b0cc docs: add SRS summary, architecture, data model and API list   (tag v0.1.0)
```

2. Branch model and naming rules



Prefix	Use	Example in this repo
`feature/CCR-<n>-<slug>`	User-facing story	`feature/CCR-3-citizen-request-submission`
`fix/CCR-<n>-<slug>`	Non-urgent defect	`fix/CCR-14-...` (Phase 10)
`ci/<slug>`	Jenkins / Docker / Ansible	`ci/jenkins-pipeline` (Phase 7–8)
`docs/<slug>`	Documentation only	—
`release/<x.y.z>`	Release hardening	`release/1.0.0`
`hotfix/<x.y.z>`	Urgent production fix	—

Rules: lowercase, hyphen-separated, the backlog key is mandatory for `feature/` and `fix/`, and nobody commits directly to `main` or `develop`. Everything arrives through a pull request.

3. Commit convention

Conventional Commits with a `Refs: CCR-<n>` trailer. Each commit is one logical change that builds on its own. Examples from the history:

```
feat(validation): validate applicant data and type-specific rules
feat(documents): validate uploads by magic bytes and store with SHA-256
fix(security): resolve CSRF tokens eagerly to avoid committed-response errors
fix(validation): evaluate ages in IST instead of the JVM default zone
refactor(officer): detect anonymous users by type and flag truncated queues
```

4. Issues, labels and milestones

Artefact	Content
Issues #1–#15	One per MVP backlog item (issue *n* = CCR-*n*), body = story + acceptance criteria from `backlog.csv`
Labels	`type:story`, `type:task`, `type:bug`, `epic:citizen`, `epic:officer`, `epic:platform`, `priority:must`, `priority:should`, `status:*`
Milestones	Sprint 0 → Sprint 4 with due dates from the sprint plan

5. Phase 5: feature branch, pull request and review

Feature 1: citizen journey (CCR-3 … CCR-7) on `feature/CCR-3-citizen-request-submission`.

```
git checkout -b feature/CCR-3-citizen-request-submission develop
# … work …
git add <files>; git commit -m "feat(domain): …"     # 6 focused commits
git push -u origin feature/CCR-3-citizen-request-submission
gh pr create --base develop --title "feat: citizen request submission …" --body-file pr.md
git pull --ff-only origin develop                     # after merge
git log --oneline --graph
```

Step	Evidence
Pull request	#16 with the PR template completed
Review comments	3 inline comments: orphaned documents on failed save, UTC vs IST age calculation, rate limiting (deferred)
Changes after review	`41e1335`, `cecef29`, `18fcf05`, `2b87f5c` (screenshots), each replied to on the thread
Merge	Merge commit `cb3f3f2` into `develop`
Screenshots	`docs/evidence/phase-05-feature-branch/`

6. Phase 6: second feature branch and merge conflict

Two branches were developed **in parallel** from the same `develop` commit (`cb3f3f2`):

Branch	PR	Touches
`feature/CCR-13-status-api`	#17, merged first	`SecurityConfig` (CSRF exemption for `/api/**`), repository, service, README
`feature/CCR-8-officer-review-workflow`	#18	`SecurityConfig` (form login, roles), repository, service, README

After #17 merged, GitHub reported #18 as **`CONFLICTING`**. Resolution, done on the feature branch so `develop` was never broken:

```
$ git merge origin/develop
CONFLICT (content): Merge conflict in README.md
CONFLICT (content): Merge conflict in src/main/java/com/sanad/portal/config/SecurityConfig.java
CONFLICT (content): Merge conflict in src/main/java/com/sanad/portal/domain/CertificateRequestRepository.java
CONFLICT (content): Merge conflict in src/main/java/com/sanad/portal/service/CertificateRequestService.java
Automatic merge failed; fix conflicts and then commit the result.
```

File	Decision
`SecurityConfig.java`	Semantic merge: officer login + role rules from the feature **and** `ignoringRequestMatchers("/api/**")` from develop
Repository / service	Additive. Both sides' methods kept. The first attempt dropped a closing brace; `./mvnw clean verify` caught it before the merge commit
`README.md`	Both feature lists combined

Merge commit `bb45e4a`, then a cross-feature test (`ff81763`) that approves a request and verifies its certificate through the new API, which neither branch could test alone. Full log: `merge-conflict.log`. Screenshots of the complete MVP: `docs/evidence/phase-06-mvp/`.

7. Release v1.0.0 and tagging

```
git checkout -b release/1.0.0 develop
./mvnw versions:set -DnewVersion=1.0.0 -DgenerateBackupPoms=false
# CHANGELOG.md, docs → PR release/1.0.0 → main
git tag -a v1.0.0 -m "GetDocs v1.0.0 — MVP"
git push origin v1.0.0
git checkout develop && git merge --no-ff main          # back-merge
./mvnw versions:set -DnewVersion=1.1.0-SNAPSHOT         # next development iteration
```

The tagged commit on `main` is the **release-ready source baseline**. `./mvnw clean verify` passes on it (65 tests), it builds `citizen-certificate-portal-1.0.0.war`, and `/actuator/info` reports version `1.0.0` with the tag's commit.

8. Backlog status at v1.0.0

Key	Story	Issue	Delivered by	Status
CCR-1	Repository & project skeleton	#1	v0.1.0	✅ Done
CCR-2	Design system & UI shell	#2	v0.1.0	✅ Done
CCR-3	Certificate request submission	#3	PR #16	✅ Done
CCR-4	Applicant data validation	#4	PR #16	✅ Done
CCR-5	Document upload & validation	#5	PR #16	✅ Done
CCR-6	Tracking ID & acknowledgement	#6	PR #16	✅ Done
CCR-7	Status tracking with timeline	#7	PR #16	✅ Done
CCR-8	Officer authentication	#8	PR #18	✅ Done
CCR-9	Review queue with filters & KPIs	#9	PR #18	✅ Done
CCR-10	Review actions (state machine)	#10	PR #18	✅ Done
CCR-11	Certificate issuance & verification	#11	PR #18	✅ Done
CCR-12	Audit trail	#12	PR #16, #18	✅ Done
CCR-13	Health, info & JSON API	#13	PR #17	✅ Done
CCR-14	Selenium E2E suite	#14	—	⏳ Sprint 3
CCR-15	Containerised CI/CD & provisioning	#15	—	⏳ Sprint 4

13 of 15 items done. 55 of 76 story points burned.

9. Everyday Git command reference

Goal	Command
Start a story	`git switch develop && git pull && git switch -c feature/CCR-<n>-<slug>`
Stage interactively by file	`git add <path>`; review with `git diff --staged`
Commit	`git commit` (Conventional Commit subject + `Refs:` trailer)
Publish	`git push -u origin <branch>`
Stay current	`git fetch && git merge origin/develop` (never rebase shared branches)
History	`git log --oneline --graph --decorate --all`
Who changed a line	`git blame -L <from>,<to> <file>`
Undo a pushed commit	`git revert <sha>` (never force-push shared branches)
Inspect a release	`git show v1.0.0`, `git diff v0.1.0..v1.0.0 --stat`

Verification Evidence & Screenshots (Task 4)

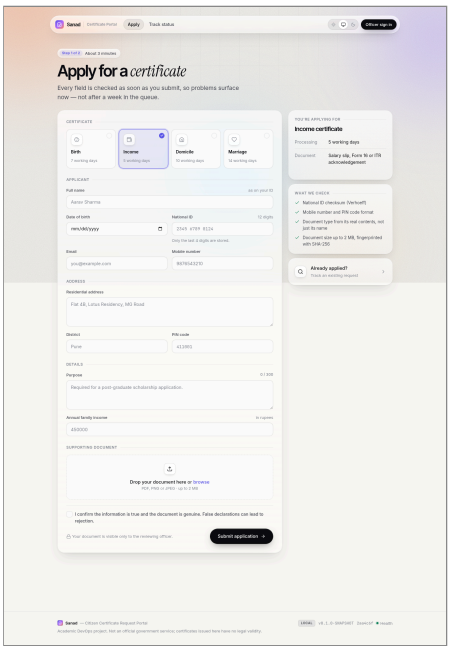


Figure 4.1: Citizen Certificate Application Form

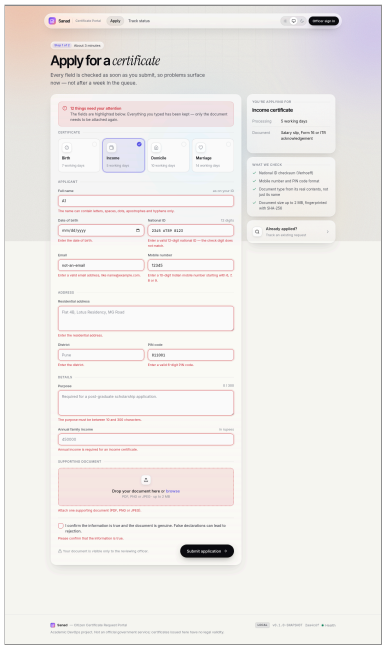


Figure 4.2: Client & Server-side Form Validation Error Feedback

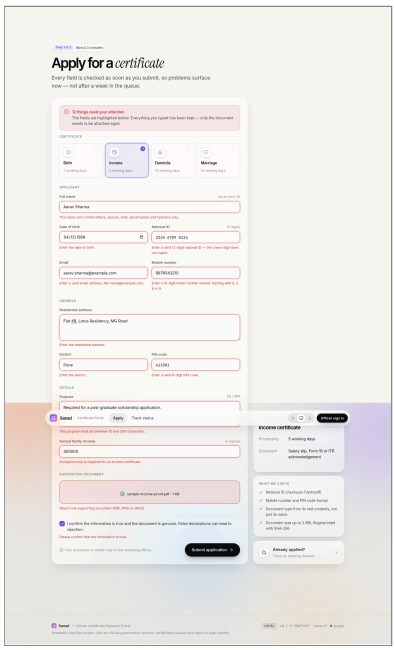


Figure 4.3: Completed Application with Validated Document Upload

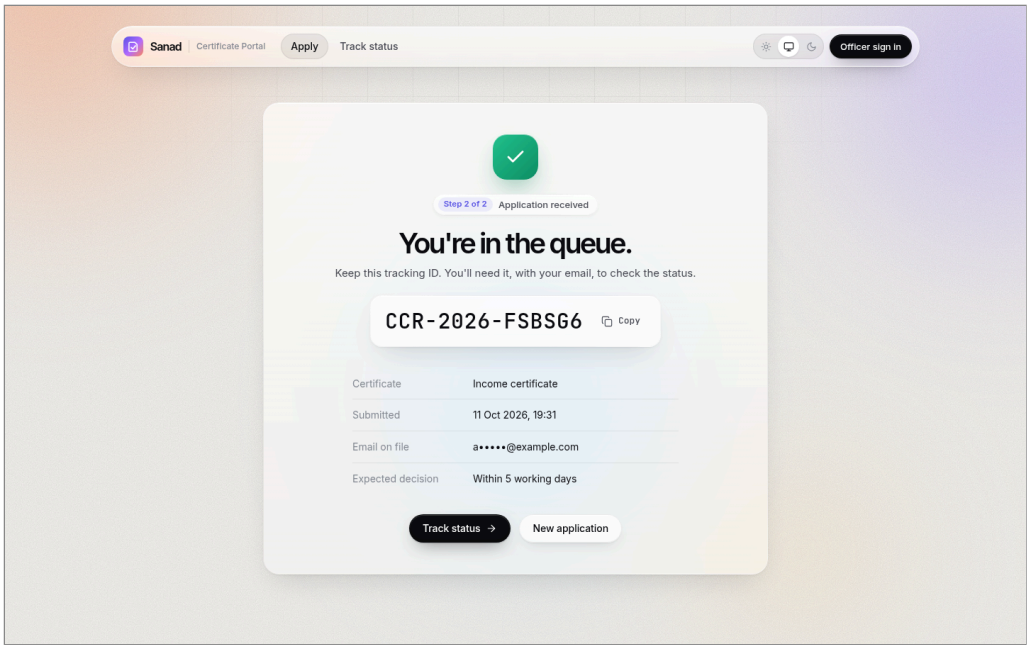


Figure 4.4: Submission Acknowledgement with Generated Tracking ID

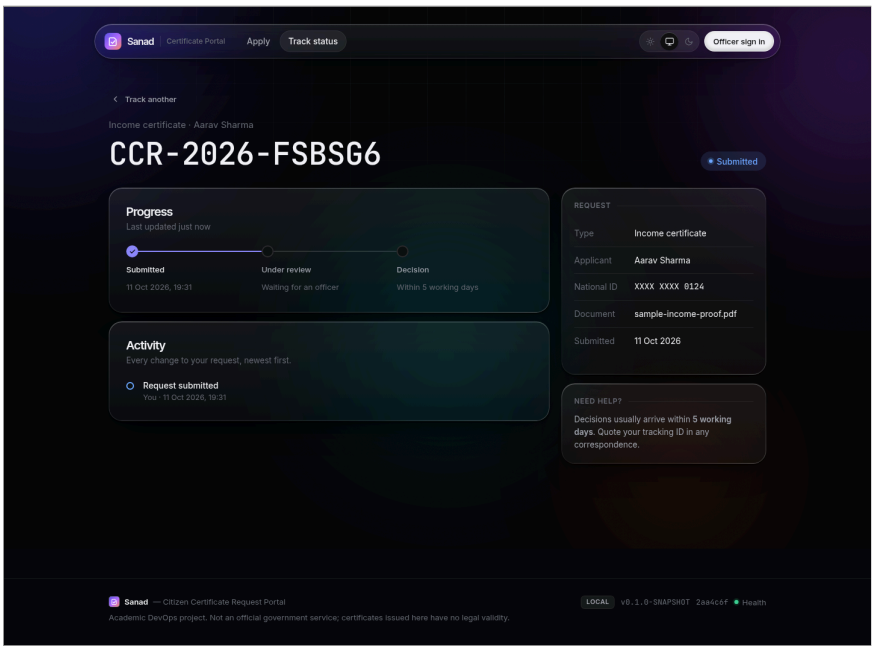


Figure 4.5: Citizen Application Tracking View (Dark Mode)

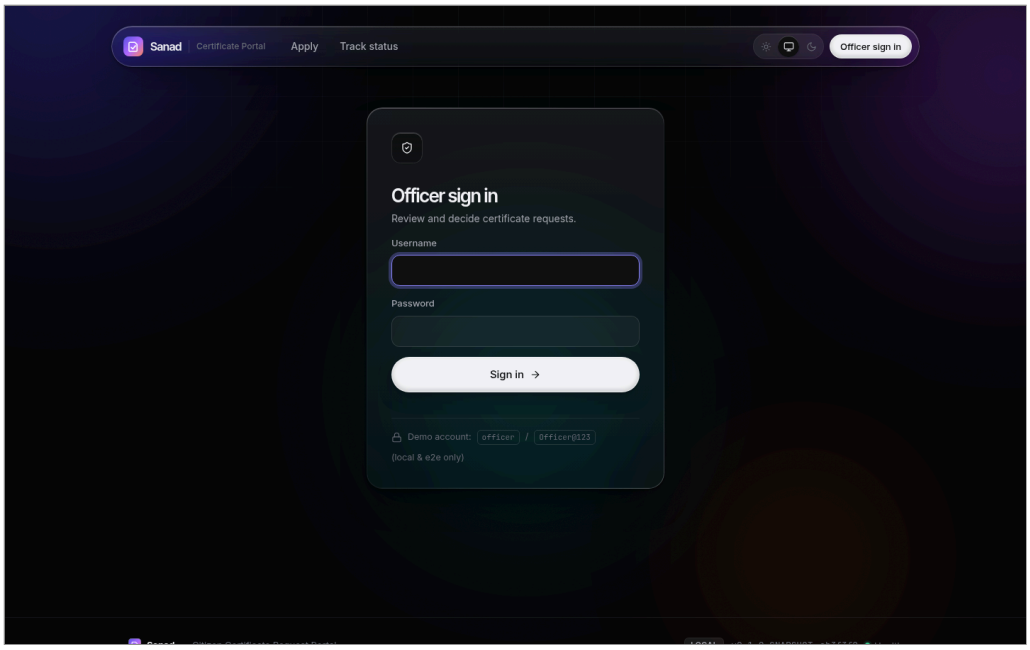


Figure 4.6: Officer Authentication Portal

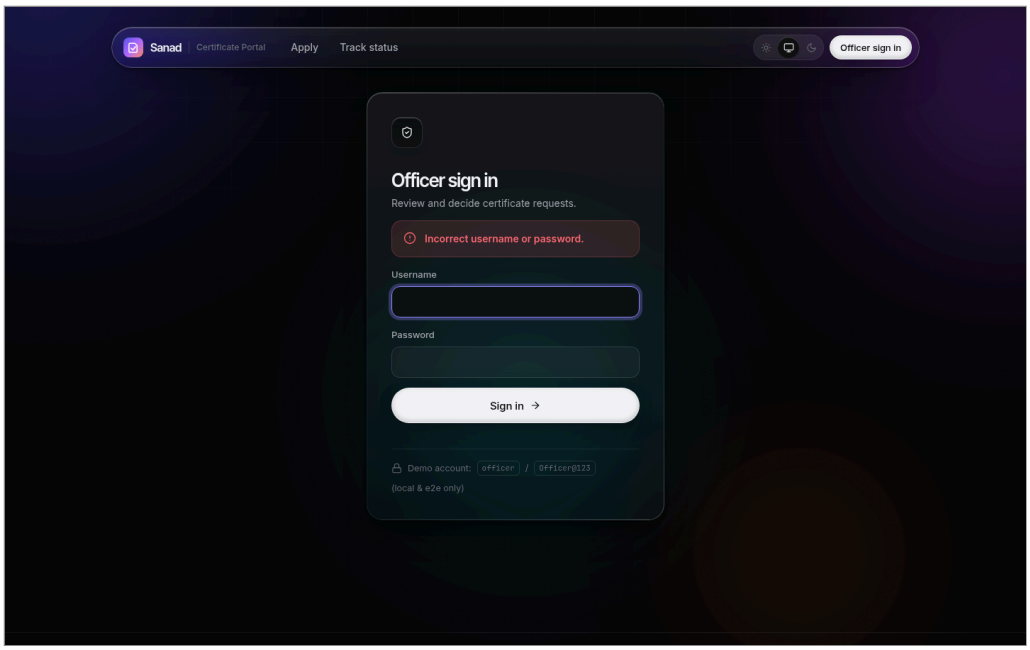


Figure 4.7: Officer Authentication Guardrail & Error Handling

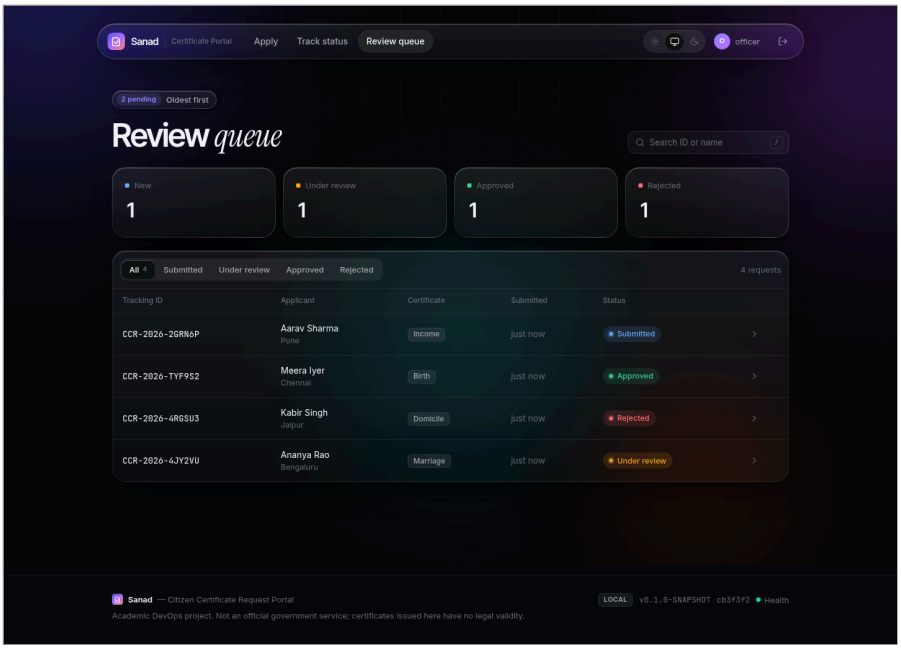


Figure 4.8: Officer Consolidated Review Queue with KPI Filters

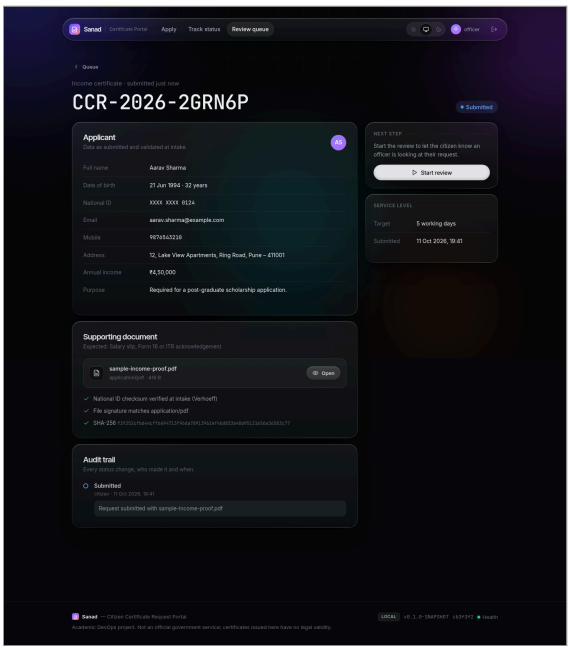


Figure 4.9: Application Details Inspection & Document Preview

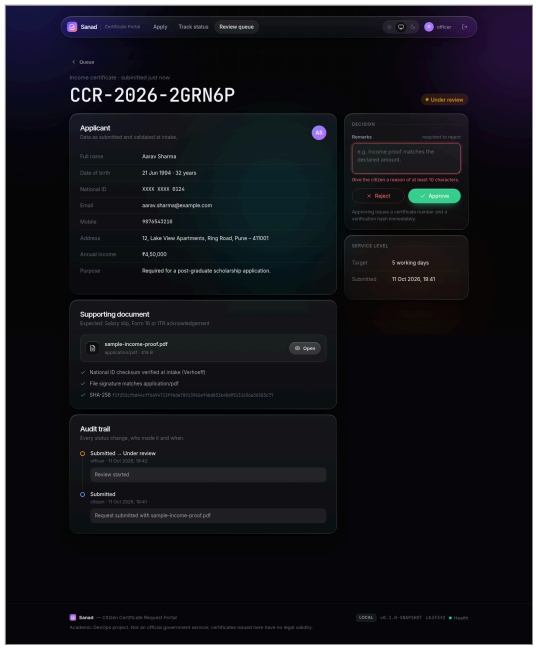


Figure 4.10: Guardrail Enforcing Mandatory Reason on Application Rejection

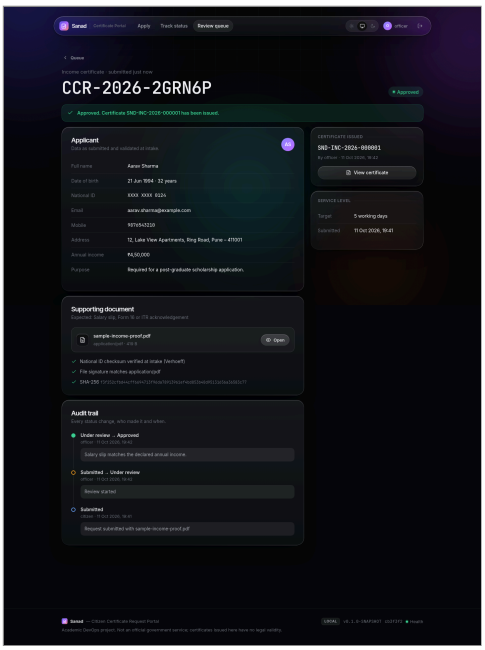


Figure 4.11: Approval Decision with Automatic Certificate Generation

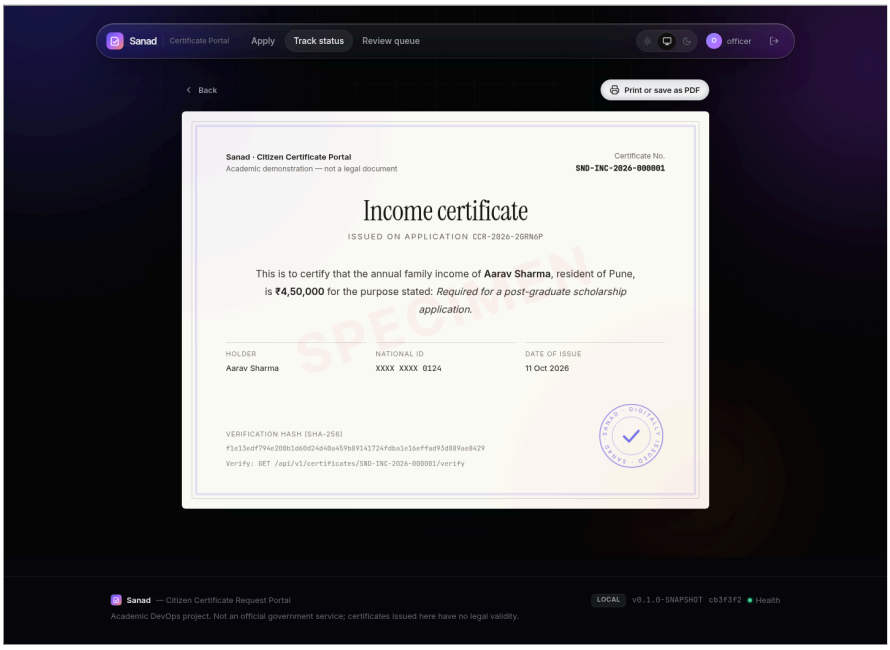


Figure 4.12: Issued Digital Certificate Specimen with Cryptographic SHA-256 Hash

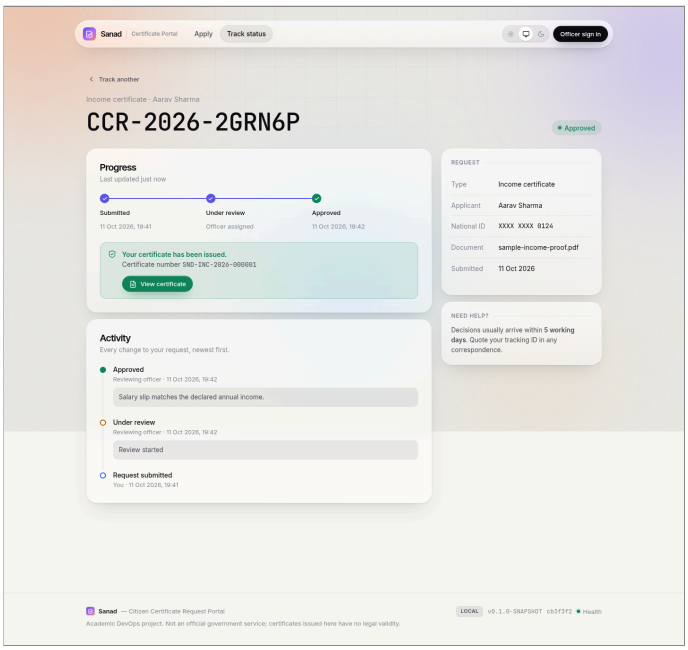


Figure 4.13: Citizen View of Approved Application with Certificate Download

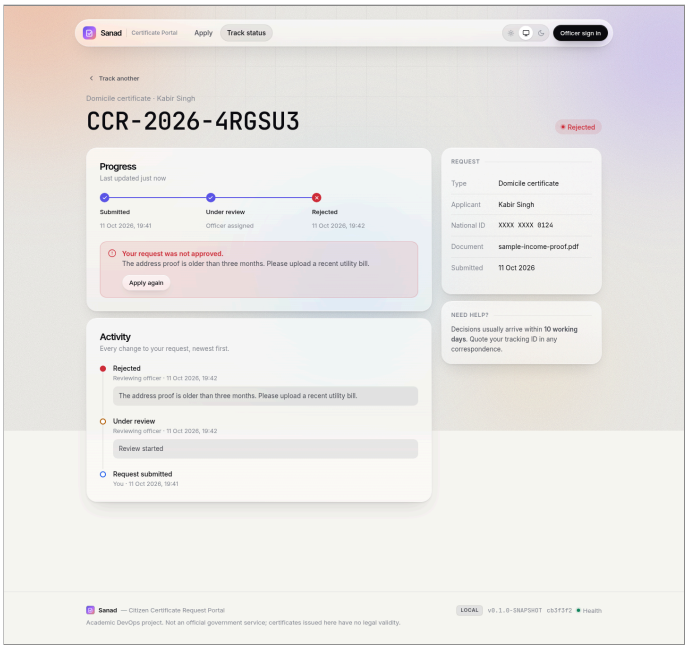


Figure 4.14: Citizen View of Rejected Application Showing Official Reason

TASK 05 • PHASES 7 - 8

Continuous Integration with Jenkins & Tomcat Deployment

Reference Document: CCRP-DOC-05

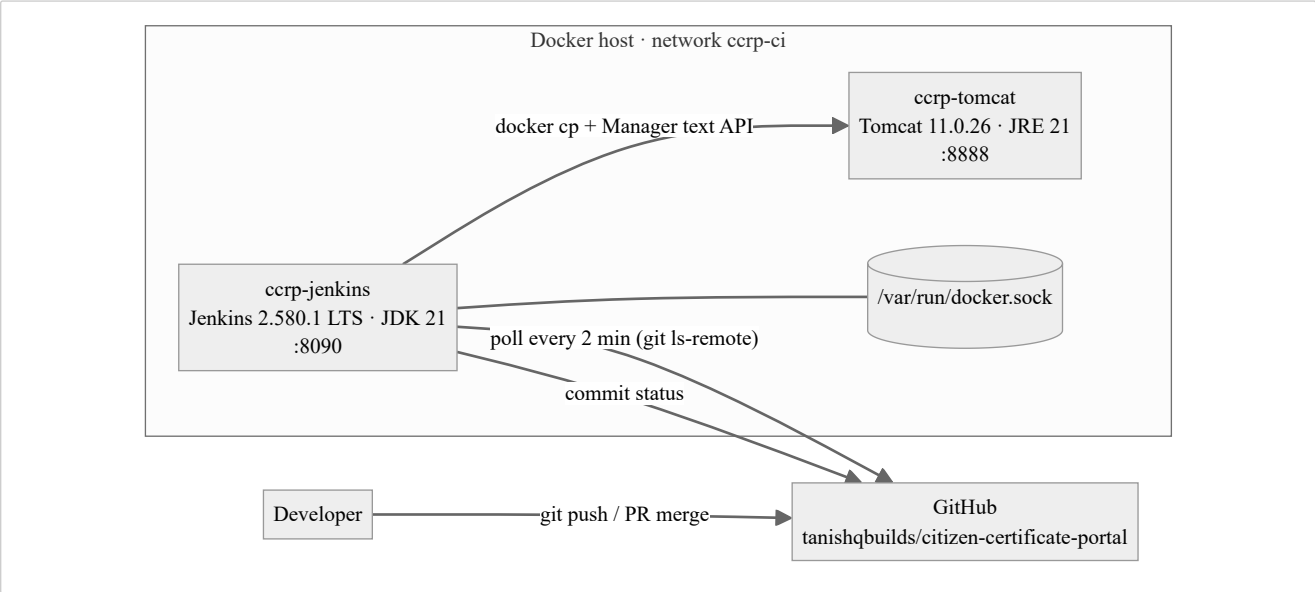
Field	Value
Document ID	CCRP-DOC-05
Version	1.0
Status	Approved
Owner	Tanishq
Last updated	2026-10-11

Contents

1. Toolchain topology
2. Installing Jenkins (as code)
3. Phase 7: the CI job
4. Phase 8: Jenkinsfile
5. Tomcat deployment and the `TARGET_ENV` parameter
6. Evidence
7. Problems found and fixed
8. Operating the toolchain

---

1. Toolchain topology



All CI/CD components run in Docker and are described in code:

Path	Content
`ci/docker-compose.yml`	Services, ports, volumes, network `ccrp-ci`
`ci/jenkins/Dockerfile`	Jenkins LTS + Docker CLI/buildx + pinned plugins, setup wizard disabled
`ci/jenkins/plugins.txt`	15 pinned top-level plugins (JCasC, Job DSL, Pipeline, Git, JUnit, Coverage, …)
`ci/jenkins/casc/jenkins.yaml`	Security realm, authorisation, credentials, global env vars
`ci/jenkins/casc/jobs.groovy`	Job DSL for `ccrp-ci-build` and `ccrp-pipeline`
`infra/tomcat/`	Tomcat image, Manager restrictions, `manager-script` user template
`scripts/ci-up.sh`	Generates `ci/.env` with random secrets, builds, starts, prints URLs

2. Installing Jenkins (as code)

```
scripts/ci-up.sh                    # first run creates ci/.env (chmod 600) with random secrets
# optional: let Jenkins publish commit statuses
GITHUB_TOKEN=$(gh auth token) scripts/ci-up.sh
```

Concern	How it is handled
Admin user	JCasC `securityRealm.local` from `JENKINS_ADMIN_PASSWORD`. Sign-up disabled, anonymous read off
Secrets	Jenkins credentials `tomcat-deployer`, `officer-password` and `github-token`, created by JCasC from env vars. Never in Git
Plugins	Installed at image build time by `jenkins-plugin-cli`, versions pinned
Jobs	Created by Job DSL at start-up. Nothing is configured by clicking
Docker access	Host socket mounted; container joins the socket's group (`DOCKER_GID`)
Reproducibility	`docker compose down -v && scripts/ci-up.sh` rebuilds an identical controller

3. Phase 7: the CI job

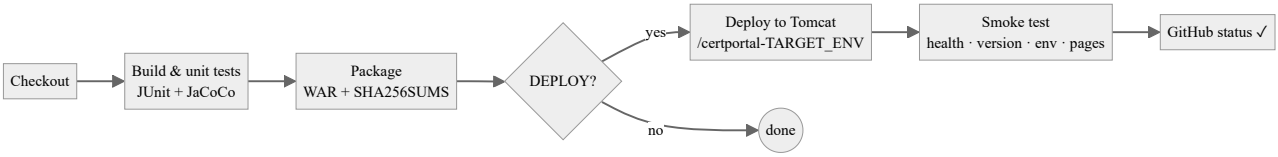
`ccrp-ci-build`, a freestyle job (Job DSL):

Setting	Value
SCM	`https://github.com/tanishqbuilds/citizen-certificate-portal.git`, branch `*/develop`, clean before checkout
Trigger	**SCM polling** `H/2 * * * *`. A GitHub webhook cannot reach a laptop without a public tunnel, so polling is the reliable trigger
Build	`./mvnw -B -ntp clean verify` (Maven wrapper 3.9.16: compile, 65 tests, JaCoCo, package)
Post-build	JUnit report; `target/*.war` archived with a fingerprint, only on success
Hygiene	20-minute timeout, timestamps, keep 30 builds / 10 artefacts

Trigger evidence: the polling log shows the change detection, and build #2 records `Started by an SCM change`:

```
[poll] Last Built Revision: Revision 27977e4c… (refs/remotes/origin/develop)
 > git ls-remote -h -- https://github.com/tanishqbuilds/citizen-certificate-portal.git
[poll] Latest remote head revision on refs/heads/develop is: debded13…
Changes found
```

4. Phase 8: Jenkinsfile



Stage	Key steps
Checkout	`checkout scm`; derives `GIT_COMMIT`, `APP_VERSION` (from the POM); names the build `#n · version · sha`; GitHub status `pending`
Build & unit tests	`./mvnw clean verify`; `junit`; `recordCoverage` (JaCoCo)
Package	Archive `citizen-certificate-portal-<version>.war` + `SHA256SUMS`, fingerprinted
Deploy to Tomcat	`scripts/deploy-tomcat.sh target/ccrp.war "$TARGET_ENV"` with credentials bound from Jenkins
Smoke test	`scripts/smoke-test.sh` asserts health `UP`, `build.version == APP_VERSION`, `app.environment == TARGET_ENV`, pages render, `/officer` redirects
post	GitHub commit status `success` / `failure` (context `jenkins/ccrp-pipeline`); workspace cleaned

Options: `timestamps`, `ansiColor`, 30-minute timeout, `disableConcurrentBuilds`, build discarder.

5. Tomcat deployment and the `TARGET_ENV` parameter

Parameter	Values	Effect
`TARGET_ENV`	`staging` (default), `production`	Context path `/certportal-<env>`, `app.environment`, data directory `/usr/local/tomcat/data/<env>`
`DEPLOY`	`true` (default), `false`	Skip deployment (build-only run)

The same WAR serves both environments. `scripts/deploy-tomcat.sh` generates a **context descriptor** whose `<Parameter>` elements become ServletContext init parameters, which Spring Boot reads as configuration:

```
<Context>
  <Parameter name="app.environment" value="production" override="false"/>
  <Parameter name="app.data-dir"    value="/usr/local/tomcat/data/production" override="false"/>
  <Parameter name="app.officer.password" value="********" override="false"/>
</Context>
```

and deploys through the Manager text API:

```
GET /manager/text/undeploy?path=/certportal-production                      (if already running)
GET /manager/text/deploy?path=/certportal-production
        &config=file:/usr/local/tomcat/deployments/certportal-production.xml
        &war=file:/usr/local/tomcat/deployments/certportal-production.war
        &tag=1.1.0-SNAPSHOT
OK - Deployed application at context path [/certportal-production]
```

The Manager is restricted to loopback and private networks (`RemoteCIDRValve`), and the deploy account has only `manager-script` (text API, no GUI).

URL	Environment
http://localhost:8888/certportal-staging/	staging (polled builds)
http://localhost:8888/certportal-production/	production (parameterised build #3)

6. Evidence

Phase	Item	File
7	Jenkins dashboard, job page with artefact and test trend	`phase-07-jenkins-ci/01-02`
7	Successful build log (`Started by an SCM change`, 65 tests, archived WAR)	`build-2-success-console.log`
7	Polling log	`scm-polling.log`
7	Archived artefact + fingerprint	`build-2-artifacts.json`
8	Stage view and stage graph	`phase-08-pipeline-tomcat/01, 02b`
8	Parameters (`TARGET_ENV=production`)	`03-build-with-parameters.png`, `04-build-3-parameters.png`
8	Pipeline logs (#1 failure, #2 staging, #3 production)	`pipeline-build-*.log`
8	Manager list, context descriptor, `/actuator/info` per env, GitHub statuses	`tomcat-deployment-evidence.txt`
8	Deployed application on Tomcat	`08-tomcat-staging-home.png`, `09-tomcat-production-apply.png`

7. Problems found and fixed

#	Symptom	Root cause	Fix
1	Jenkins kept restarting	`excludeClientIPFromCrumb` no longer exists in the crumb issuer	`crumbIssuer: "standard"`

#	Symptom	Root cause	Fix
2	Polled build queued but never ran	Built-in node went **offline**: host root disk at 100% (Disk Space Monitor < 1 GB)	Freed space (Docker build cache), node back online
3	`ccrp-ci-build` #1: login test failed	Jenkins exports `APP_OFFICER_PASSWORD`; env vars outrank `application-test.yml`	Test pins the password via `@SpringBootTest(properties=…)` (#22)
4	Context failed on Tomcat: `NoClassDefFoundError: …/WebServer`	Maven nearest-wins put `spring-boot-web-server` in `lib-provided`	Declare it explicitly at compile scope (#23)
5	`ccrp-pipeline` #1: deploy called with empty env	Jenkinsfile parameters only exist after the first run	Parameters also in Job DSL; defaults in the Jenkinsfile (#24)
6	Smoke checks silently skipped	`curl … | grep -q` closes the pipe early (curl exit 23) and `&&` swallowed it	Fetch into a variable, then assert

8. Operating the toolchain

Task	Command
Start / update	`scripts/ci-up.sh`
Stop (keep data)	`docker compose -f ci/docker-compose.yml --env-file ci/.env stop`
Reset completely	`docker compose -f ci/docker-compose.yml --env-file ci/.env down -v`
Trigger a production deploy	Jenkins → *CCRP · Delivery pipeline* → *Build with Parameters* → `TARGET_ENV=production`
Same from the CLI	`curl -u admin:$PW -X POST "http://localhost:8090/job/ccrp-pipeline/buildWithParameters?TARGET_ENV=production"` (crumb required)
Deploy by hand	`TOMCAT_URL=http://localhost:8888 TOMCAT_USER=deployer TOMCAT_PASSWORD=… OFFICER_PASSWORD=… scripts/deploy-tomcat.sh target/*.war staging`
Tomcat logs	`docker logs -f ccrp-tomcat`

Verification Evidence & Screenshots (Task 5)

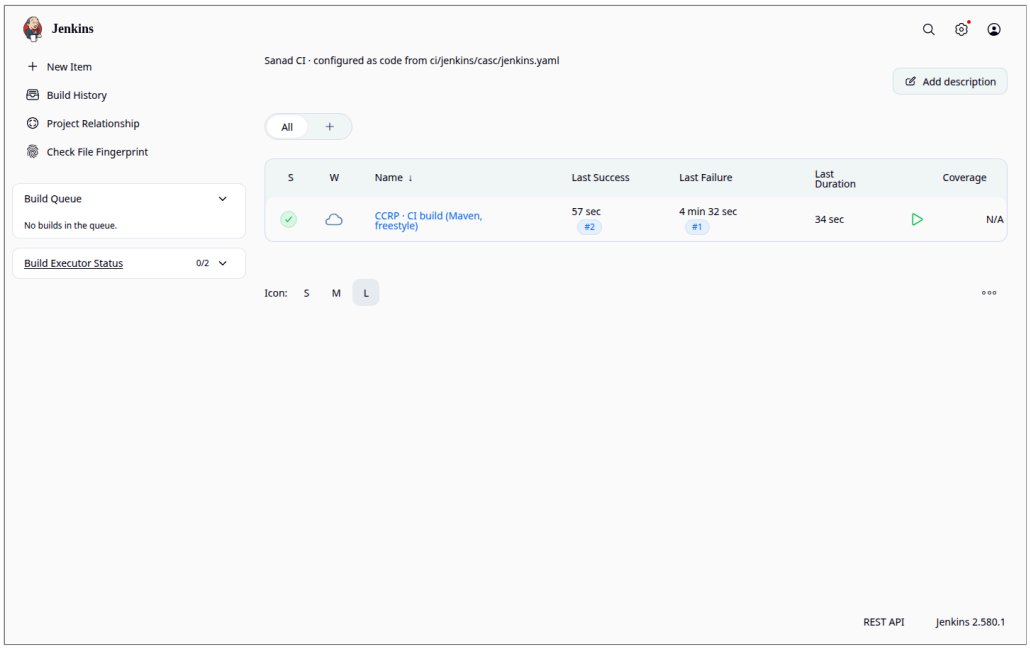


Figure 5.1: Jenkins Configuration-as-Code Controller Dashboard



Figure 5.2: Automated CI Build Pipeline Job

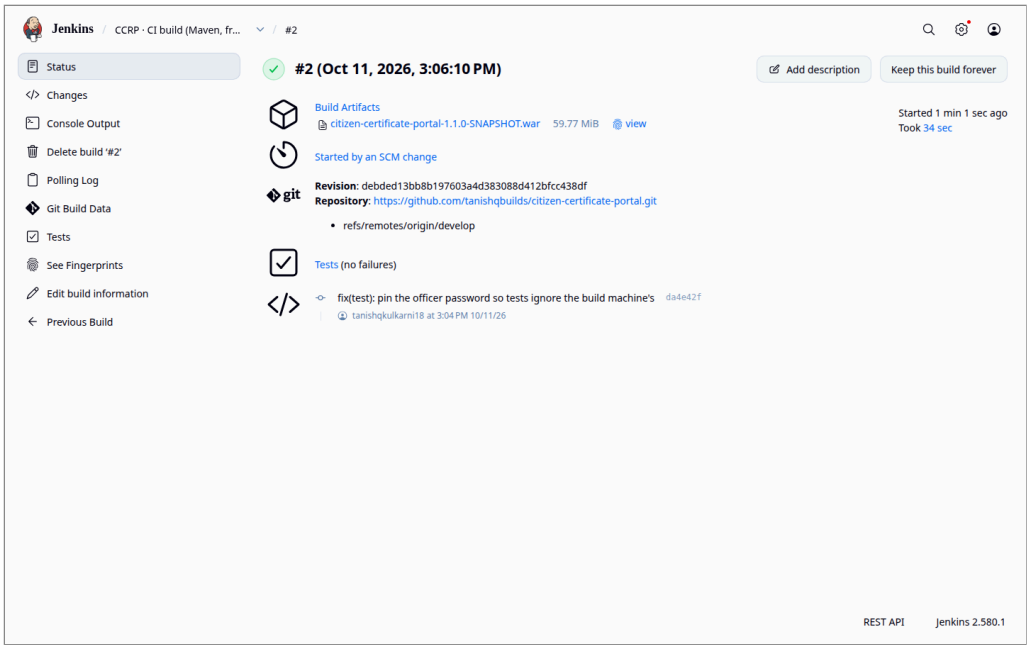


Figure 5.3: Successful Build Status & SCM Revision Tracking

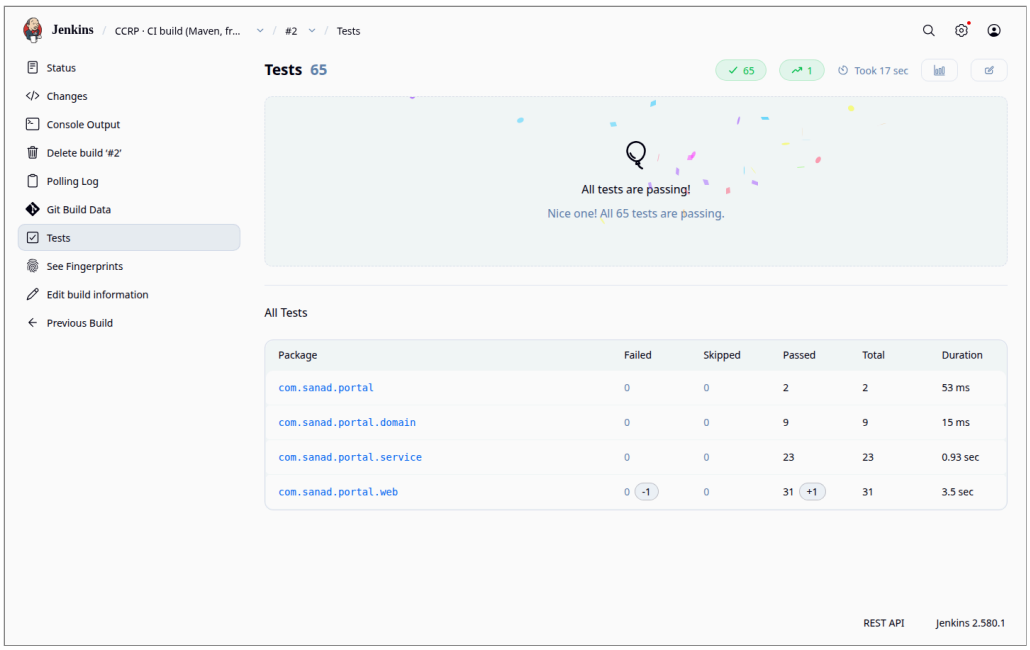


Figure 5.4: Automated Test Execution Results (65 Tests Passing)

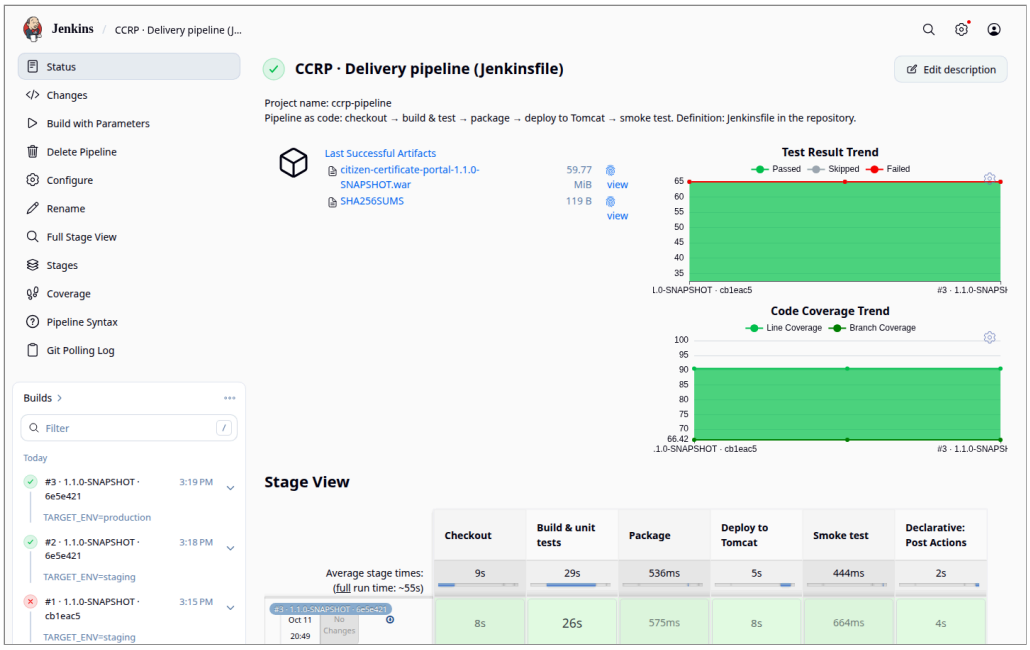


Figure 5.5: Multi-Stage Declarative Pipeline Stage View

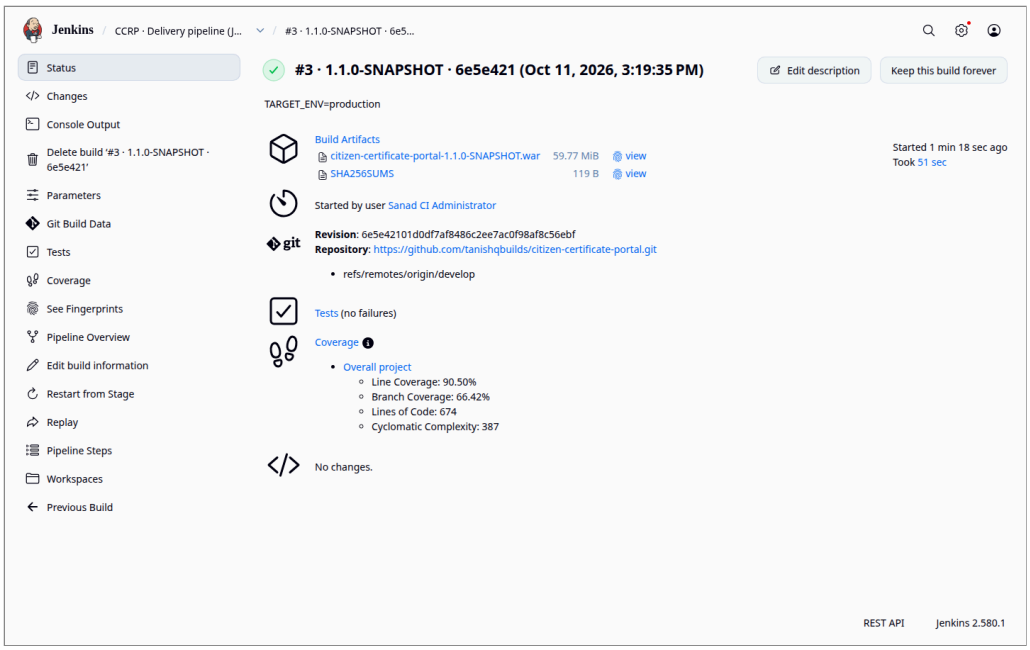


Figure 5.6: Pipeline Build Summary with Archived Artifacts

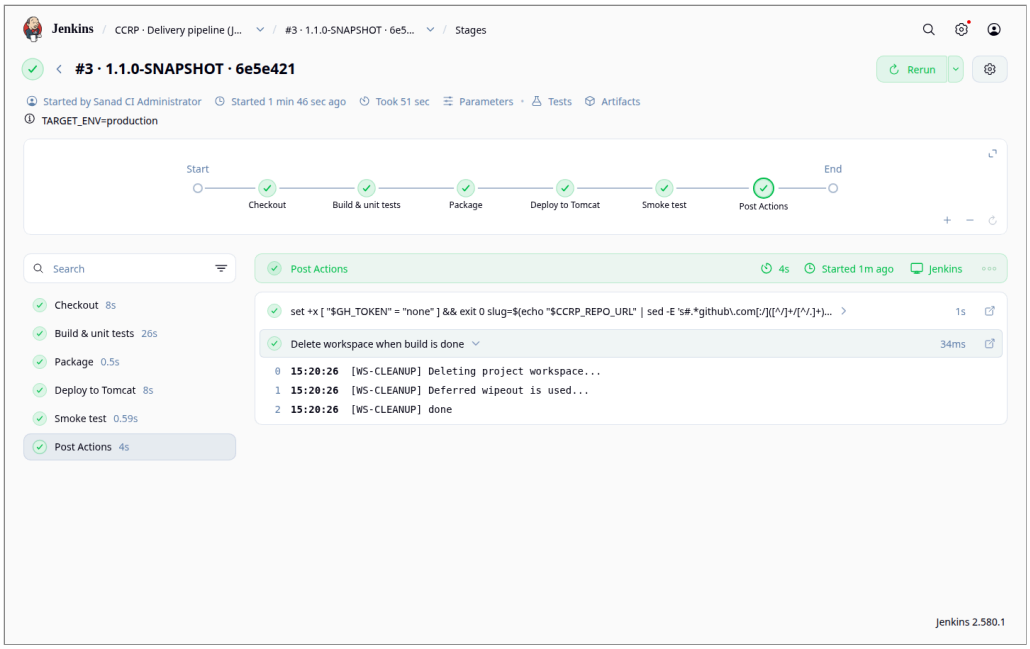


Figure 5.7: Declarative Pipeline Stage Execution Graph

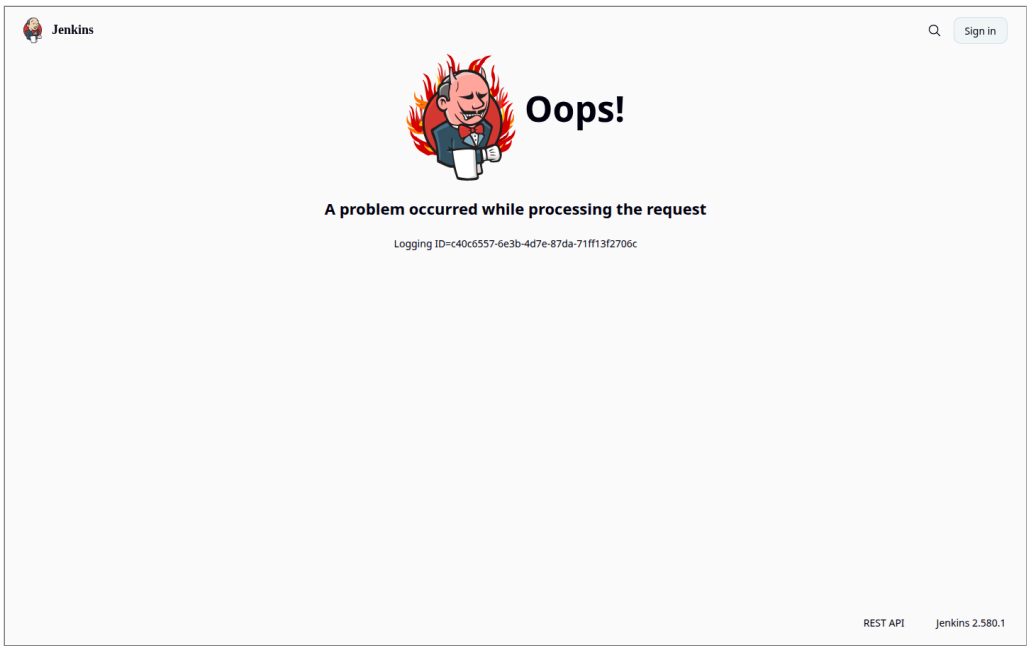


Figure 5.8: Parameterized Deployment Trigger Interface

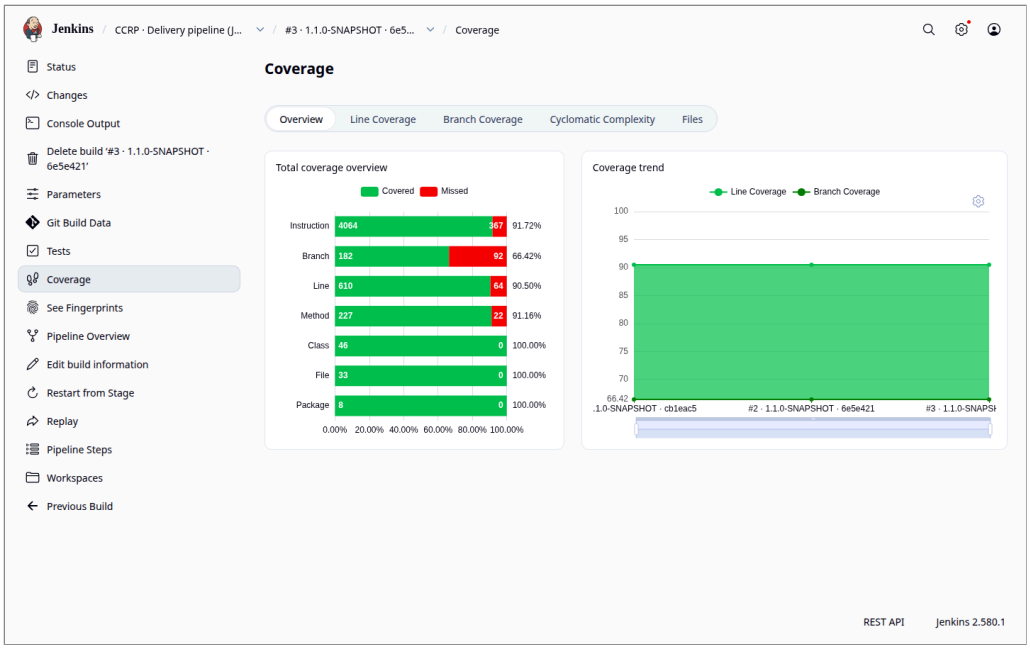


Figure 5.9: JaCoCo Code Coverage Report (90.5% Coverage)

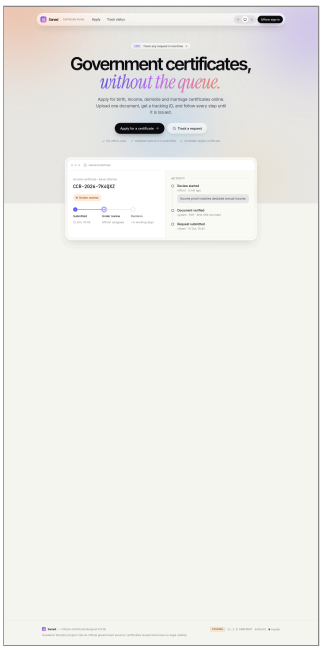


Figure 5.10: Deployed Application on Apache Tomcat 11 Staging

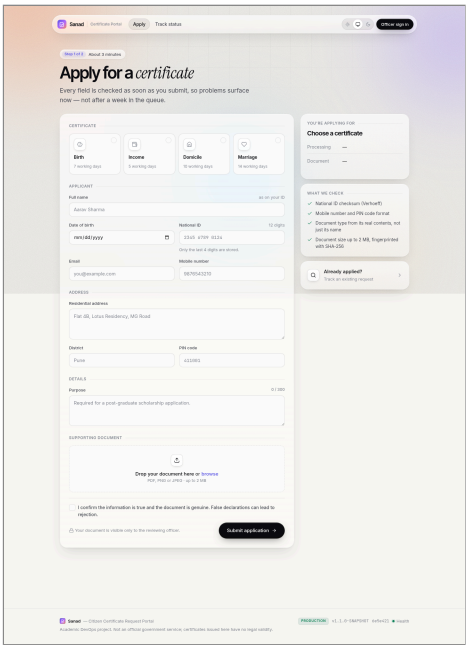


Figure 5.11: Production Deployment on Tomcat 11

TASK 06 • PHASE 9

Automated Browser Testing with Selenium WebDriver & Selenium Grid

Reference Document: CCRP-DOC-06

Field	Value
Document ID	CCRP-DOC-06
Version	1.0
Status	Approved
Owner	Tanishq
Last updated	2026-10-11

Contents

1. Scope and objectives
2. Test strategy (pyramid)
3. Critical user journeys
4. Test cases
5. Test data
6. Framework design
7. Running the suite
8. Failure screenshots
9. Results
10. Defects found while building the suite
11. Entry / exit criteria

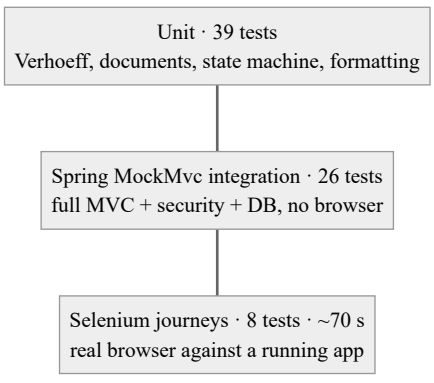
---

1. Scope and objectives

The suite verifies the deployed portal **through a real browser**, as a citizen and an officer use it. It is the release quality gate (Phase 10): a failing journey stops deployment.

In scope	Out of scope
Citizen application, validation, tracking, certificate	Load and performance testing
Officer sign-in, queue, start review, approve, reject	Cross-browser matrix (Chrome only for the MVP)
Access control of the officer area	Visual regression / pixel diffs
Server-side validation as seen by the user	Accessibility audit (manual check only)

2. Test strategy



Level	Tool	Runs in	When
Unit	JUnit 5, AssertJ, Mockito	`surefire` (`./mvnw verify`)	Every build

Level	Tool	Runs in	When
Integration	Spring Boot Test, MockMvc, H2	`surefire`	Every build
End to end	Selenium WebDriver 4.51, Chrome	`failsafe` (`-Pe2e`)	Every pipeline run, before deployment

3. Critical user journeys

ID	Journey	Why it is critical
CJ-1	Citizen applies and receives a tracking ID	The core service. If this breaks, nobody can apply
CJ-2	Invalid data and documents are stopped at the door	Protects officers and data quality; the MVP's main promise
CJ-3	Citizen tracks a request with tracking ID + email	Replaces office visits and phone calls; privacy-sensitive
CJ-4	Officer reviews and approves; citizen gets a certificate	The end-to-end value chain across two roles and two browsers
CJ-5	Officer rejects with a reason the citizen can read	Accountability: no silent rejections
(security)	Officer area closed to anonymous users and wrong passwords	Protects personal data

4. Test cases

TC	Class · method	Journey	Steps (abridged)	Assertions
TC-01a	`CitizenApplicationJourney.citizenAppliesAndSeesSubmittedStatus`	CJ-1	Home → Apply → fill *income* applicant → declare → submit → Track status	Tracking ID matches `CCR-\d{4}-[A-Z2-9]{6}`; type label; status *Submitted*; timeline = [Request submitted]
TC-01b	`…conditionalFieldsFollowTheCertificateType`	CJ-1	Choose Income, then Marriage	Income field shown only for Income; spouse only for Marriage
TC-02a	`ApplicationValidationJourney.invalidApplicationIsRejectedWithFieldMessages`	CJ-2	Submit the *invalid* applicant with a text file renamed `.pdf`	10 fields flagged incl. national-ID checksum, PIN, document type; typed values kept
TC-02b	`…singleInvalidFieldIsPinpointed`	CJ-2	Valid applicant + fake PDF	Exactly one error (`document`), banner says "One thing…"
TC-03	`StatusTrackingJourney.trackingRequiresMatchingEmail`	CJ-3	Apply in browser A; in browser B look up with wrong email, malformed ID, then correct email with different case	Generic not-found message; validation error; result with *Submitted*

TC	Class · method	Journey	Steps (abridged)	Assertions
TC-04	`OfficerApprovalJourney.approvalIssuesACertificate`	CJ-4	Citizen applies; officer (2nd browser) signs in, searches, opens, starts review, approves; citizen refreshes and opens certificate	KPI > 0; citizen sees *Under review* then *Approved*; `SND-INC-YYYY-NNNNNN`; audit trail of 3 entries; certificate shows number, 64-char hash, name, ₹4,50,000
TC-05	`OfficerRejectionJourney.rejectionReasonReachesTheCitizen`	CJ-5	Officer starts review, rejects with "No", then with a full reason	Short reason refused (status unchanged); full reason → *Rejected*; citizen sees the exact reason
TC-06	`AccessControlJourney.officerAreaIsProtected`	security	Open `/officer` anonymously; wrong password; correct password; sign out	Redirect to login; error message; queue shown; `?signedOut`

5. Test data

Source	Content
`src/test/resources/e2e/applicants.csv`	4 valid applicants (one per certificate type) and 1 deliberately invalid one. National IDs carry valid Verhoeff check digits, except the invalid row's
`src/test/resources/e2e/documents/`	`income-proof.pdf` (real PDF), `address-proof.png` (real PNG), `not-really-a-pdf.pdf` (plain text with a `.pdf` name)
Run isolation	Emails get a per-run suffix (`aarav.sharma+e2e-<run>@example.com`), so repeated runs against a shared staging environment never collide

6. Framework design

```
src/test/java/com/sanad/portal/e2e/
├── support/
│   ├── E2EConfig                      system properties: base URL, grid URL, headless, credentials, timeouts
│   ├── DriverFactory                  local Chrome (Selenium Manager) or RemoteWebDriver (Grid) + LocalFileDe
│   ├── BaseJourney                    fresh browser per test, @Tag("e2e"), screenshot extension
│   ├── ScreenshotOnFailureExtension   PNG + page source + URL on failure (before the browser closes)
│   └── TestData                       CSV loader, document resolver, unique emails
├── pages/                             Page Object Model (one class per screen)
│   ├── Page                           explicit waits, data-testid locators, reloadsPage()
│   ├── HomePage · ApplyPage · AcknowledgementPage · TrackPage · CertificatePage
│   └── OfficerLoginPage · OfficerQueuePage · OfficerRequestPage
└── *Journey.java                      the six journey classes (TC-01 … TC-06)
```

Design rules:

- **Stable locators:** every interactive element has a `data-testid`. Tests never depend on CSS classes or text layout.
- **Explicit waits only:** implicit wait is 0. `reloadsPage()` waits for the old `<html>` element to go stale before looking for the next state, which prevents matching the previous page.

- **Independent journeys:** each test opens its own browser and creates its own data, so any order or subset can run.
- **Two-actor journeys** (citizen + officer) use two browsers, as in real life.

7. Running the suite

Mode	Command	What happens
Local (self-contained)	`./mvnw verify -Pe2e`	Unit and integration tests, package, `spring-boot:start` on :18080 with a fresh data dir, 8 journeys in headless Chrome, `spring-boot:stop`, HTML report
Local, visible browser	`./mvnw verify -Pe2e -De2e.headless=false`	Same, but watch Chrome drive the app
Against a running app	`./mvnw verify -Pe2e -De2e.external=true -De2e.baseUrl=http://localhost:8080`	No app start; useful against Tomcat or a container
Through a Selenium Grid (CI)	`… -De2e.remoteUrl=http://ccrp-selenium:4444`	`RemoteWebDriver`; files uploaded via `LocalFileDetector`
Capture every journey	`-De2e.captureAll=true`	Also screenshots passing journeys (`target/selenium-screenshots/passed/`)
One journey	`-Dit.test=OfficerApprovalJourney`	

Reports: `target/failsafe-reports/*.xml` (JUnit XML for Jenkins) and **`target/reports/failsafe.html`** (HTML).

8. Failure screenshots

`ScreenshotOnFailureExtension` runs in JUnit's `AfterTestExecutionCallback`, before `@AfterEach` closes the browser. For every failed test it writes:

```
target/selenium-screenshots/failed/<Class>_<method>_<yyyyMMdd-HHmmss>.png    what the user saw
                                                                  .html   full page source
                                                                  .txt    URL + exception
```

A real example from development, when Chrome's password-breach bubble swallowed clicks (§10), is in `evidence/phase-09-selenium/failure-screenshot-example/`. Jenkins archives this folder on every run (Phase 10).

9. Results

Local run, `./mvnw clean verify -Pe2e -De2e.captureAll=true` (log):

Suite	Tests	Failures	Errors	Time
Unit + integration (surefire)	65	0	0	~20 s
Selenium journeys (failsafe)	8	0	0	66 s

Sanad — Citizen Certificate Request Portal

Last Published: 2026-10-11 | Version: 1.1.0-SNAPSHOT

Failsafe Report

Summary

[Summary] [Package List] [Test Cases]

Tests	Errors	Failures	Skipped	Succes
8	0	0	0	100%

Note: failures are anticipated and checked for with assertions while errors are unanticipated.

Package List

[Summary] [Package List] [Test Cases]

Package	Tests	Errors	Failures	Skipped	
(default package)	8	0	0	0	100%

Note: package statistics are not computed recursively, they only sum up all of its testsuites numbers.

(default package)

-	Class	Tests	Errors	Failures	
	TC-06 Officer access control	1	0	0	
	TC-01 Citizen submits a certificate application	2	0	0	
	TC-05 Officer rejects with a mandatory reason	1	0	0	
	TC-03 Citizen tracks a request	1	0	0	
	TC-02 Server-side validation of data and documents	2	0	0	
	TC-04 Officer approves and the citizen receives a certificate	1	0	0	

Test Cases

[Summary] [Package List] [Test Cases]

TC-06 Officer access control

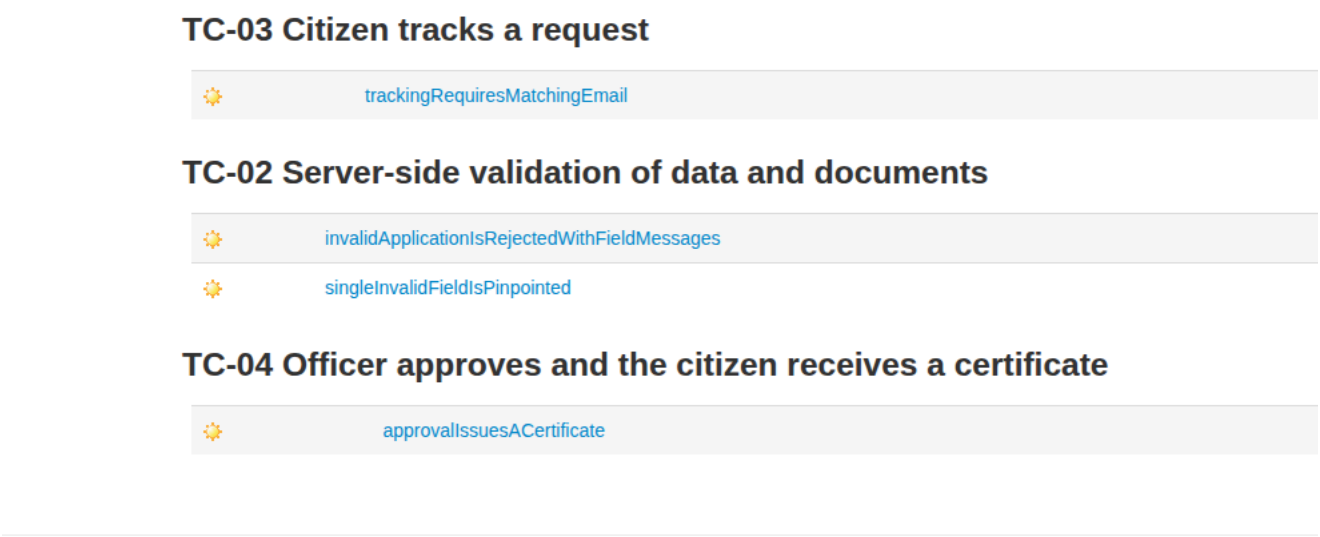
	officerAreaIsProtected

TC-01 Citizen submits a certificate application

	conditionalFieldsFollowTheCertificateType
	citizenAppliesAndSeesSubmittedStatus

TC-05 Officer rejects with a mandatory reason

	rejectionReasonReachesTheCitizen



Screenshots of the final state of every journey are in `evidence/phase-09-selenium/passed-journeys/`.

10. Defects found while building the suite

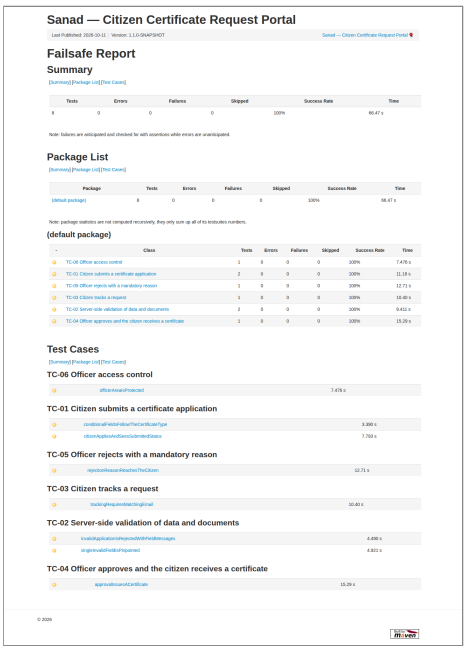
#	Found by	Defect	Fix
1	TC-01/TC-02	**Product bug:** clicking the declaration *text* did nothing. The label had `for="declaration"` but Thymeleaf renders the checkbox as `id="declaration1"`	Label wraps the input without `for`
2	TC-01	`html { scroll-behavior: smooth }` made Selenium click while the page was still scrolling (click intercepted)	Smooth scrolling removed; it added nothing to the UX
3	TC-06	Chrome flagged `Officer@123` as breached and opened a native *Change your password* bubble that swallowed all clicks after sign-in	`DriverFactory` disables password-leak detection and the password manager
4	TC-04/05	Stale elements: waits matched the previous page after a form post	`Page.reloadsPage()` waits for staleness of the old document
5	Debugging	Running e2e against `java -jar target/*.war` while Maven rebuilt that same file broke class loading	Run the app under test from a copy (CI uses a container)

11. Entry / exit criteria

	Criteria
Entry	Build green (unit + integration); app healthy at `e2e.baseUrl`; Chrome or a Grid reachable
Exit (release gate)	**All journeys pass.** Any failure stops the pipeline before deployment (Phase 10). No retries, so flaky tests are fixed rather than hidden

---

Verification Evidence & Screenshots (Task 6)



Sanad — Citizen Certificate Request Portal

Last Published: 2026-10-11 | Version: 1.1.0-SNAPSHOT

Sanad — Citizen Certificate Request Portal

Failsafe Report

Summary

[Summary] [Package List] [Test Cases]

Tests	Errors	Failures	Skipped	Success Rate	Time
8	0	0	0	100%	86.47 s

Note: failures are anticipated and checked for with assertions while errors are unanticipated.

Package List

[Summary] [Package List] [Test Cases]

Package	Tests	Errors	Failures	Skipped	Success Rate	Time
(default package)	8	0	0	0	100%	86.47 s

Note: package statistics are not computed recursively, they only sum up all of its testsuites numbers.

(default package)

Class	Tests	Errors	Failures	Skipped	Success Rate	Time
TC-06 Officer access control	1	0	0	0	100%	7.476 s
TC-01 Citizen submits a certificate application	2	0	0	0	100%	11.18 s
TC-05 Officer rejects with a mandatory reason	1	0	0	0	100%	12.71 s
TC-03 Citizen tracks a request	1	0	0	0	100%	10.40 s
TC-02 Server-side validation of data and documents	2	0	0	0	100%	9.411 s
TC-04 Officer approves and the citizen receives a certificate	1	0	0	0	100%	15.29 s

Test Cases

[Summary] [Package List] [Test Cases]

TC-06 Officer access control

	officerAreaIsProtected	7.476 s

TC-01 Citizen submits a certificate application

	conditionalFieldsFollowTheCertificateType	3.390 s
	citizenAppliesAndSeesSubmittedStatus	7.793 s

TC-05 Officer rejects with a mandatory reason

	rejectionReasonReachesTheCitizen	12.71 s

TC-03 Citizen tracks a request

	trackingRequestMatchingEmail	10.40 s

TC-02 Server-side validation of data and documents

	invalidApplicationIsRejectedWithFieldMessages	4.496 s
	singleInvalidFieldIsFlagged	4.801 s

TC-04 Officer approves and the citizen receives a certificate

	approvalIssuesACertificate	15.29 s

© 2026

maven

Figure 6.1: Maven Failsafe HTML Test Execution Report

TASK 07 • PHASE 10

Continuous Testing & Automated Staging Quality Gate

Reference Document: CCRP-DOC-07

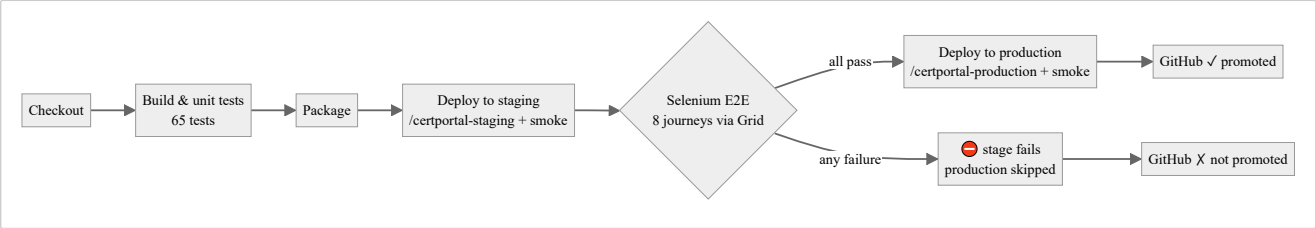
Field	Value
Document ID	CCRP-DOC-07
Version	1.0
Status	Approved
Owner	Tanishq
Last updated	2026-10-11

Contents

1. Pipeline with the quality gate
2. How the gate works
3. Reports published by Jenkins
4. Deliberate defect: timeline
5. Evidence
6. Lessons

---

1. Pipeline with the quality gate



Stage	Purpose	Fails the build when
Build & unit tests	Fast feedback (~30 s)	Any unit or integration test fails
Deploy to staging	Real Tomcat, real configuration	Manager deploy or smoke test fails
Selenium E2E (quality gate)	The 8 browser journeys against staging	**Any journey fails**
Deploy to production	Promotion	Only reached when every earlier stage is green

2. How the gate works

```
./mvnw -B -ntp verify -Pe2e \
    -De2e.external=true \                                  # the app is already deployed on staging
    -De2e.baseUrl="$TOMCAT_URL/certportal-staging" \
    -De2e.remoteUrl="$SELENIUM_URL" \                      # http://ccrp-selenium:4444 (Chrome on the CI netwo
    -Dtest=NoUnitTestsInThisStage -Dsurefire.failIfNoSpecifiedTests=false   # unit tests already ran
```

- `maven-failsafe-plugin:verify` turns a failed journey into a non-zero exit, so the `sh` step fails and so does the stage.
- Declarative Pipeline skips every later stage once a stage fails, so `Deploy to production` is **skipped**.
- The officer password comes from the Jenkins credential `officer-password` (`E2E_OFFICER_PASSWORD`). It is never hard-coded in tests.
- Uploads work across machines: `RemoteWebDriver` + `LocalFileDetector` streams the test documents from the Jenkins workspace to the Chrome node.
- No automatic retries. A flaky journey must be fixed, not re-run until green.

3. Reports published by Jenkins

Report	Where
JUnit results, unit and E2E (trend chart)	*Tests* tab, *Test Result Trend* on the job page
Selenium E2E report (failsafe HTML)	Side menu *Selenium E2E report* (HTML Publisher, kept per build)
Failure screenshots + page source + URL	Build artefacts `target/selenium-screenshots/failed/`
Code coverage (JaCoCo)	*Coverage* (90.5 % lines)
Commit status	GitHub `jenkins/ccrp-pipeline` on every commit

4. Deliberate defect: timeline

Time (UTC)	Event	Production runs
15:47	**#6** first gated run: 65 + 8 tests green → promoted	`e82599b`
15:51	PR #28 (`deliberate-defect`) merged: *Track status* on the acknowledgement page points to `/track` instead of `/track/{id}`. **All 65 unit and integration tests still pass**	`e82599b`
15:52	**#7** polled. Unit stage green → staging deployed → **Selenium gate: 3 errors** (TC-01, TC-04, TC-05: `result-status` never appears) → *Deploy to production skipped* → GitHub ✗	**`e82599b`** (unchanged ✓)
15:56	PR #29: link restored + MockMvc regression check (verified to fail against the defect)	`e82599b`
15:58	**#8** polled rerun: 65 + 8 green → promoted → GitHub ✓	**`bf60d08`**

```
#7 [ERROR]   CitizenApplicationJourney.citizenAppliesAndSeesSubmittedStatus:31 » Timeout … [data-testid='result
#7 [ERROR]   OfficerApprovalJourney.approvalIssuesACertificate:26 » Timeout …
#7 [ERROR]   OfficerRejectionJourney.rejectionReasonReachesTheCitizen:25 » Timeout …
#7 [ERROR] Tests run: 8, Failures: 0, Errors: 3, Skipped: 0
#7 Quality gate FAILED: the release is not promoted.
#7 Stage "Deploy to production" skipped due to earlier failure(s)
#7 Finished: FAILURE
```

The archived failure screenshot shows exactly what a citizen would have seen: the empty lookup form instead of their request.

5. Evidence

All under `evidence/phase-10-quality-gate/`:

File	Shows
`01-build-7-failed-stages.png`	Stage graph: gate red, *Deploy to production* skipped
`02-build-7-test-report.png`, `03-build-7-selenium-html-report.png`	The 3 failing journeys
`jenkins-archived-failure-screenshots/`	Screenshots captured by the suite and archived by Jenkins
`build-7-quality-gate-failed.log`	Full failed console log
`production-before-defect.txt`, `production-after-failed-build.txt`	Production commit unchanged by the failed run
`05-build-8-rerun-stages.png`, `build-8-rerun-success.log`	Successful rerun
`06-pipeline-history-and-trend.png`	Build history and test trend with the dip at #7
`production-after-fix-and-github-statuses.txt`	Production on the fix, GitHub statuses ✗ for #28's merge and ✓ for #29's

6. Lessons

1. **Each test level catches different bugs.** A broken link is invisible to unit and MockMvc tests, which never click. The browser gate caught it before production.
2. **Shift left after a catch.** The fix adds a cheap regression assertion, so the same class of defect now fails in ~30 s instead of ~90 s.

3. **Staging protects production.** The gate needs a real deployment to test, so staging absorbs the risk and production only receives builds that passed.

Verification Evidence & Screenshots (Task 7)

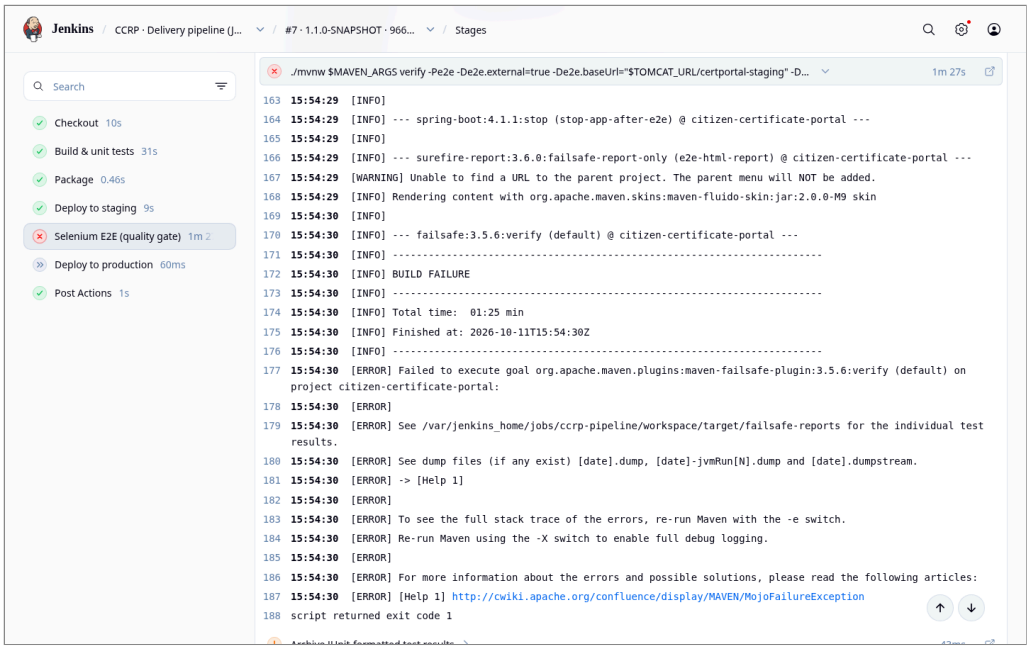


Figure 7.1: Quality Gate Activation - Build #7 Halted on Staging E2E Failure

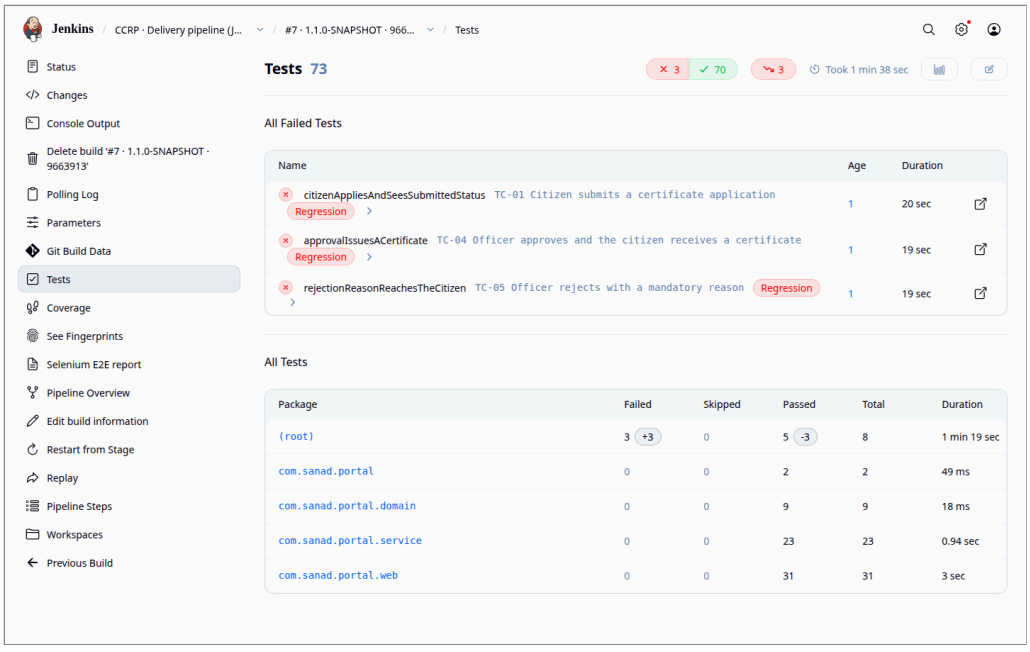


Figure 7.2: Selenium Test Failure Report Identifying Injected Defect

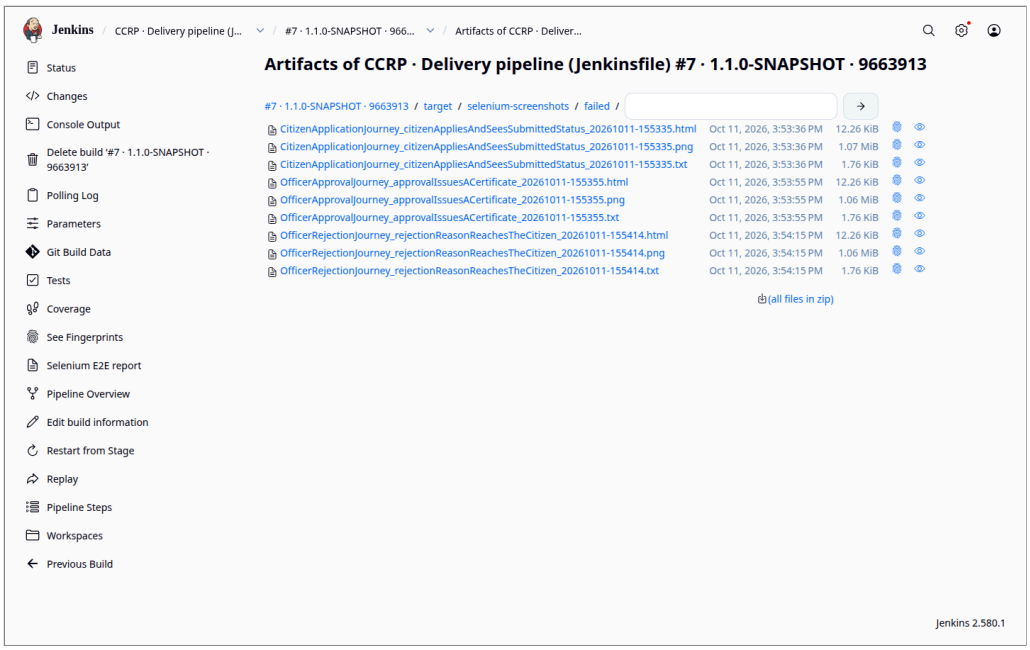


Figure 7.3: Automatically Captured Failure Screenshots in Jenkins

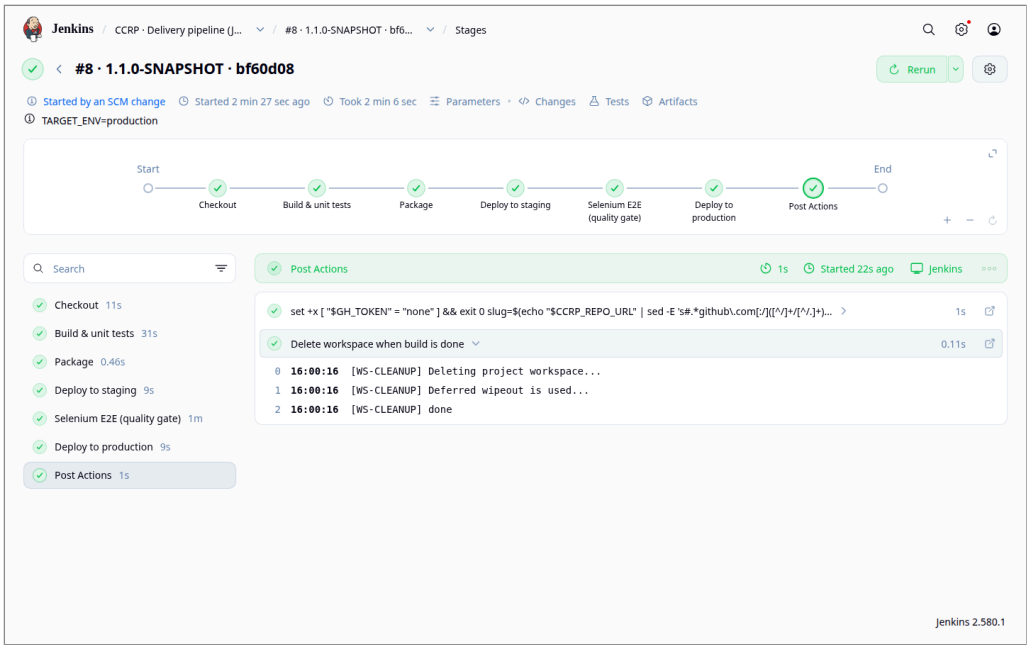


Figure 7.4: Green Quality Gate on Bug Fix - Build #8 Deployed to Production

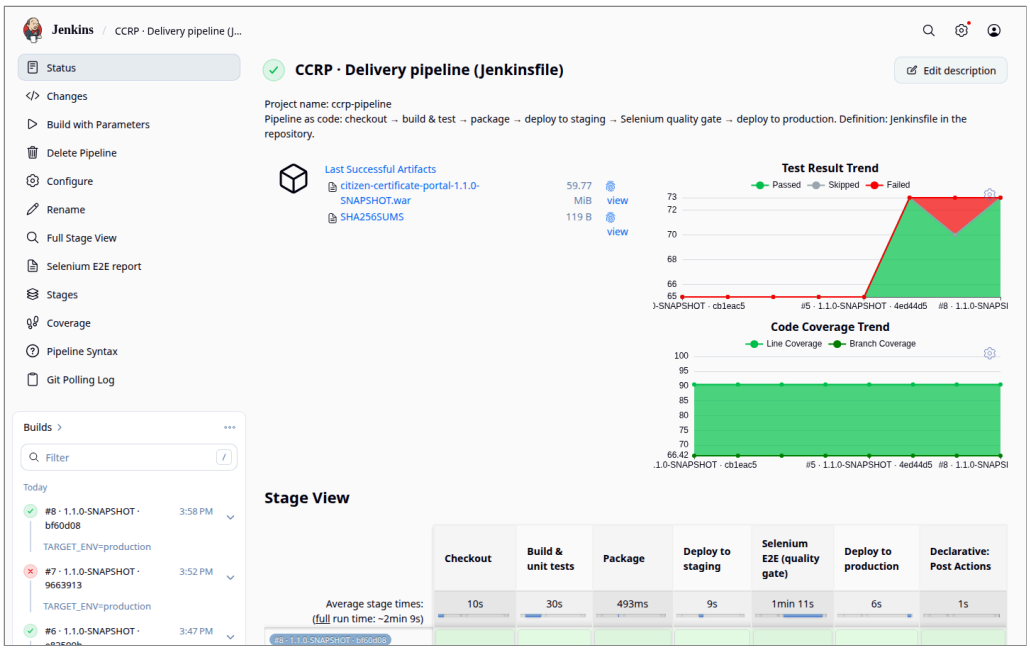


Figure 7.5: Jenkins Pipeline Build History & Test Trend Graph

TASK 08 • PHASES 11 - 12

Containerization with Docker, Private Registry & Continuous Deployment

Reference Document: CCRP-DOC-08

Field	Value
Document ID	CCRP-DOC-08
Version	1.0
Status	Approved
Owner	Tanishq
Last updated	2026-10-11

Contents

1. Image design
2. Image details
3. Container lifecycle (Phase 11)
4. Commit-to-container pipeline (Phase 12)
5. Versioning and the registry
6. Production container deployment and automatic rollback
7. Evidence
8. Operations cheat sheet

---

1. Image design

Decision	Reason
Build once, deploy many: the image packages `target/*.war` instead of compiling inside Docker	The artefact that passed the unit tests is byte-for-byte the one that ships; no second build that could differ
Two stages: `layers` (extract) → runtime	The runtime image contains only the JRE and the application
Spring Boot **layers** (`-Djarmode=tools extract --layers`)	Dependencies (62.5 MB) change rarely; a code change rebuilds only the 590 kB `application` layer
`eclipse-temurin:21.0.12.1_1-jre-noble`, pinned	Reproducible builds; JRE, not JDK
Non-root `ccrp` (uid 10001)	Least privilege. Only `/var/lib/ccrp` is writable
`VOLUME /var/lib/ccrp`	H2 database and uploaded documents outlive containers
`HEALTHCHECK` on `/actuator/health/liveness`	Docker, the pipeline and the deploy script all use the same health signal
Exec-form `ENTRYPOINT`	The JVM is PID 1 and receives SIGTERM, so `docker stop` triggers a graceful shutdown
`-XX:MaxRAMPercentage=75`	Heap follows the container memory limit
OCI labels (`version`, `revision`, `created`, `source`)	Any running container can be traced to its commit
`.dockerignore` allow-list	Only the WAR is sent to the daemon: no `.git`, no secrets, a small context

2. Image details

```
$ docker image inspect getdocs-ccrp:1.2.0-SNAPSHOT
user=ccrp ports={"8080/tcp":{}} volumes={"/var/lib/ccrp":{}}
entrypoint=["java","org.springframework.boot.loader.launch.WarLauncher"]

$ docker history getdocs-ccrp:1.2.0-SNAPSHOT
590kB     COPY /build/extracted/application/ ./
4.1kB     COPY /build/extracted/snapshot-dependencies/
696kB     COPY /build/extracted/spring-boot-loader/ ./
62.5MB    COPY /build/extracted/dependencies/ ./
```

Property	Value
Size	~532 MB (Ubuntu Noble + Temurin JRE 21 ≈ 470 MB, app ≈ 64 MB)
Configuration	Environment variables only: `APP_ENVIRONMENT`, `APP_DATA_DIR`, `APP_OFFICER_PASSWORD`, `SERVER_PORT`, `JAVA_TOOL_OPTIONS`

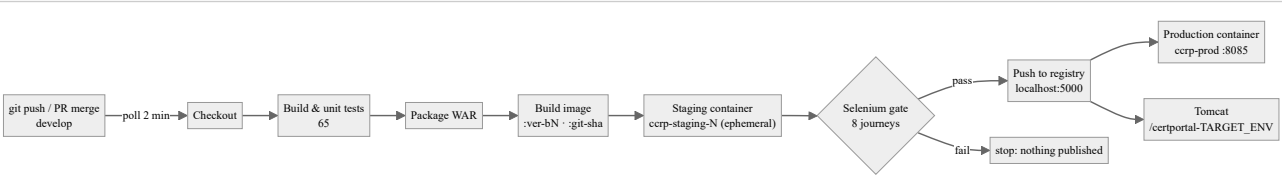
3. Container lifecycle (Phase 11)

`scripts/docker-lifecycle-demo.sh` performs and records every step: docker-lifecycle.log.



Step	Command	Observed
Build	`docker build --build-arg VERSION=… --build-arg REVISION=… -t getdocs-ccrp:1.2.0-SNAPSHOT .`	2-stage build
Tag	`docker tag … getdocs-ccrp:<sha>`, `…:latest`	3 tags, same image ID
Run	`docker run -d --name ccrp-demo -p 8086:8080 -e APP_OFFICER_PASSWORD=… -v ccrp-demo-data:/var/lib/ccrp --restart unless-stopped`	`healthy`
Use	`curl /actuator/health`, submit an application	`UP`, tracking ID issued
Inspect	`docker logs`, `docker exec … id` (uid 10001), `docker stats` (≈ 436 MiB)	
Stop	`docker stop`	`Exited (143)`, log: *Graceful shutdown complete*
Start / restart	`docker start`, `docker restart`	`healthy` again
Remove + recreate	`docker rm -f` → `docker run` with the same volume	**The earlier request is still there**
Clean up	`docker rm -f`, `docker volume rm`, `docker rmi`	

4. Commit-to-container pipeline (Phase 12)



Key properties:

- **The gate tests the image that ships.** Staging is a container started from the freshly built image, and that image is pushed only after the journeys pass.
- Staging containers are per-build (`ccrp-staging-<N>`, label `ccrp.ephemeral=true`) and are always removed in `post`.
- Production container and Tomcat deploy **in parallel**.
- Local images older than the newest six are pruned after each build; the registry keeps every version.

5. Versioning and the registry

Tag	Example	Purpose
`<version>-b<build>`	`1.2.0-SNAPSHOT-b13`	Immutable, traceable to the Jenkins build
`git-<sha>`	`git-249c529`	Traceable to the commit
`latest`		Moving pointer to the last image that passed the gate

```
$ curl -s http://localhost:5000/v2/getdocs-ccrp/tags/list
{"name":"getdocs-ccrp","tags":["1.2.0-SNAPSHOT-b12","1.2.0-SNAPSHOT-b13","git-249c529","latest"]}
```

The registry is `registry:3.1.2` in the CI stack (volume `registry_data`). Docker trusts `localhost` registries without TLS; a shared registry would need TLS and authentication. Docker Hub would need only a `docker login` with a Jenkins credential.

6. Production container deployment and automatic rollback

`scripts/deploy-container.sh <image> [name] [port]`:

1. Records the image of the running `ccrp-prod` (the rollback target).
2. Removes it and starts a **fresh** container from the new image (`--restart unless-stopped`, network `ccrp-ci`, port 8085, volume `ccrp-prod-data`, label `ccrp.deployed-by=<BUILD_TAG>`).
3. Waits for `healthy`. If the container is unhealthy or times out, it prints the logs, **restarts the previous image** and exits 1.

```
==> Deploying localhost:5000/getdocs-ccrp:1.2.0-SNAPSHOT-b13 as ccrp-prod on :8085 (previous: …:1.2.0-SNAPSHOT-
==> Healthy: localhost:5000/getdocs-ccrp:1.2.0-SNAPSHOT-b13 · started 2026-10-11T16:24:…
```

The automatic rollback is demonstrated in Phase 14 (12-container-auto-rollback.log).

7. Evidence

Item	File
Container lifecycle log	`phase-11-docker/docker-lifecycle.log`
Commit-to-container run	`phase-12-container-cd/build-12-commit-to-container.log`
Fresh container replacing the previous one	`build-13-replaces-running-container.log`
Registry catalogue, tags, digest; `ccrp-prod` inspect	`registry-and-production-container.txt`
Pipeline stage graph and stage view	`01-build-13-stages.png`, `02-pipeline-stage-view.png`
Production container serving the portal	`03-production-container-8085.png`

8. Operations cheat sheet

Task	Command
Build locally	`./mvnw -DskipTests package && docker build -t getdocs-ccrp:local .`
Run locally	`docker run -d -p 8086:8080 -e APP_OFFICER_PASSWORD=ChangeMe123 -v ccrp-data:/var/lib/ccrp getdocs-ccrp:local`
What is in production	`docker inspect ccrp-prod --format '{{.Config.Image}} {{index .Config.Labels "org.opencontainers.image.revision"}}'`
Redeploy a known-good image	`OFFICER_PASSWORD=… scripts/deploy-container.sh localhost:5000/getdocs-ccrp:<tag>`
Registry contents	`curl -s localhost:5000/v2/getdocs-ccrp/tags/list`
Logs	`docker logs -f ccrp-prod`

Verification Evidence & Screenshots (Task 8)

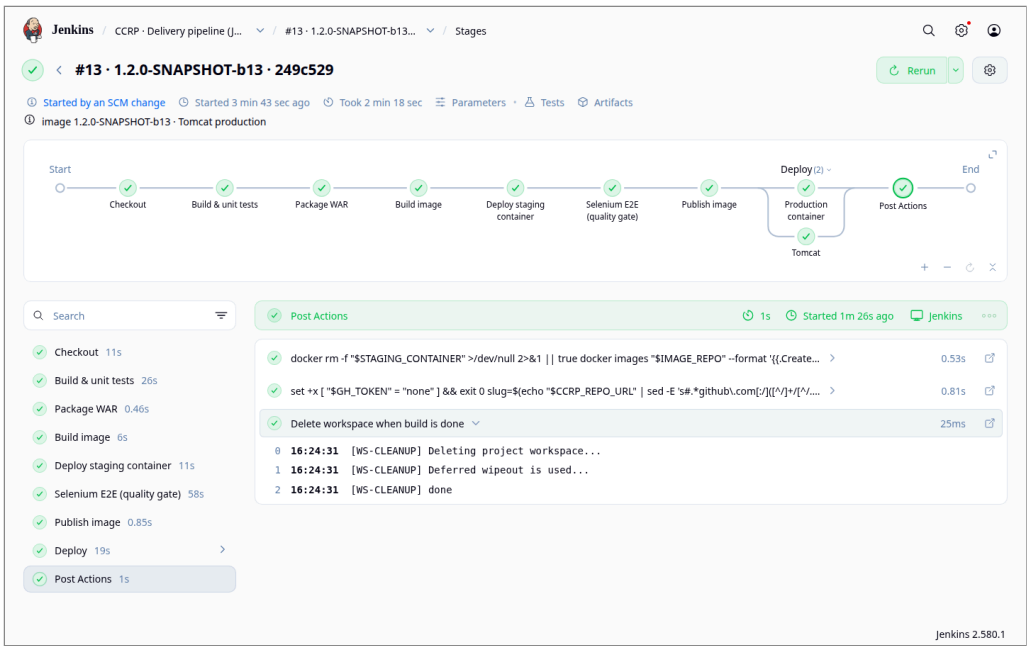


Figure 8.1: Docker Build, Layer Extraction & Registry Push Stages in Jenkins

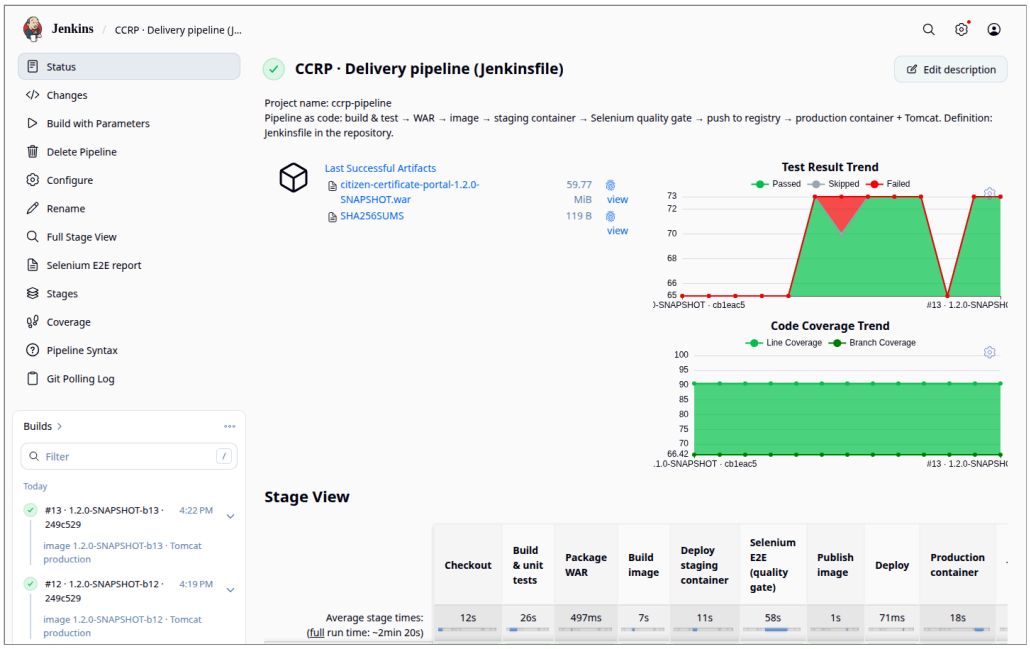


Figure 8.2: End-to-End Container Pipeline Execution Timeline

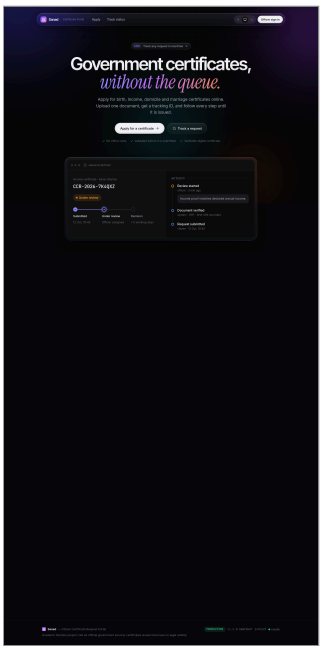


Figure 8.3: Live Production Container Serving on Port 8085 with Active Health Probes

TASK 09 • PHASES 13 - 14

Configuration Management with Ansible, Idempotency & Reliability

Reference Document: CCRP-DOC-09

Field	Value
Document ID	CCRP-DOC-09
Version	1.0
Status	Approved
Owner	Tanishq
Last updated	2026-10-11

Contents

1. Why Ansible
2. Server prerequisites (configuration specification)
3. Project layout
4. Roles and playbooks
5. Release layout and rollback model
6. Secrets
7. The target node
8. Phase 13: first execution
9. Phase 14: provisioning, idempotency, health check, rollback and recovery
10. Commands

---

1. Why Ansible

Criterion	Ansible (chosen)	Puppet
Agent on the node	None, only SSH + Python	Puppet agent and a server/CA
Model	Ordered tasks in YAML, idempotent modules	Declarative catalog, pull model
Fit for a single-team MVP	Fast to adopt, easy to review in a PR	Better for very large fleets

2. Server prerequisites (configuration specification)

Category	Item	Value	Managed by
OS	Distribution	Ubuntu 24.04 LTS, systemd	node image
Packages	Base	`ca-certificates`, `curl`, `jq`, `tzdata`	`common`
	Runtime	`openjdk-21-jre-headless` (Java 21)	`java`
Users / groups	Service account	`ccrp:ccrp`, system user, shell `/usr/sbin/nologin`, home `/opt/ccrp`	`app`
	Automation account	`ansible`, SSH key only, passwordless sudo	node image
Folders	`/opt/ccrp`	root:ccrp 0755, application home	`app`
	`/opt/ccrp/releases/<x.y.z>/`	root 0755, one immutable WAR per release	`app`
	`/opt/ccrp/current`	symlink to the active release	`app`
	`/etc/ccrp`	root:ccrp 0750, configuration	`app`
	`/var/lib/ccrp`	ccrp:ccrp 0750, H2 database + uploaded documents (the only writable path)	`app`
Files	`/etc/ccrp/application.env`	root:ccrp **0640**: environment, data dir, port, officer password, JVM options	`app` (template)

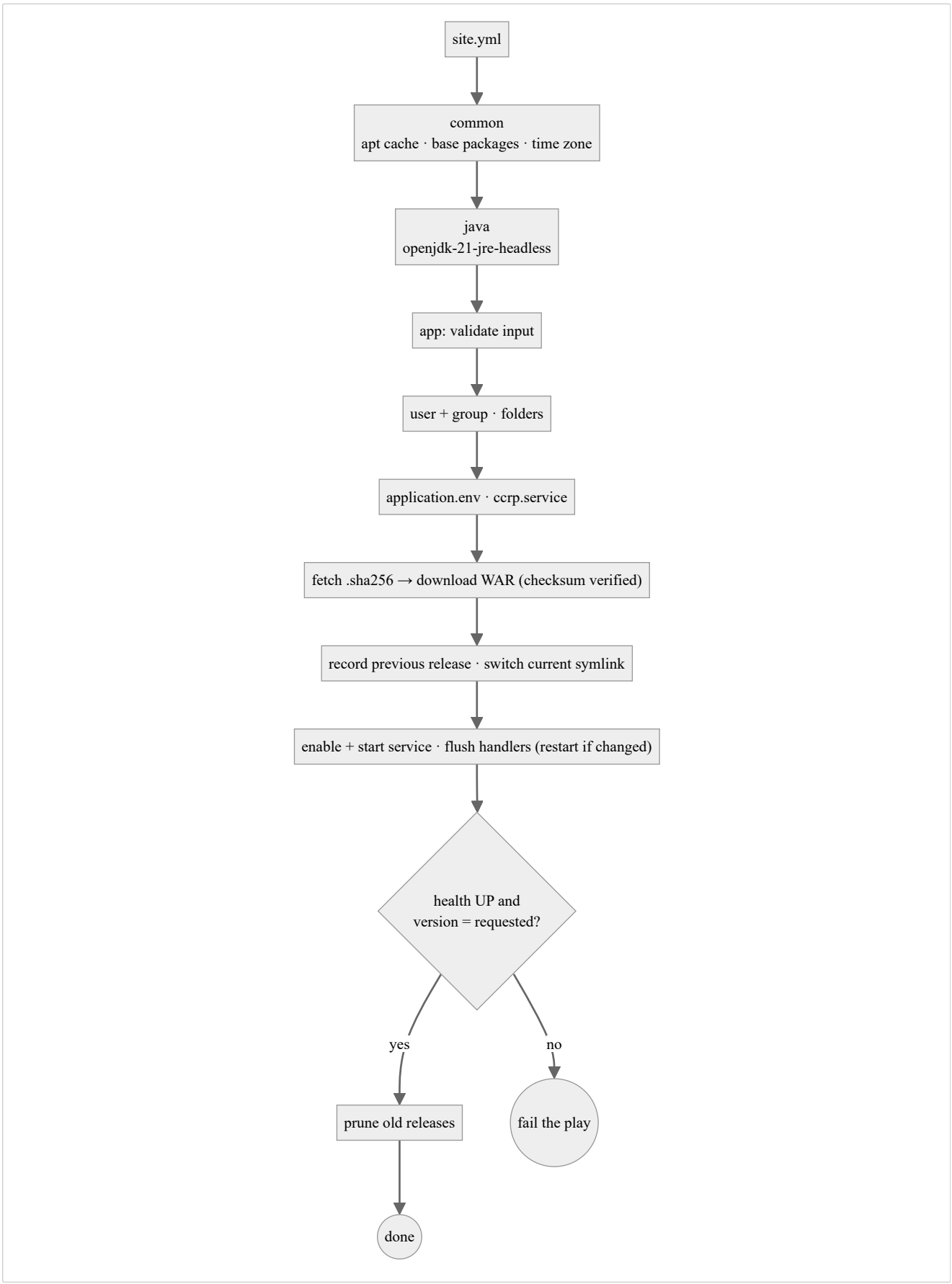
Category	Item	Value	Managed by
	`/etc/systemd/system/ccrp.service`	0644, service unit with hardening	`app` (template)
	`/opt/ccrp/.previous_release`	Version to return to on rollback	`app`
	`/etc/localtime`, `/etc/timezone`	Asia/Kolkata	`common`
Ports	22/tcp	SSH (automation)	node
	8080/tcp	Portal HTTP (published as host :8095 in the lab)	`app`
Services	`ssh.service`	enabled	node image
	`ccrp.service`	enabled, started, `Restart=on-failure`	`app`
Verification	Health	`GET /actuator/health` = `UP`	`app/healthcheck.yml`
	Version	`GET /actuator/info` `build.version` = requested version	`app/healthcheck.yml`

`ccrp.service` hardening: `NoNewPrivileges`, `PrivateTmp`, `ProtectSystem=strict`, `ProtectHome`, `ReadWritePaths=/var/lib/ccrp`. `SuccessExitStatus=143` treats a graceful SIGTERM shutdown as success.

3. Project layout

```
infra/ansible/
├── ansible.cfg                     inventory, roles path, vault password file, SSH pipelining
├── inventory/
│   ├── hosts.ini                   [portal_nodes] ccrp-node-1 (127.0.0.1:2222)
│   └── group_vars/portal_nodes/
│       ├── vars.yml                every tunable (version, paths, packages, port, heap)
│       └── vault.yml               AES-256 encrypted secrets (ansible-vault)
├── site.yml                        provision + deploy (common → java → app)
├── rollback.yml                    return to the previous (or a given) release
├── healthcheck.yml                 read-only status, health and version check
├── roles/
│   ├── common/tasks/main.yml       apt cache, base packages, time zone
│   ├── java/tasks/main.yml         Java 21 runtime + version report
│   └── app/
│       ├── tasks/main.yml          user, folders, config, unit, release, service
│       ├── tasks/healthcheck.yml   health + version assertions (reused by every playbook)
│       ├── tasks/prune.yml         keep the newest N releases
│       ├── templates/              application.env.j2, ccrp.service.j2
│       ├── handlers/main.yml       reload systemd, restart ccrp
│       └── defaults/main.yml
└── node/Dockerfile                 the clean Ubuntu 24.04 + systemd target node
scripts/ansible.sh                  run a playbook with the project config
scripts/ansible-node.sh             create / recreate the clean node
```

4. Roles and playbooks



Every task uses an idempotent module (`apt`, `user`, `file`, `template`, `get_url` with checksum, `systemd_service`). Commands that only read are marked `changed_when: false`. Restarts happen only through handlers, so **a second run changes nothing**.

5. Release layout and rollback model

```
/opt/ccrp/
├── releases/
│   ├── 1.0.0/ccrp.war
│   └── 1.1.0/ccrp.war
├── current -> /opt/ccrp/releases/1.1.0
└── .previous_release            "1.0.0"
```

- Releases are downloaded from GitHub Releases (`v<x.y.z>/citizen-certificate-portal-<x.y.z>.war`) and verified against the published `.sha256` asset.
- Deploying a new version records the old one in `.previous_release`.
- `rollback.yml` points `current` at the previous release, swaps the two values (so a second rollback goes forward again), restarts, and runs the same health and version assertions.
- The data directory is outside the releases, so rollbacks never touch citizen data.

6. Secrets

- `vault.yml` is encrypted with **ansible-vault (AES-256)** and committed. The vault password is in `infra/ansible/.vault-pass`, which is git-ignored and readable only by the owner.
- The task that writes `application.env` sets `diff: false`, so the password never appears in output, even with `--diff`.
- New environment: `ansible-vault create inventory/group_vars/portal_nodes/vault.yml` with `vault_app_officer_password: "…"`.

7. The target node

A real VM would be the production equivalent. For the lab, `scripts/ansible-node.sh` starts a **clean Ubuntu 24.04 container that boots systemd** and contains only `openssh-server`, `python3` and `sudo`: no Java, no service user, nothing from the portal.

Property	Value
SSH	`127.0.0.1:2222`, user `ansible`, key `infra/ansible/.ssh/id_ed25519` (generated, git-ignored)
Portal	http://localhost:8095
Trade-off	Runs `--privileged` so systemd can manage cgroups inside Docker. Acceptable for a disposable lab node; use a VM in real environments

8. Phase 13: first execution

`scripts/ansible.sh site.yml -e app_version=1.0.0` against the clean node: log

```
TASK [app : The running version is the one requested]
ok: [ccrp-node-1] => msg: ccrp-node-1 healthy · version 1.0.0 · commit df0f8cf · environment ansible-node

PLAY RECAP
ccrp-node-1 : ok=27   changed=14   unreachable=0   failed=0   skipped=2
```

9. Phase 14: provisioning, idempotency, health check, rollback and recovery

All logs: `evidence/phase-14-reliability/`.

#	Step	Command	Result
1	**Clean environment**	`scripts/ansible-node.sh --recreate`	Fresh node: `java: not installed`, no `ccrp` user, `/opt` empty, systemd `running` (01)
2	**Provision**	`site.yml -e app_version=1.0.0`	`ok=27 changed=14 failed=0`, healthy 1.0.0 (02)
3	**Idempotency**	same command again	**`changed=0`** (03)
4	Upgrade	`site.yml -e app_version=1.1.0`	`changed=5` (release dir, download, previous marker, symlink, restart), healthy 1.1.0 (04)
5	**Idempotency**	same command again	**`changed=0`** (05)
6	**Health check**	`healthcheck.yml`	`ccrp.service is active (running), enabled`; healthy 1.1.0 (06)

#	Step	Command	Result
7	Data before rollback	`POST /apply` on 1.1.0	`CCR-2026-ZCGFVR` SUBMITTED (07)
8	**Rollback**	`rollback.yml`	`Switch current: 1.1.0 → 1.0.0`, healthy 1.0.0, `changed=3` (08)
9	Data after rollback	`POST /api/v1/status`	Same request still `SUBMITTED`; `.previous_release=1.1.0` (09)
10	**Recovery** from a crash	`kill -9 <java pid>`	systemd `Restart=on-failure`: `NRestarts=1`, **UP again after ~17 s** (10)
11	Final health	`healthcheck.yml`	healthy 1.0.0 (11)
12	**Container rollback**	`deploy-container.sh …:broken-demo`	New container fails (`Failed to load driver class org.postgresql.Driver`), **automatic rollback** to `1.2.0-SNAPSHOT-b14`, production healthy (12)

```
# idempotency, run 2 and run 4
ccrp-node-1 : ok=25  changed=0  unreachable=0  failed=0  skipped=2

# rollback
TASK [Switch current: 1.1.0 → 1.0.0]
ok: [ccrp-node-1] => msg: ccrp-node-1 healthy · version 1.0.0 · commit df0f8cf · environment ansible-node

# crash recovery (journalctl -u ccrp)
ccrp.service: Main process exited, code=killed, status=9/KILL
ccrp.service: Scheduled restart job, restart counter is at 1.
Started ccrp.service - GetDocs — Citizen Certificate Request Portal.
```

Recovery levels covered

Failure	Mechanism	Time to recover
Process crash	systemd `Restart=on-failure` (node), Docker `--restart unless-stopped` (container)	~17 s (JVM start)
Bad release on a node	`rollback.yml`: previous release kept on disk, symlink switch	< 1 min
Bad container image	`deploy-container.sh`: health-gated, automatic rollback to the previous image	~2 min (bounded by the health timeout)
Bad code reaching CI	Selenium quality gate: nothing is published or deployed	n/a (prevented)

10. Commands

Task	Command
Create a clean node	`scripts/ansible-node.sh --recreate`
Connectivity	`cd infra/ansible && ansible portal_nodes -m ping`
Dry run with diff	`scripts/ansible.sh site.yml --check --diff`
Provision / deploy a version	`scripts/ansible.sh site.yml -e app_version=1.1.0`
Only redeploy the app	`scripts/ansible.sh site.yml --tags app -e app_version=1.1.0`
Health and version	`scripts/ansible.sh healthcheck.yml`
Roll back to the previous release	`scripts/ansible.sh rollback.yml`
Roll back to a specific release	`scripts/ansible.sh rollback.yml -e rollback_to=1.0.0`
Service logs on the node	`ssh -p 2222 -i infra/ansible/.ssh/id_ed25519 ansible@127.0.0.1 journalctl -u ccrp -n 50`

TASK 10 • PHASE 15

Final Delivery Report, Multi-Environment Release & Roadmap

Reference Document: CCRP-DOC-10

Field	Value
Document ID	CCRP-DOC-10
Version	1.0
Status	Final
Owner	Tanishq
Release	v1.2.0
Last updated	2026-10-11

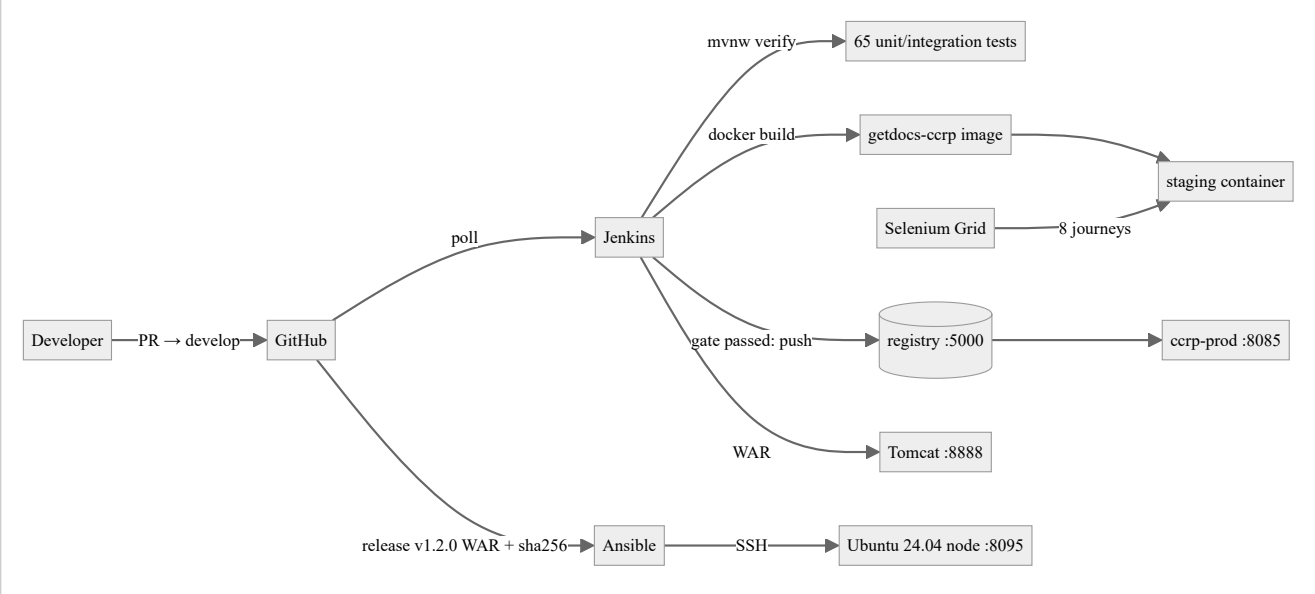
1. Summary

All 15 MVP backlog items (CCR-1 … CCR-15) are delivered across three releases:

Release	Content
v0.1.0	Governance, planning documents, skeleton, design system
v1.0.0	Citizen and officer workflows, certificates, audit trail, JSON API
v1.1.0	Jenkins CI, Jenkinsfile + Tomcat, Selenium suite and quality gate
v1.2.0	Docker image, container CD with registry and rollback, Ansible provisioning

GitHub: 38 pull requests, 15 issues, 5 milestones, protected `main` and `develop`, tags `v0.1.0` – `v1.2.0`.

2. Architecture



3. End-to-end workflow run (v1.2.0)

Step	Tool	Result	Evidence
Commit → PR → merge to `develop`	Git, GitHub	Reviewed PRs, merge commits	docs/04
Build + 65 tests + coverage	Jenkins, Maven	Green, 90.5 % line coverage	phase-12 log
Image → staging container → Selenium gate	Docker, Selenium Grid	8/8 journeys pass	same
Push → production container + Tomcat	Registry, Docker, Tomcat Manager	Healthy, smoke tests pass	registry evidence
Release `v1.2.0`	GitHub Releases	WAR + SHA-256 published	release

Step	Tool	Result	Evidence
Provision / deploy node	Ansible	`failed=0`, *healthy · version 1.2.0*	ansible log

Final environment state (final-environments.txt):

Environment	URL	Version
Production container	http://localhost:8085	1.2.0-SNAPSHOT (latest develop build)
Tomcat production	http://localhost:8888/certportal-production	1.2.0-SNAPSHOT
Ansible node	http://localhost:8095	**1.2.0** (release)

4. Quality results

Check	Result
Unit + integration tests	65 / 65
Selenium journeys	8 / 8 (6 journeys, TC-01 … TC-06)
Quality gate demo	Deliberate defect → build #7 failed, production untouched → fix → #8 green
Idempotency	`changed=0` on re-runs
Rollback	Ansible 1.1.0 → 1.0.0 with data preserved; container auto-rollback on a bad image
Recovery	systemd restarted a killed JVM in ~17 s

5. Troubleshooting guide

Symptom	Cause	Fix
`Cannot create a session after the response has been committed`	Deferred CSRF token on large pages	Already fixed (eager CSRF handler)
Tomcat: `NoClassDefFoundError …/WebServer`	`spring-boot-web-server` in `lib-provided`	Keep the explicit dependency in `pom.xml`
Jenkins job queued but never runs	Built-in node offline: disk < 1 GB	Free disk, then bring the node online (Manage Jenkins → Nodes)
First pipeline run deploys with empty env	Jenkinsfile parameters register after the first run	Parameters also in Job DSL (already done)
Jenkins build "terminated externally"	Controller restarted mid-build	Use *Safe Restart*
Selenium clicks ignored after login	Chrome password-breach bubble	Disabled in `DriverFactory`
Selenium `element click intercepted`	`scroll-behavior: smooth`	Removed
Officer login test fails only in CI	Env var `APP_OFFICER_PASSWORD` overrides test config	Test pins the property
`ccrp-tomcat` unhealthy	`curl -f /` returns 404	Fixed health check
Contexts gone after recreating Tomcat	Writable layer	Named volumes (already done)
Ansible: `requires blocking IO`	Non-interactive shell	`scripts/ansible.sh` redirects stdin
App on node not starting	—	`journalctl -u ccrp -n 100` on the node; `scripts/ansible.sh healthcheck.yml`

6. Limitations

- H2 file database. A single instance; no horizontal scaling.
- One officer account in memory; no roles hierarchy or user management.
- No rate limiting on `/track` (enumeration-safe, but not throttled).
- No email/SMS notifications, payments, e-KYC or DigiLocker integration (out of scope by design).
- Jenkins runs builds on the controller; registry without TLS/auth (lab only).
- The Ansible target is a privileged container, a stand-in for a VM.

- Queue capped at 200 rows (no pagination).

7. Future enhancements

1. PostgreSQL (Flyway migrations are already portable) and multiple app instances behind Nginx.
2. Officer accounts in the database, roles (reviewer/approver), password reset.
3. Notifications (email/SMS) on every status change.
4. Rate limiting (Bucket4j) and CAPTCHA on lookup and apply.
5. Jenkins agents, a TLS registry or Docker Hub, image scanning (Trivy) and dependency scanning as gates.
6. Observability: Prometheus metrics (Micrometer), Grafana, centralised logs.
7. Kubernetes manifests / Helm with rolling updates; Ansible for VM fleets.
8. DigiLocker/e-KYC integration and QR-code certificate verification.